

GANDAKI UNIVERSITY

Faculty of Science and Technology

WEB TECHNOLOGY I

Complete Study Notes • Chapters 1 – 5

Course Code: CIT123

Programme: Bachelor of Information Technology (BIT)

Year: 1 | Semester: II | Credit Hours: 3

Nature of Course: Theory + Practical (TH + PR)

Prepared and Compiled by

Mr. Dammar Khadayat

Lecturer, Department of Information Technology

Gandaki University, Pokhara, Nepal

Academic Session 2026

For internal academic use of BIT students

Table of Contents

This table of contents is a live field. After opening the file in Microsoft Word, press **Ctrl + A** and then **F9** (or right-click the table and choose **Update Field** → **Update entire table**) to fill in the correct page numbers.

Table of Contents	1
Preface and How to Use These Notes	7
Marking Scheme and Question Pattern	7
Course Objectives (from the syllabus)	8
Syllabus Map — What Each Chapter Covers	8
Chapter 0: Complete Development Environment Setup	9
0.1 What Software Do You Actually Need?	9
0.2 Installing Visual Studio Code	9
Installation steps	9
Essential extensions to install	10
Recommended VS Code settings	10
0.3 Installing and Configuring XAMPP	11
Installation steps (Windows)	11
Installation on macOS and Linux	11
Where do I save my files?	12
Verifying that PHP works	12
Troubleshooting the four most common XAMPP problems	12
0.4 Browser Developer Tools — Your Debugging Microscope	13
0.5 Recommended Project Folder Structure	13
0.6 Online Tools You Should Bookmark	14
Chapter 1: Basic Concept	15
Chapter Learning Outcomes	15
1.1 Introduction to Internet, WWW, Web Browser, Web Servers and URL	15
1.1.1 The Internet	15
1.1.2 The World Wide Web (WWW)	16
1.1.3 Client–Server Architecture of the Web	17
1.1.4 Web Browser	18
1.1.5 Web Server	20
1.1.6 URL — Uniform Resource Locator	21

1.2 Overview of Different Protocols: HTTP, HTTPS, POP, SMTP, FTP, WAP	24
1.2.1 What is a Protocol?	24
1.2.2 HTTP — HyperText Transfer Protocol	24
1.2.3 HTTPS — HyperText Transfer Protocol Secure	26
1.2.4 SMTP — Simple Mail Transfer Protocol	28
1.2.5 POP3 — Post Office Protocol version 3	28
1.2.6 FTP — File Transfer Protocol	29
1.2.7 WAP — Wireless Application Protocol	30
1.2.8 Summary Comparison of All Protocols	31
1.3 Domain Name and Hierarchy, Domain Name Registration Process, Web Hosting	32
1.3.1 Domain Name	32
1.3.2 Domain Name Hierarchy	32
1.3.3 How DNS Resolution Works	33
1.3.4 Domain Name Registration Process.....	35
1.3.5 Web Hosting	36
1.4 Exam-Based Questions — Chapter 1.....	38
Chapter 2: HTML and XHTML.....	40
Chapter Learning Outcomes	40
2.1 Structuring Documents for the Web: Introducing HTML and XHTML, Tags and Elements, Basic Syntax	40
2.1.1 What is HTML?	40
2.1.2 Tags, Elements and Attributes	41
2.1.3 Basic Syntax and Rules of HTML	43
2.1.4 Structure of an HTML Document.....	43
2.1.5 Block-level and Inline Elements	44
2.1.6 Semantic Elements in HTML5.....	45
2.1.7 XHTML — Extensible HyperText Markup Language	47
2.2 Basic Text Formatting, Presentational Elements, Phrase Elements.....	49
2.2.1 Headings	49
2.2.2 Paragraphs, Line Breaks and Horizontal Rules.....	49
2.2.3 Character Entities	50
2.2.4 Presentational Elements	50
2.2.5 Phrase (Semantic) Elements.....	51
2.3 Lists, Creating Links, Images, Audio and Video	53
2.3.1 Lists.....	53

2.3.2 Creating Links (Hyperlinks)	55
2.3.3 Images	56
2.3.4 Audio and Video	58
2.4 Tables: Basic Table Elements and Attributes, Spanning Columns and Rows	60
2.4.1 Definition and Purpose.....	60
2.4.2 Basic Table Elements.....	60
2.4.3 Table Attributes.....	62
2.4.4 Spanning Columns with `colspan`	62
2.4.5 Spanning Rows with `rowspan`	63
2.5 Forms and Form Controls	65
2.5.1 Introduction to Forms.....	65
2.5.2 Creating a Form — the ` <form>` Element</form>	66
2.5.3 The ` <input/> ` Element and Its Types	67
2.5.4 Other Form Controls	68
2.5.5 Complete Worked Example — Student Registration Form.....	70
2.5.6 Forms and Accessibility — a Short Checklist	72
2.6 Exam-Based Questions — Chapter 2.....	72
Chapter 3: Introducing Cascading Style Sheets	76
Chapter Learning Outcomes	76
3.1 Introduction, Where You Can Add CSS Rules, Levels of Style Sheets	76
3.1.1 What is CSS?	76
3.1.2 Basic Syntax of CSS	78
3.1.3 Levels of Style Sheets — Where You Can Add CSS	79
3.1.4 CSS Selectors.....	81
3.1.5 The Cascade, Specificity and Inheritance	83
3.2 Style Specification Formats, Style Classes, Properties and Values, Colour, Positioning and Layout	85
3.2.1 Colour in CSS	85
3.2.2 Text and Font Properties	86
3.2.3 The CSS Box Model	87
3.2.4 The `display` Property	89
3.2.5 Positioning	90
3.2.6 Layout with Flexbox	91
3.2.7 Layout with CSS Grid.....	92
3.2.8 Responsive Design and Media Queries.....	93

3.2.9 Transitions, Transforms and Animation	94
3.2.10 Complete Worked Example — A Fully Styled Responsive Page	95
3.3 Exam-Based Questions — Chapter 3.....	98
Chapter 4: Learning JavaScript.....	101
Chapter Learning Outcomes	101
4.1 Introduction to JavaScript	101
4.1.1 Definition	101
4.1.2 Adding JavaScript to a Page	103
4.1.3 Basic Syntax Rules.....	104
4.2 Variables, Data Types and Scope	104
4.2.1 Declaring Variables.....	104
4.2.2 Data Types	106
4.3 Operators.....	108
4.4 Conditional Statements	109
4.5 Looping Statements.....	111
4.6 Functions.....	113
4.6.1 Definition	113
4.7 The Document Object Model (DOM).....	115
4.7.1 Definition	115
4.7.2 Selecting Elements	116
4.7.3 Reading and Changing Content	117
4.8 Events and Event Handling.....	118
4.8.1 Definition	118
4.8.2 The Three Models of Event Handling.....	119
4.8.3 The DOM Event Model — Bubbling and Capturing.....	120
4.9 Changing Colours, Fonts, Content and Element Positioning.....	122
4.9.1 Changing Style with the `style` Property	122
4.9.2 Positioning Elements with JavaScript.....	123
4.10 Form Validation	125
4.10.1 Definition and Importance	125
4.10.2 HTML5 Built-in Validation	126
4.10.3 Complete JavaScript Form Validation	126
4.11 Exam-Based Questions — Chapter 4.....	130
Chapter 5: Programming in PHP and MySQL.....	134
Chapter Learning Outcomes	134

5.1 PHP's Place in the Web World, Basic Rules of PHP Programs, Uses of PHP	134
5.1.1 Client-Side versus Server-Side Programming	134
5.1.2 What is PHP?	135
5.1.3 Basic Rules of PHP Programs.....	136
5.2 Text, Numbers and Variables in PHP	137
5.2.1 Variables and Data Types	137
5.2.2 Working with Strings.....	138
5.2.3 Working with Numbers.....	139
5.2.4 Operators in PHP	139
5.3 Primitives, Operations, Expressions and Control Statements	140
5.3.1 Conditional Statements	140
5.3.2 Looping Statements.....	141
5.3.3 Include and Require — Reusing Code.....	142
5.4 Arrays, Functions and File Handling	142
5.4.1 Arrays.....	142
5.4.2 Functions.....	144
5.5 Handling Form Data in PHP	146
5.5.1 Superglobal Variables	146
5.5.2 Receiving and Validating Form Data.....	146
5.6 File Handling in PHP	149
5.7 Working with Databases: Organizing Data, Connecting, Creating a Table, CRUD.....	151
5.7.1 Organizing Data in a Database.....	151
5.7.2 Connecting PHP to MySQL.....	154
5.7.3 CRUD Operations — the Complete Application.....	155
5.7.4 SQL Injection and Prepared Statements	161
5.8 Cookies and Session Tracking	162
5.8.1 Why State Management is Needed	162
5.8.2 Cookies	163
5.8.3 Sessions.....	164
5.8.4 Putting It All Together — the Complete Application	166
5.9 Exam-Based Questions — Chapter 5.....	167
Appendix A: Quick Reference and Revision Material	171
A.1 Most Frequently Asked Comparison Questions.....	171
A.2 HTML Quick Reference	172
A.3 CSS Quick Reference.....	172

A.4 JavaScript Quick Reference	173
A.5 PHP and MySQL Quick Reference.....	174
A.6 Model Question Paper.....	175
A.7 Glossary of Key Terms	177
A.8 A Suggested Study Plan	178
A.9 Recommended References.....	179

Preface and How to Use These Notes

These notes have been prepared strictly according to the approved syllabus of **Web Technology I (CIT123)**, a three-credit course offered in the second semester of the Bachelor of Information Technology (BIT) programme at Gandaki University. Every unit, sub-unit and even the ordering of topics follows the official course of study, so that nothing you read here is outside the examinable syllabus and nothing in the syllabus is left uncovered.

The course is written for **absolute beginners**. It assumes that you have never written a line of HTML before. Because of that, each chapter follows the same predictable rhythm so that you always know what is coming next:

- **Definition** — a precise, exam-ready one or two sentence statement of what the concept is.
- **Why it matters / Importance** — the reason the concept exists and the problem it solves.
- **Uses / Applications** — where the concept appears in real websites.
- **Syntax** — the exact grammar you must reproduce in the practical examination.
- **Block diagram** — a labelled picture that fixes the idea in your visual memory.
- **Code example** — a complete, runnable program, never a fragment that will not run.
- **Differentiation table** — the comparison questions that examiners love to ask.
- **Exam-based questions** — grouped into 2-mark, 5-mark and 7.5-mark patterns.

Marking Scheme and Question Pattern

The end-semester theory examination for this course carries **60 marks** and the internal assessment carries **40 marks** (assignments, laboratory work, attendance and the mini-project). The theory question paper generally follows this shape:

Group	Type of Question	Marks	Pattern	What the examiner expects
A	Very short answer	2 marks	Attempt all (about 5 questions)	A definition plus one example or one syntax line. Three to five sentences.
B	Short answer	5 marks	Attempt any five or six	Definition, explanation, a small labelled diagram or a short code snippet, and a concluding line.
C	Long answer	7.5 marks	Attempt any two	Full treatment: definition, importance, block diagram, complete working code, comparison table and conclusion.

Exam Tip: For every 7.5-mark answer, always draw at least one labelled diagram and write at least one complete code example. A written answer with a correct diagram consistently scores one to two marks more than a plain paragraph answer of the same length.

Course Objectives (from the syllabus)

This course is designed to serve as an ideal beginner's guide to writing web pages using HTML, using CSS to make pages more appealing, and adding interactivity to pages using JavaScript. After completing the course, students will be familiar with recent technologies such as web technology, client-side programming and server-side programming, and will be able to develop web-based applications.

Syllabus Map — What Each Chapter Covers

Ch.	Unit Title	Hours	Core Skills You Must Master
1	Basic Concept	3	Internet, WWW, browsers, web servers, URL, protocols (HTTP, HTTPS, POP, SMTP, FTP, WAP), DNS hierarchy, domain registration, web hosting
2	HTML and XHTML	8	Document structure, tags and elements, text formatting, phrase elements, lists, links, images, audio and video, tables with colspan and rowspan, forms and every form control
3	Introducing Cascading Style Sheet	8	Three levels of style sheets, selectors, specificity, colour, the box model, :focus and :active pseudo-classes, positioning, floats, Flexbox, Grid and responsive layout
4	Learning JavaScript	10	The DOM, variables and scope, operators, functions, conditionals, loops, events and event handling, DOM event model, element positioning, changing colours and fonts, complete form validation
5	Programming in PHP and MySQL	16	Server-side model, PHP syntax, strings and numbers, control statements, arrays, functions, file handling, MySQL database design, connection, full CRUD, prepared statements, cookies and sessions

Note: Chapters 1 to 5 together account for 45 lecture hours. Chapter 5 alone carries 16 hours, which is more than a third of the course — plan your revision time accordingly.

Chapter 0: Complete Development Environment Setup

Before writing a single tag you must build a working development environment on your own machine. This chapter is a complete, step-by-step installation guide. Follow it once, carefully, and you will be able to complete every laboratory exercise in Chapters 1 to 5 without further installation.

0.1 What Software Do You Actually Need?

Purpose	Software	Version (2026)	Approx. Size	Required?
Writing code	Visual Studio Code	Latest stable	~110 MB	Yes
Viewing and testing pages	Google Chrome (and Mozilla Firefox as second browser)	Latest stable	~120 MB	Yes
Web server + PHP + MySQL	XAMPP (Apache + MariaDB/MySQL + PHP + phpMyAdmin)	PHP 8.2 or later	~150 MB	Yes (from Chapter 5)
Version control (good habit)	Git	Latest stable	~60 MB	Recommended
Image preparation	GIMP or any image editor	Any	~250 MB	Optional
Validating markup	W3C Validator (online, no install)	—	—	Recommended

Note: XAMPP is an acronym: **X** (cross-platform), **A**pache, **M**ariaDB/MySQL, **P**HP, **P**erl. Installing XAMPP installs a real web server, a real database server and the PHP interpreter in one step, which is exactly the stack you need for Chapter 5.

0.2 Installing Visual Studio Code

Visual Studio Code (VS Code) is a free, lightweight code editor from Microsoft. It is the recommended editor for this course because of its built-in emmet abbreviations, integrated terminal and rich extension ecosystem.

Installation steps

- **Step 1:** Open a browser and go to <https://code.visualstudio.com>.
- **Step 2:** The site auto-detects your operating system. Click the large download button (Windows: `.exe` User Installer; macOS: `.zip`; Linux: `.deb` or `.rpm`).
- **Step 3 (Windows):** Run the downloaded installer, accept the licence, and on the *Select Additional Tasks* screen tick "**Add to PATH**", "**Create a desktop icon**", and both "**Add 'Open with Code' action**" options. These make right-click → *Open with Code* available on any folder.
- **Step 4:** Click *Install*, then *Finish*. VS Code opens.
- **Step 5 (Linux):** `sudo dpkg -i code_*.deb` followed by `sudo apt --fix-broken install`.

Essential extensions to install

Open the Extensions panel with **Ctrl + Shift + X** (macOS: **Cmd + Shift + X**), search each name and click ***Install***:

Extension	Publisher	What it does for you
Live Server	Ritwick Dey	Right-click an HTML file → "Open with Live Server". The page opens in the browser and refreshes automatically every time you save. Essential for Chapters 2, 3 and 4.
Prettier – Code formatter	Prettier	Reformats messy HTML/CSS/JS into clean, correctly indented code with Shift + Alt + F. Your teachers can read your submission and so can you.
Auto Rename Tag	Jun Han	When you rename an opening tag, the closing tag renames itself. Removes an entire class of beginner bugs.
HTML CSS Support	ecmel	Suggests your own class and id names inside HTML while you type.
PHP Intelephense	Ben Mewburn	Autocomplete, parameter hints and error detection for PHP. Needed from Chapter 5 onwards.
ESLint	Microsoft	Highlights JavaScript mistakes as you type, before you run the page.
indent-rainbow	oderwat	Colours indentation levels, which makes nested HTML and nested loops far easier to read.

Recommended VS Code settings

Press **Ctrl + Shift + P**, type **Preferences: Open User Settings (JSON)** and paste the following. These settings enforce consistent indentation and automatic formatting.

```
SETTINGS.JSON
{
  "editor.fontSize": 15,
  "editor.tabSize": 2,
  "editor.insertSpaces": true,
  "editor.wordWrap": "on",
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.bracketPairColorization.enabled": true,
  "files.autoSave": "afterDelay",
  "files.autoSaveDelay": 1000,
  "emmet.triggerExpansionOnTab": true,
  "liveServer.settings.donotShowInfoMsg": true
}
```

Exam Tip: Type **!** in an empty **.html** file and press **Tab**. VS Code's Emmet engine expands it into a complete HTML5 skeleton. This alone saves you several minutes in every practical examination.

0.3 Installing and Configuring XAMPP

XAMPP provides the **Apache** web server, the **MySQL/MariaDB** database server, the **PHP** interpreter and the **phpMyAdmin** graphical database manager. You need all four for Chapter 5.

Installation steps (Windows)

- **Step 1:** Go to <https://www.apachefriends.org> and download the XAMPP installer for your operating system. Choose the build that includes **PHP 8.2 or newer**.
- **Step 2:** Right-click the installer and choose ***Run as administrator***. If an antivirus or UAC warning appears, allow it.
- **Step 3:** On the component screen keep **Apache**, **MySQL**, **PHP** and **phpMyAdmin** ticked. You may untick FileZilla, Mercury and Tomcat — they are not used in this course.
- **Step 4:** Install to **C:\xampp**. Do **not** install inside **C:\Program Files**, because Windows file-permission rules there will block Apache from writing files.
- **Step 5:** Finish the installation and tick ***Start the Control Panel now***.
- **Step 6:** In the XAMPP Control Panel, click **Start** next to **Apache** and **Start** next to **MySQL**. Both module names should turn green.
- **Step 7:** Open a browser and visit <http://localhost>. The XAMPP welcome dashboard confirms Apache is running.
- **Step 8:** Visit <http://localhost/phpmyadmin>. The database manager confirms MySQL is running.

Installation on macOS and Linux

- **macOS:** download the **.dmg**, drag XAMPP to ***Applications***, open ***manager-osx***, then start Apache and MySQL from the ***Manage Servers*** tab. The web root is **/Applications/XAMPP/xamppfiles/htdocs**.
- **Linux:** download the **.run** file, then run **chmod +x xampp-linux-*.run** and **sudo ./xampp-linux-*.run**. Start with **sudo /opt/lampp/lampp start**. The web root is **/opt/lampp/htdocs**.

Figure 0.1 How the XAMPP stack serves one page request

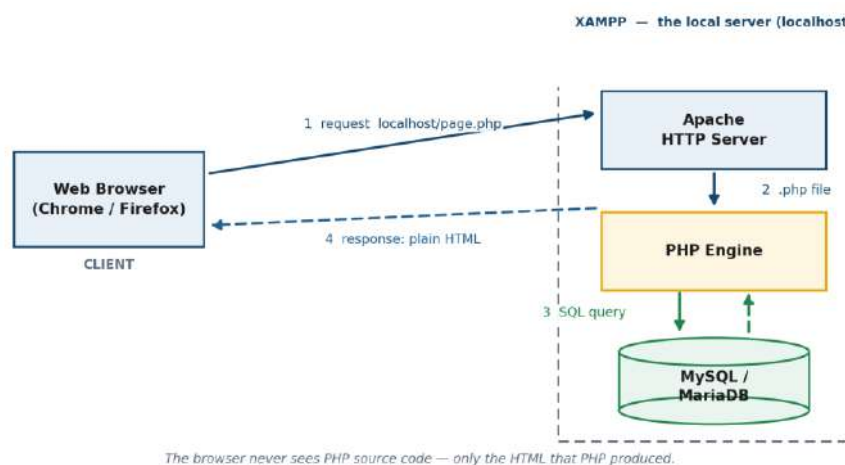


Figure 0.1 — Apache receives the request, PHP executes the script, MySQL supplies the data, and only plain HTML travels back to the browser.

Where do I save my files?

Apache only serves files that live inside its **document root**, called **htdocs**:

Operating System	Document root path	URL to open it
Windows	C:\xampp\htdocs\	http://localhost/
macOS	/Applications/XAMPP/xamppfiles/htdocs/	http://localhost/
Linux	/opt/lampp/htdocs/	http://localhost/

Create one folder per laboratory exercise. A file saved at **C:\xampp\htdocs\lab5\login.php** is opened in the browser as **http://localhost/lab5/login.php**.

Common Mistake: Never open a PHP file by double-clicking it or by dragging it into the browser. That produces the address **file:///C:/...** and the browser will display the raw PHP source code instead of running it. PHP files must always be requested through **http://localhost/**.

Verifying that PHP works

Create the file **C:\xampp\htdocs\test.php** with the following content and open **http://localhost/test.php**.

```
PHP
<?php
// If you see a large purple configuration table, PHP is working.
phpinfo();
?>
```

Troubleshooting the four most common XAMPP problems

Symptom	Cause	Solution
Apache will not start; log says "Port 80 in use"	Skype, IIS, VMware or another web server has taken port 80.	XAMPP Control Panel → Config → httpd.conf. Change Listen 80 to Listen 8080 and ServerName localhost:80 to localhost:8080. Save, restart Apache, and browse to http://localhost:8080/.
MySQL will not start; log says "Port 3306 in use"	A previously installed standalone MySQL service is already running.	Open *Services* (services.msc), stop the MySQL service, set its startup type to Manual, then start MySQL from the XAMPP panel.
Browser shows the PHP source code as text	The file was opened with file:// instead of http://localhost/, or the extension is .html instead of .php.	Save with the .php extension inside htdocs and open through http://localhost/.

Symptom	Cause	Solution
"Access denied for user root@localhost"	A password was set on the root account.	In your connection code use the password you set, or reset it from phpMyAdmin → User accounts. The XAMPP default is user root with an empty password.

0.4 Browser Developer Tools — Your Debugging Microscope

Every modern browser contains a full debugging suite. Open it with **F12** or **Ctrl + Shift + I** (macOS: **Cmd + Option + I**). You will use it in every chapter from 2 onwards.

Panel	Used in Chapter	What you do with it
Elements / Inspector	2 and 3	See the live DOM tree, edit HTML in place, and view every CSS rule that applies to a selected element together with the computed box model.
Console	4	Read JavaScript error messages and the output of <code>console.log()</code> . Also lets you type and run JavaScript against the live page.
Network	1 and 5	Watch each HTTP request: its method, status code, headers, timing and size. This is where the theory of Chapter 1 becomes visible.
Sources / Debugger	4	Set breakpoints and step through JavaScript line by line while inspecting variable values.
Application / Storage	5	Inspect cookies and session identifiers, exactly as discussed in unit 5.6.

0.5 Recommended Project Folder Structure

Use the same tidy structure for every assignment. Marks are frequently awarded for organisation.

FOLDER STRUCTURE

```

htdocs/
my-website/
  index.html      <- home page (always name it index)
  about.html
  contact.php
  css/
    style.css     <- all external style sheets
  js/
    script.js     <- all external JavaScript
  images/
    logo.png
    gallery/
  includes/
    db.php        <- database connection, header, footer
    uploads/      <- files uploaded through forms

```

Note: Name your home page `index.html` (or `index.php`). Apache serves that file automatically when a visitor requests a folder, so `http://localhost/my-website/` will work without typing a file name.

0.6 Online Tools You Should Bookmark

Tool	Address	Purpose
W3C Markup Validation Service	validator.w3.org	Checks that your HTML is valid and standards compliant.
W3C CSS Validation Service	jigsaw.w3.org/css-validator	Checks your CSS for syntax errors and unknown properties.
MDN Web Docs	developer.mozilla.org	The most trustworthy reference for HTML, CSS and JavaScript.
PHP Manual	php.net/manual	The official reference for every PHP function.
Can I Use	caniuse.com	Tells you which browsers support a given feature.
CodePen / JSFiddle	codepen.io	Quick experiments without creating files.

Exam Tip: In the practical examination you will normally have no internet access. Practise writing the HTML skeleton, a CSS rule set, a JavaScript validation function and a PHP database connection **from memory**. These four blocks appear in almost every practical paper.

Chapter 1: Basic Concept

Allotted time: 3 hours

Note: This chapter builds the vocabulary of the whole course. Every later chapter — HTML, CSS, JavaScript and PHP — is an answer to a question raised here: how does a document travel from a machine in another country to the screen in front of you, and who does what along the way? Examiners draw heavily from this chapter for 2-mark and 5-mark questions.

Chapter Learning Outcomes

After completing this chapter you will be able to:

- Define the Internet and the World Wide Web and clearly distinguish between them.
- Explain the client–server architecture of the web and draw its block diagram.
- Describe the internal components of a web browser and the page rendering process.
- Define a web server and compare static and dynamic content delivery.
- Decompose any URL into its component parts.
- Explain the purpose, working and default port of HTTP, HTTPS, POP, SMTP, FTP and WAP.
- Draw the DNS hierarchy and trace the complete name-resolution process.
- Describe the domain registration process and compare the types of web hosting.

1.1 Introduction to Internet, WWW, Web Browser, Web Servers and URL

1.1.1 The Internet

Definition

The **Internet** is a global, publicly accessible network of interconnected computer networks that communicate with one another using a common set of rules known as the **TCP/IP protocol suite**. In simple words, it is a "network of networks" — a physical and logical infrastructure that allows any connected device in the world to exchange data with any other connected device.

Brief history

Year	Milestone	Significance
1969	ARPANET created by the U.S. Department of Defense	The first packet-switched network; four university nodes. Direct ancestor of the Internet.
1972	E-mail invented by Ray Tomlinson	The @ symbol was chosen to separate the user from the host.
1983	TCP/IP adopted as the standard protocol	This is regarded as the official "birthday" of the Internet.
1989–91	Tim Berners-Lee invents the World Wide Web at CERN	HTML, HTTP and the first browser/editor were created here.

Year	Milestone	Significance
1993	Mosaic, the first graphical browser, is released	Images could be displayed inline; public use exploded.
1998	ICANN formed	Central coordination of domain names and IP addresses.
2004 →	Web 2.0, mobile web, cloud, Web 3.0 and AI-driven web	The web becomes read–write, social, mobile-first and programmable.

Key characteristics

- **Decentralised** — no single organisation owns or controls the whole Internet.
- **Packet switched** — data is broken into small packets that travel independently and are reassembled at the destination.
- **Protocol driven** — every device follows the same TCP/IP rules, so different hardware and operating systems can interoperate.
- **Redundant** — if one route fails, packets automatically take another path.
- **Scalable** — new networks can join without the existing network being redesigned.

Uses of the Internet

- Communication: e-mail, instant messaging, voice and video calling.
- Information access: the World Wide Web, digital libraries, online journals.
- E-commerce and digital payments: Daraz, Amazon, eSewa, Khalti.
- E-governance: online licence renewal, tax filing, national ID services.
- E-learning: virtual classrooms, MOOCs, university learning-management systems.
- Entertainment and social networking, cloud storage, remote work, and the Internet of Things.

1.1.2 The World Wide Web (WWW)

Definition

The **World Wide Web** is an information system of interlinked **hypertext documents and other resources**, identified by URLs, transferred over the Internet using the **HTTP/HTTPS** protocol and viewed through a web browser. It was invented by **Sir Tim Berners-Lee** at CERN in 1989.

The Web is therefore a **service that runs on top of the Internet** — one of many. E-mail, FTP file transfer and online gaming are other services that use the same Internet infrastructure but are not part of the Web.

The three foundational technologies of the Web

Technology	Full form	Role
HTML	HyperText Markup Language	The language used to write and structure the documents.

Technology	Full form	Role
URL	Uniform Resource Locator	The addressing scheme that gives every resource a unique address.
HTTP	HyperText Transfer Protocol	The rules by which documents are requested and delivered.

Evolution of the Web

Generation	Period	Nature	Example
Web 1.0	1990–2004	Read-only. Static pages, no user contribution.	Early company brochure sites
Web 2.0	2004–2015	Read–write. User-generated content, social, interactive.	Facebook, YouTube, Wikipedia
Web 3.0	2015 →	Read–write–own. Semantic, decentralised, AI-assisted.	Blockchain apps, semantic search, AI assistants

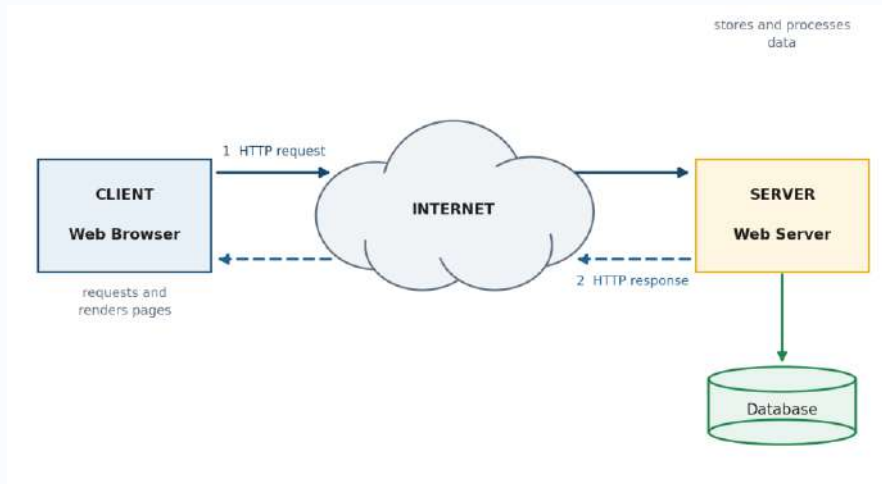
Difference between the Internet and the World Wide Web

Basis	Internet	World Wide Web (WWW)
Meaning	A global network of interconnected computer networks — the physical infrastructure.	A service or application that runs on the Internet — a collection of linked hypertext documents.
Nature	Hardware and infrastructure (cables, routers, satellites, servers).	Software and content (web pages, images, videos, links).
Invented by	Concept from ARPANET, 1969; TCP/IP standard 1983.	Tim Berners-Lee, 1989 at CERN.
Protocol used	TCP/IP suite (many protocols).	HTTP and HTTPS, running on top of TCP/IP.
Identification	Devices are identified by IP addresses.	Resources are identified by URLs.
Dependency	Can exist and function without the Web.	Cannot exist without the Internet.
Scope	Broader — includes e-mail, FTP, VoIP, IoT, online gaming.	A subset — only the documents accessible over HTTP/HTTPS.
Access tool	Requires a network connection and networking software.	Requires a web browser.

Exam Tip: "Differentiate between Internet and WWW" is one of the most frequently repeated 5-mark questions in this course. Memorise at least five points and always begin with the sentence: **the Internet is the infrastructure; the Web is a service running on that infrastructure.**

1.1.3 Client–Server Architecture of the Web

The web works on the **client–server model**. The **client** (your browser) requests a resource; the **server** locates or generates that resource and sends it back. The two communicate over HTTP.

Figure 1.1 Client–server architecture of the World Wide Web*Figure 1.1 — The request–response cycle. Steps 2 and 3 occur only for dynamic pages.****The request–response cycle in words***

- **Step 1:** The user types a URL or clicks a link. The browser resolves the domain name to an IP address using DNS.
- **Step 2:** The browser opens a TCP connection to that IP address on port 80 (HTTP) or 443 (HTTPS) and sends an HTTP request message.
- **Step 3:** The web server receives the request. For a static file it simply reads it from disk; for a dynamic page it passes the request to an interpreter such as PHP, which may query a database.
- **Step 4:** The server returns an HTTP response containing a status code, headers and the body (usually HTML).
- **Step 5:** The browser parses the HTML, requests every linked resource (CSS, JavaScript, images) with further requests, and renders the finished page.

1.1.4 Web Browser***Definition***

A **web browser** is a client-side application program that is used to locate, retrieve, interpret and display resources on the World Wide Web. It sends HTTP requests to web servers, receives the responses, converts HTML, CSS and JavaScript into a visual page, and provides navigation facilities such as back, forward, bookmarks and history.

Popular browsers and their engines

Browser	Developer	Rendering (layout) engine	JavaScript engine
Google Chrome	Google	Blink	V8
Mozilla Firefox	Mozilla Foundation	Gecko	SpiderMonkey

Browser	Developer	Rendering (layout) engine	JavaScript engine
Safari	Apple	WebKit	JavaScriptCore (Nitro)
Microsoft Edge	Microsoft	Blink	V8
Opera	Opera Software	Blink	V8

Internal components (architecture) of a web browser

Figure 1.2 Internal architecture of a web browser

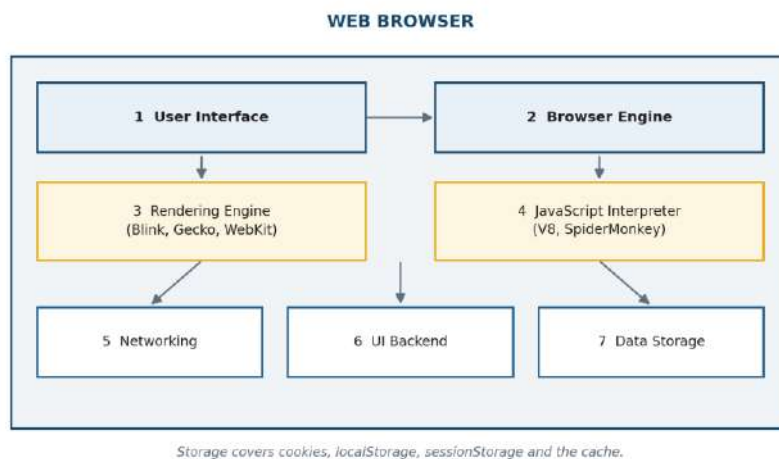


Figure 1.2 — The seven functional components of a modern web browser.

Component	Function
User Interface	Everything the user sees except the page itself: address bar, navigation buttons, bookmark menu, tabs, status bar.
Browser Engine	The coordinator. It receives commands from the UI and instructs the rendering engine to act on them.
Rendering Engine	Parses HTML into the DOM and CSS into the CSSOM, combines them into a render tree, calculates the position and size of every box (layout/reflow) and paints them.
Networking	Performs DNS lookups, opens TCP connections, handles the TLS handshake, and issues HTTP/HTTPS requests. Also manages the browser cache.
JavaScript Interpreter	Compiles and executes JavaScript code, which can modify the DOM and the CSSOM and trigger a re-render.
UI Backend	Draws the basic operating-system widgets such as combo boxes, checkboxes and window controls.
Data Persistence	Stores data on the local machine: cookies, localStorage, sessionStorage, IndexedDB and the cache.

How a browser renders a page (critical rendering path)

Figure 1.3 The critical rendering path

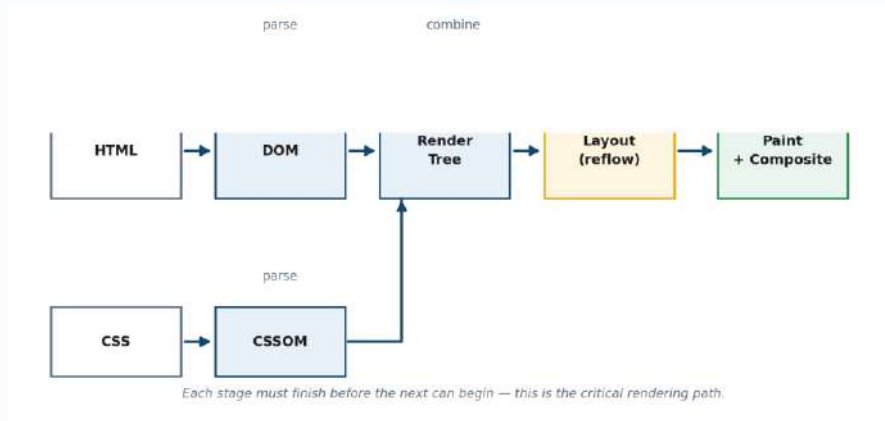


Figure 1.3 — Steps a browser performs between receiving HTML and showing pixels.

1.1.5 Web Server

Definition

A **web server** is a computer program (and by extension the physical machine that runs it) that stores website files and delivers them to clients on request using the HTTP protocol. It listens continuously on a port — normally **port 80** for HTTP and **port 443** for HTTPS — accepts incoming requests, locates or generates the requested resource, and returns an HTTP response.

Functions of a web server

- Listening for and accepting HTTP/HTTPS connections from clients.
- Parsing the request line, headers and body.
- Mapping the requested URL path to a file inside the document root (for example `htdocs`).
- Serving static content directly, or handing dynamic content to an interpreter such as PHP.
- Generating the correct HTTP status code and headers (content type, length, caching, cookies).
- Enforcing security: authentication, access control, TLS encryption, directory protection.
- Logging every request into access and error logs.
- Virtual hosting — serving many different websites from one physical machine.

Popular web server software

Web server	Vendor / Origin	Notable characteristics
Apache HTTP Server	Apache Software Foundation	Free and open source, modular, cross-platform, configured with <code>httpd.conf</code> and <code>.htaccess</code> . Bundled inside XAMPP.

Web server	Vendor / Origin	Notable characteristics
Nginx	Nginx Inc. / F5	Event-driven architecture, very high concurrency, widely used as a reverse proxy and load balancer.
Microsoft IIS	Microsoft	Integrated with Windows Server and the ASP.NET platform.
LiteSpeed	LiteSpeed Technologies	Apache-compatible configuration with higher performance; common in shared hosting.
Node.js (http module)	OpenJS Foundation	The application itself is the server; used for JavaScript back ends.

Static versus dynamic web pages

Basis	Static web page	Dynamic web page
Content	Fixed; identical for every visitor until the file is edited manually.	Generated at request time; can differ for each visitor and each visit.
Technology	HTML, CSS and client-side JavaScript only.	HTML/CSS/JS plus a server-side language (PHP, ASP.NET, JSP, Node.js) and usually a database.
File extension	.html, .htm	.php, .jsp, .asp, .aspx
Server work	The file is read from disk and sent as it is.	A script is executed, a database may be queried, and HTML is produced on the fly.
Speed	Faster — no processing required.	Slower — processing and database access add time.
Cost & complexity	Cheap and simple to host.	Higher cost, requires an application server and database.
Maintenance	Every change needs the source file to be edited.	Content is updated through an admin panel or database, with no code change.
Example	A personal portfolio or a college notice page.	Facebook, an e-commerce site, a student result portal.

Note: A **web server** delivers HTTP content. An **application server** executes business logic, and a **database server** stores data. In XAMPP all three roles run on your single laptop: Apache is the web server, PHP is the application layer and MySQL is the database server.

1.1.6 URL — Uniform Resource Locator

Definition

A **Uniform Resource Locator (URL)** is the unique address of a resource on the World Wide Web. It specifies **where** the resource is located and **which protocol** should be used to retrieve it. A URL is a specific type of **URI** (Uniform Resource Identifier).

Syntax

GENERAL URL SYNTAX

scheme://[user:password@]host[:port]/path[?query][#fragment]

Anatomy of a URL — block diagram

Figure 1.4 The parts of a URL

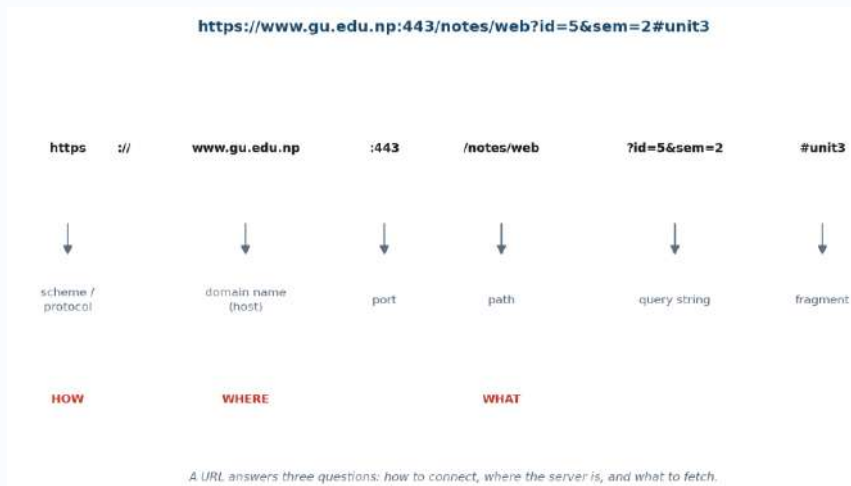


Figure 1.4 — Every component of a URL and its meaning.

Component	Example	Meaning	Compulsory?
Scheme / Protocol	https	The protocol used to fetch the resource (http, https, ftp, mailto, file, tel).	Yes
Sub-domain	www	A named division of the domain, resolved by DNS.	No
Domain name	gandakiuniversity.edu.np	The registered human-readable address that maps to an IP address.	Yes
Port	:443	The TCP port the server listens on. Omitted when default (80 for http, 443 for https).	No
Path	/bit/notes.php	The location of the resource within the server's document root.	No
Query string	?ch=1&sem=2	Data sent to the server as key-value pairs; read in PHP through <code>\$_GET</code> .	No
Fragment	#unit1	An in-page anchor. Processed by the browser only and never sent to the server.	No

Absolute versus relative URL

Basis	Absolute URL	Relative URL
Definition	The complete address including scheme and domain.	An address given relative to the current document's location.
Example	<code>https://example.com/img/logo.png</code>	<code>img/logo.png</code> or <code>../css/style.css</code>
Used for	Linking to a different website or a CDN resource.	Linking between pages of the same website.
Portability	Breaks if the domain changes.	Keeps working when the whole site is moved or renamed.
Length	Longer.	Shorter and easier to maintain.

URL vs URI vs URN

Term	Full form	Meaning	Example
URI	Uniform Resource Identifier	The superset — any string that identifies a resource, by location or by name.	<code>https://a.com/x</code> or <code>isbn:0-486-27557-4</code>
URL	Uniform Resource Locator	A URI that identifies a resource by its location and access method.	<code>https://a.com/x.html</code>
URN	Uniform Resource Name	A URI that identifies a resource by name , independently of location.	<code>urn:isbn:0451450523</code>

Common Mistake: Every URL is a URI, but not every URI is a URL. Write this line in the examination — it is the single sentence that shows you understand the relationship.

URL encoding

URLs may contain only a limited set of ASCII characters. Unsafe characters are replaced by a percent followed by two hexadecimal digits. This is called **percent encoding** or **URL encoding**.

Character	Encoded as	Character	Encoded as
space	<code>%20</code> or <code>+</code>	<code>&</code>	<code>%26</code>
<code>#</code>	<code>%23</code>	<code>=</code>	<code>%3D</code>
<code>?</code>	<code>%3F</code>	<code>/</code>	<code>%2F</code>
<code>%</code>	<code>%25</code>	<code>+</code>	<code>%2B</code>

In PHP the functions `urlencode()` and `urldecode()` perform this conversion; in JavaScript the equivalents are `encodeURIComponent()` and `decodeURIComponent()`.

1.2 Overview of Different Protocols: HTTP, HTTPS, POP, SMTP, FTP, WAP

1.2.1 What is a Protocol?

Definition

A **protocol** is a formally agreed set of rules and conventions that governs how data is formatted, transmitted, received and acknowledged between two devices on a network. Protocols define the **syntax** (format of the message), the **semantics** (meaning of each field) and the **timing** (order and speed of exchange) of communication.

Without protocols, two computers made by different manufacturers and running different operating systems could not understand each other, in exactly the same way that two people who share no common language cannot converse.

Where these protocols sit — the TCP/IP model

Figure 1.5 The four-layer TCP/IP model

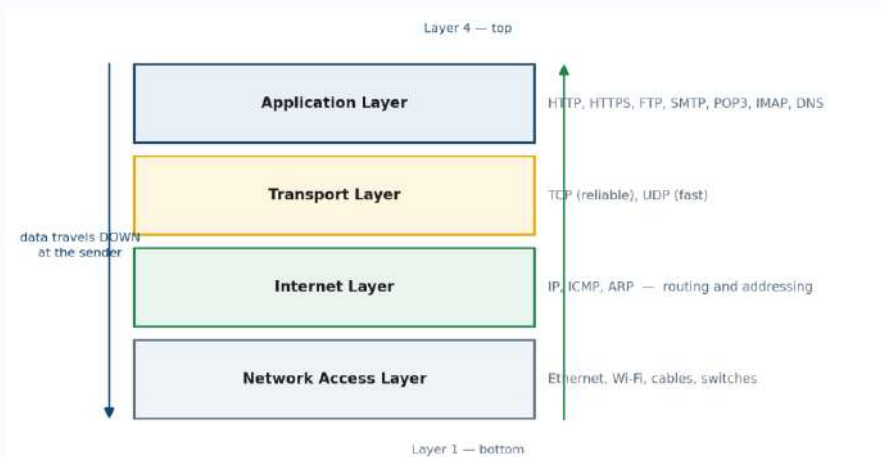


Figure 1.5 — All protocols in this unit belong to the application layer.

1.2.2 HTTP — HyperText Transfer Protocol

Definition

HTTP is a stateless, application-layer, request–response protocol used for transferring hypertext documents and other resources between a web client and a web server on the World Wide Web. It runs over TCP and uses **port 80** by default.

Key characteristics

- **Request–response based** — the client always initiates; the server only replies.
- **Stateless** — the server keeps no memory of previous requests. Each request is independent. (Cookies and sessions, studied in Chapter 5, are added to overcome this limitation.)

- **Connectionless in effect** — HTTP/1.0 closed the connection after each response; HTTP/1.1 introduced persistent connections with **Connection: keep-alive**.
- **Media independent** — any type of data can be transferred provided both sides agree on the MIME type given in the **Content-Type** header.
- **Human readable** — HTTP/1.1 messages are plain text, which makes them easy to inspect in the Network panel of the browser.

Structure of an HTTP request

HTTP REQUEST

```
GET /bit/notes.html HTTP/1.1      <-- request line: method, path, version
Host: www.gandakiuniversity.edu.np <-- header
User-Agent: Mozilla/5.0 (Windows NT 10.0)
Accept: text/html,application/xhtml+xml
Accept-Language: en-US,en;q=0.9
Connection: keep-alive
                                <-- blank line separates head and body
(request body — empty for GET, contains form data for POST)
```

Structure of an HTTP response

HTTP RESPONSE

```
HTTP/1.1 200 OK                  <-- status line: version, code, phrase
Date: Sat, 18 Jul 2026 06:20:11 GMT <-- headers
Server: Apache/2.4.58 (Win64)
Content-Type: text/html; charset=UTF-8
Content-Length: 1274
Set-Cookie: PHPSESSID=a1b2c3d4; Path=/

<!DOCTYPE html>                 <-- response body
<html> ... </html>
```

HTTP request methods

Method	Purpose	Has a body?	Safe?	Idempotent?
GET	Retrieve a resource. Data is appended to the URL as a query string.	No	Yes	Yes
POST	Submit data to be processed (form submission, file upload). Data is in the body.	Yes	No	No
PUT	Create or completely replace a resource at a known URL.	Yes	No	Yes
DELETE	Remove the specified resource.	No	No	Yes
HEAD	Identical to GET but returns only headers, no body. Used to check existence or size.	No	Yes	Yes
PATCH	Apply a partial modification to a resource.	Yes	No	No

Method	Purpose	Has a body?	Safe?	Idempotent?
OPTIONS	Ask the server which methods it supports for a resource.	No	Yes	Yes

HTTP status codes

Class	Meaning	Important examples
1xx	Informational — request received, continuing	100 Continue, 101 Switching Protocols
2xx	Success — the request was received and accepted	200 OK, 201 Created, 204 No Content
3xx	Redirection — further action is required	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Client error — the request was faulty	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found , 405 Method Not Allowed
5xx	Server error — the server failed to fulfil a valid request	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Exam Tip: The four codes most often asked in examinations are **200 OK**, **301 Moved Permanently**, **404 Not Found** and **500 Internal Server Error**. Learn what each one tells the user and the developer.

1.2.3 HTTPS — HyperText Transfer Protocol Secure

Definition

HTTPS is the secure version of HTTP in which the entire HTTP conversation is encrypted using **SSL/TLS (Secure Sockets Layer / Transport Layer Security)**. It runs on **port 443** and provides confidentiality, integrity and authentication.

The three guarantees of HTTPS

- **Confidentiality (encryption)** — anyone intercepting the traffic sees unreadable ciphertext, so passwords and card numbers are protected.
- **Integrity** — a message authentication code detects any modification of the data in transit.
- **Authentication** — a digital certificate issued by a trusted Certificate Authority proves the server really is who it claims to be, preventing man-in-the-middle and phishing attacks.

The TLS handshake (simplified)

Figure 1.6 Simplified TLS handshake used by HTTPS

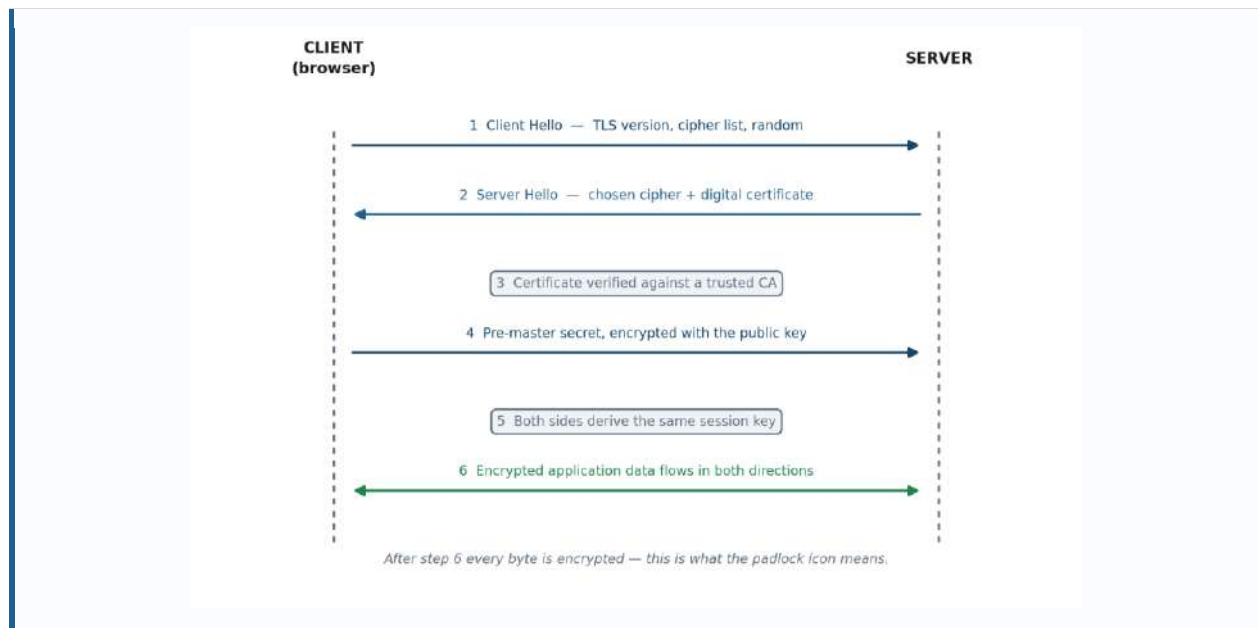


Figure 1.6 — Asymmetric cryptography is used only to agree a session key; the actual data is then encrypted with fast symmetric cryptography.

Difference between HTTP and HTTPS

Basis	HTTP	HTTPS
Full form	HyperText Transfer Protocol	HyperText Transfer Protocol Secure
Security	Data travels as plain text and can be read by anyone on the path.	Data is encrypted with SSL/TLS and is unreadable to an interceptor.
Default port	80	443
Certificate	Not required.	An SSL/TLS certificate from a Certificate Authority is mandatory.
URL prefix	http://	https://
Browser indication	Marked "Not secure" in modern browsers.	Shows a padlock icon in the address bar.
Speed	Marginally faster (no encryption overhead).	Very slightly slower, though HTTP/2 and TLS 1.3 have almost removed the gap.
SEO	Ranked lower by search engines.	Given a ranking preference by Google.
Suitable for	Public, non-sensitive static content.	Login pages, payment gateways, any personal data — today, all websites.

1.2.4 SMTP — Simple Mail Transfer Protocol

Definition

SMTP is the standard application-layer protocol used to **send and relay outgoing electronic mail** from a mail client to a mail server, and between mail servers. It uses **port 25** for server-to-server relay, **port 587** for authenticated client submission, and **port 465** for the older implicit-SSL submission.

- SMTP is a **push** protocol — it pushes messages away from the sender.
- It handles only outgoing mail. Retrieving mail is the job of POP3 or IMAP.
- Commands are simple text keywords: **HELO/EHLO**, **MAIL FROM**, **RCPT TO**, **DATA**, **QUIT**.

1.2.5 POP3 — Post Office Protocol version 3

Definition

POP3 is an application-layer protocol used by an e-mail client to **retrieve (download) messages from a mail server**. By default it downloads the messages to the local device and deletes them from the server. It uses **port 110**, or **port 995** when secured with SSL/TLS.

- POP3 is a **pull** protocol — it pulls messages towards the recipient.
- It is designed for a single device and supports offline reading well.
- **IMAP (Internet Message Access Protocol, port 143 / 993)** is the modern alternative: it keeps mail on the server and synchronises it across many devices.

The complete journey of an e-mail

Figure 1.7 SMTP pushes mail out; POP3/IMAP pulls mail in

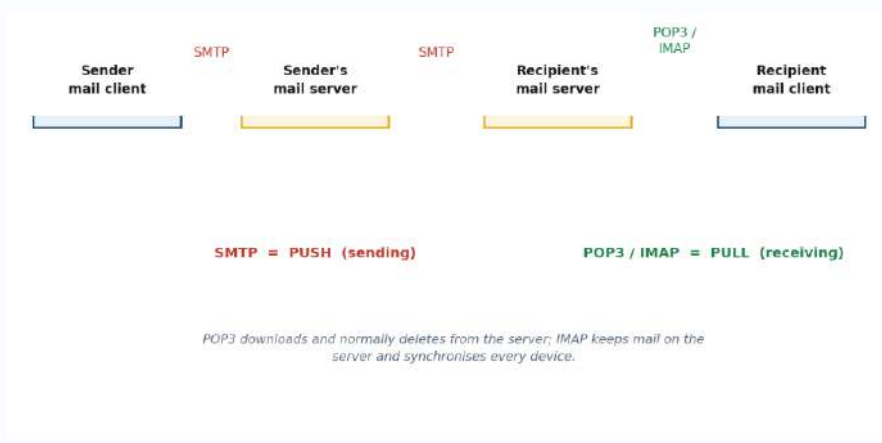


Figure 1.7 — SMTP is used for sending and relaying; POP3 or IMAP is used for retrieval.

Difference between SMTP, POP3 and IMAP

Basis	SMTP	POP3	IMAP
Purpose	Sending and relaying mail	Downloading mail	Accessing and synchronising mail
Direction	Push (outgoing)	Pull (incoming)	Pull (incoming)
Default port	25 / 587 (465 SSL)	110 (995 SSL)	143 (993 SSL)
Copy on server	Not applicable	Deleted after download by default	Retained on the server
Multiple devices	Not applicable	Poor — mail ends up on one device	Excellent — all devices stay in sync
Offline access	Not applicable	Full offline access	Limited unless cached
Storage used	Server (temporarily)	Local device	Server

1.2.6 FTP — File Transfer Protocol***Definition***

FTP is a standard application-layer protocol used to **transfer files between a client and a server** over a TCP/IP network. It uses **two separate connections**: a control connection on **port 21** for commands and replies, and a data connection (port 20 in active mode, or a negotiated port in passive mode) for the actual file content.

Modes of FTP

Mode	How the data connection is opened	Firewall friendliness
Active mode	The client tells the server which port to connect back to; the server initiates the data connection from port 20.	Often blocked by client-side firewalls and NAT.
Passive mode	The server opens a random high port and tells the client to connect to it; the client initiates both connections.	Works through most firewalls; it is the default today.

Uses of FTP

- Uploading a completed website from a developer's computer to a hosting server.
- Downloading large software distributions and archives.
- Bulk transfer and backup of files between servers.
- Anonymous public file repositories (user name **anonymous**).

FTP vs SFTP vs FTPS vs HTTP

Basis	FTP	FTPS	SFTP	HTTP
Full form	File Transfer Protocol	FTP Secure (FTP over SSL/TLS)	SSH File Transfer Protocol	HyperText Transfer Protocol
Encryption	None — credentials travel in plain text	SSL/TLS	SSH	None (HTTPS adds TLS)
Ports	21 (control), 20 (data)	990 / 21	22	80
Connections	Two (control + data)	Two	One	One
Best for	Legacy transfers on trusted networks	Secure transfer keeping FTP commands	Secure transfer, most recommended today	Serving web pages

A popular free FTP client is **FileZilla**. Typical connection details are: host `ftp.example.com`, user name, password, port 21, and passive transfer mode.

1.2.7 WAP — Wireless Application Protocol**Definition**

WAP is an open, international technical standard (specified by the WAP Forum, later the Open Mobile Alliance) that defines how information should be accessed over a **wireless network by mobile devices** with small screens, limited memory, low processing power and low bandwidth. WAP pages were written in **WML (Wireless Markup Language)**, an XML-based language, and scripted using **WMLScript**.

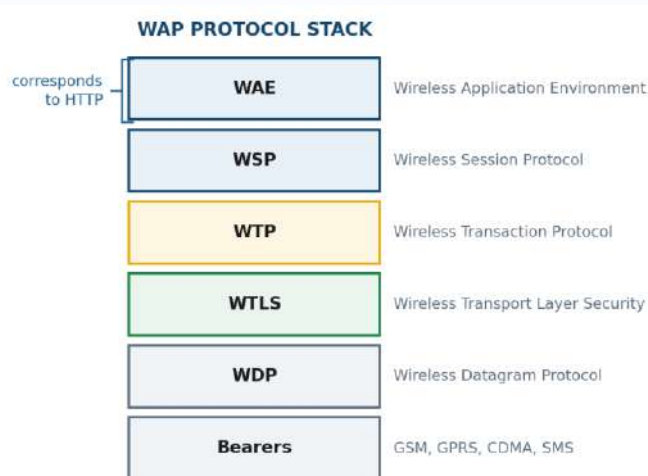
The WAP protocol stack**Figure 1.8 The WAP protocol stack**

Figure 1.8 — Each WAP layer mirrors an equivalent layer of the ordinary Internet stack.

WAP architecture

Figure 1.9 WAP architecture with the WAP gateway

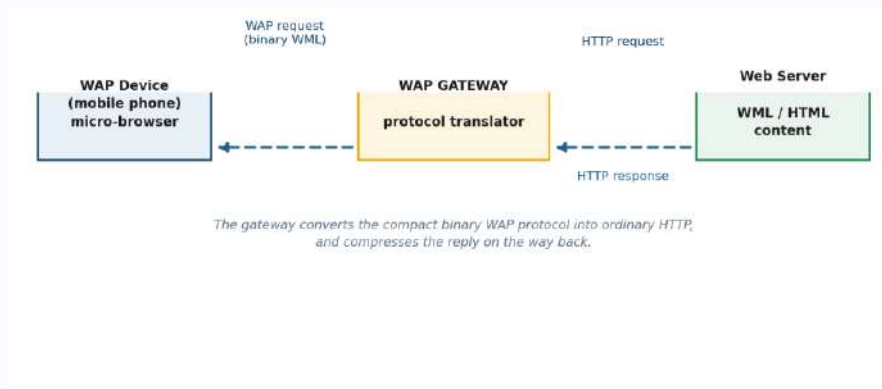


Figure 1.9 — The WAP gateway is the translator between the mobile network and the Internet.

Current status of WAP

WAP was important between roughly 1999 and 2008. It has now been made obsolete by fast mobile data (3G, 4G, 5G) and by smartphones whose browsers render ordinary HTML5 pages. The modern replacement for WAP's goal — good mobile experience — is **responsive web design using CSS media queries**, which you will study in Chapter 3.

1.2.8 Summary Comparison of All Protocols

Protocol	Full form	Main purpose	Default port	Transport
HTTP	HyperText Transfer Protocol	Transfer web pages, unencrypted	80	TCP
HTTPS	HTTP Secure	Transfer web pages, encrypted with TLS	443	TCP
SMTP	Simple Mail Transfer Protocol	Send / relay outgoing e-mail	25, 587, 465	TCP
POP3	Post Office Protocol v3	Download incoming e-mail	110, 995	TCP
IMAP	Internet Message Access Protocol	Access and sync e-mail on server	143, 993	TCP
FTP	File Transfer Protocol	Transfer files, two connections	21 and 20	TCP
SFTP	SSH File Transfer Protocol	Secure file transfer	22	TCP
DNS	Domain Name System	Resolve domain names to IP addresses	53	UDP (TCP for zone transfer)
WAP	Wireless Application Protocol	Web access from limited mobile devices	9200–9203	WDP over bearer

1.3 Domain Name and Hierarchy, Domain Name Registration Process, Web Hosting

1.3.1 Domain Name

Definition

A **domain name** is a human-readable, easy-to-remember text address that uniquely identifies a location on the Internet and maps to a numeric **IP address**. Computers communicate using IP addresses such as `142.250.183.14`; people prefer names such as `google.com`. The **Domain Name System (DNS)** performs the translation between the two.

Why domain names are needed

- IP addresses are difficult for humans to remember, especially IPv6 addresses such as `2404:6800:4007:80e::200e`.
- A website can change its hosting server and IP address while keeping the same domain name, so links and bookmarks do not break.
- Domain names carry branding value and communicate the nature of an organisation.
- One IP address can host many domain names through virtual hosting.

DNS — Domain Name System

DNS is a hierarchical and distributed naming database that stores the mapping between domain names and IP addresses, and answers queries from clients. It is often called the "phone book of the Internet". It normally uses **UDP on port 53**.

1.3.2 Domain Name Hierarchy

The DNS namespace is an inverted tree. Reading a domain name from **right to left** takes you from the most general level to the most specific.

Figure 1.10 The hierarchical structure of the Domain Name System

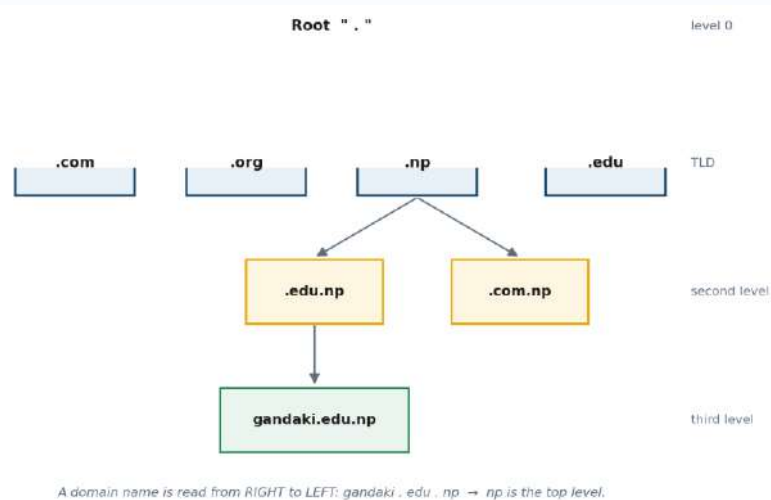


Figure 1.10 — DNS is read from right to left, from the root down to the host.

Level	Name	Description	Examples
0	Root	The unnamed top of the tree, written as a trailing dot. Served by 13 logical root server clusters worldwide.	. (usually invisible)
1	Top-Level Domain (TLD)	The right-most label. Divided into generic (gTLD), country-code (ccTLD) and sponsored TLDs.	.com, .org, .net, .np, .in
2	Second-Level Domain (SLD)	The name an organisation actually registers and pays for.	google in google.com
3	Sub-domain / Third level	A subdivision created freely by the domain owner; needs no registration.	www, mail, exam, blog
4	Host name	The name of a specific machine or service.	server1.exam.gu.edu.np

Types of Top-Level Domain

Type	Meaning	Examples
gTLD — generic	Open, general-purpose domains available to anyone.	.com (commercial), .net (network), .org (organisation), .info, .biz, .xyz
sTLD — sponsored	Restricted; the registrant must satisfy eligibility rules set by a sponsor.	.edu (education), .gov (US government), .mil (military), .int, .museum
ccTLD — country code	Two-letter codes assigned to countries and territories under ISO 3166.	.np (Nepal), .in (India), .uk, .cn, .jp
New gTLD	Descriptive extensions introduced from 2013 onwards.	.app, .dev, .tech, .shop, .online, .ai

Note: In Nepal, the .np ccTLD is administered by **Mercantile Communications Pvt. Ltd.**, which has historically offered .com.np, .org.np, .edu.np and .gov.np registrations free of charge to eligible Nepali applicants, subject to submission of supporting documents.

1.3.3 How DNS Resolution Works

When you type **www.example.com** into the browser, the following recursive lookup takes place, usually in under 100 milliseconds:

Figure 1.11 Step-by-step DNS resolution

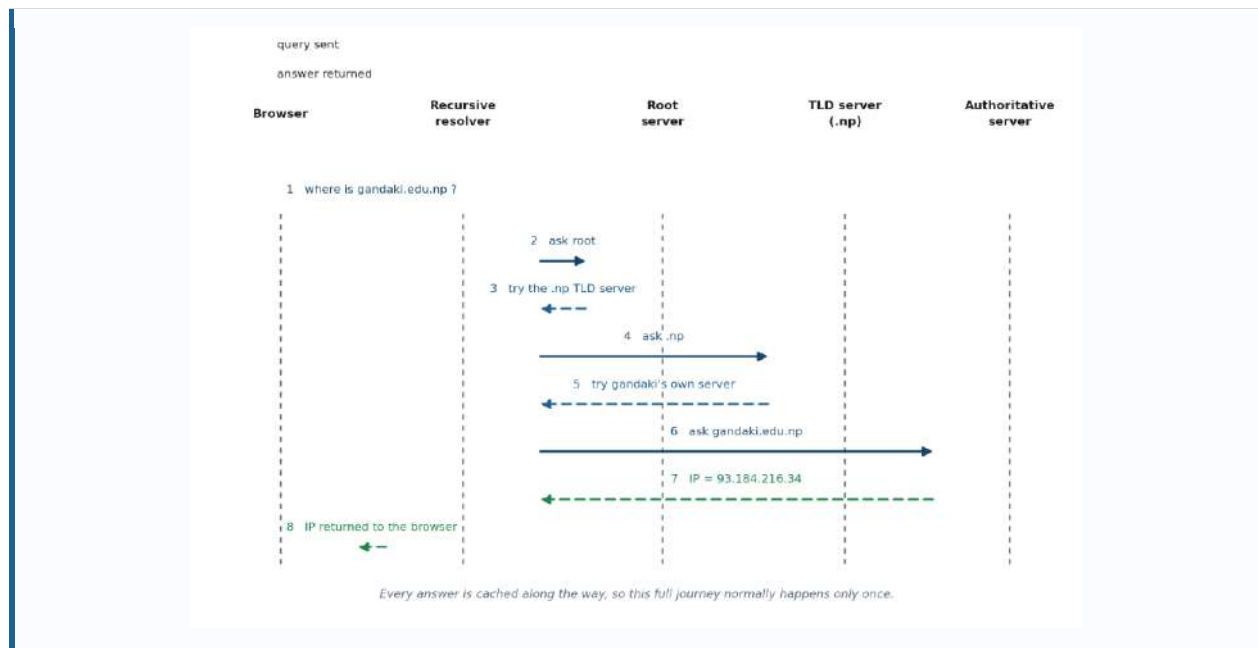


Figure 1.11 — The recursive resolver does the work; the client asks only once.

Common DNS record types

Record	Stands for	Purpose	Example value
A	Address	Maps a domain name to an IPv4 address.	93.184.216.34
AAAA	Quad-A	Maps a domain name to an IPv6 address.	2606:2800:220:1::
CNAME	Canonical Name	Makes one name an alias of another name.	www → example.com
MX	Mail Exchange	Names the mail servers that accept e-mail for the domain.	mail.example.com (priority 10)
NS	Name Server	Names the authoritative name servers for the zone.	ns1.hostgator.com
TXT	Text	Arbitrary text; used for SPF, DKIM and ownership verification.	v=spf1 include:_spf...
SOA	Start of Authority	Administrative information about the zone.	serial, refresh, retry, TTL

Exam Tip: You can perform a DNS lookup yourself. Open Command Prompt and type `nslookup gandakiuniversity.edu.np`, or on Linux/macOS `dig gandakiuniversity.edu.np`. Use `ipconfig /flushdns` on Windows to clear the local DNS cache.

1.3.4 Domain Name Registration Process

The parties involved

Party	Role
ICANN	The Internet Corporation for Assigned Names and Numbers — the global non-profit body that coordinates the DNS root, accredits registrars and sets policy.
Registry	The organisation that operates a particular TLD and maintains its master database (for example Verisign for .com , Mercantile for .np).
Registrar	An ICANN-accredited company that sells domain names to the public and submits registrations to the registry (GoDaddy, Namecheap, Google Domains, local Nepali providers).
Registrant	You — the individual or organisation that registers and holds the right to use the name.
WHOIS	The public directory that records who registered a domain, when it was registered, and when it expires.

Step-by-step registration procedure

- **Step 1 — Choose a name.** Keep it short, memorable, easy to spell, related to your brand, and free of hyphens and digits where possible. Check that it does not infringe a trademark.
- **Step 2 — Choose the extension (TLD).** **.com** for a business, **.org** for a non-profit, **.edu.np** for a Nepali educational institution, **.com.np** for a Nepali company.
- **Step 3 — Check availability.** Use a registrar's search box or a WHOIS lookup. If the name is taken, try a variation or a different TLD.
- **Step 4 — Select a registrar** and add the domain to the cart. Compare the first-year price with the renewal price, which is often much higher.
- **Step 5 — Choose the registration period**, normally 1 to 10 years, and decide whether to enable auto-renewal.
- **Step 6 — Supply registrant details:** full name, organisation, postal address, e-mail and telephone. These go into the WHOIS database. For **.np** domains you must additionally submit supporting documents such as a company registration certificate or a letter from the institution.
- **Step 7 — Add WHOIS privacy protection** if offered, so that your personal contact details are hidden from public lookups.
- **Step 8 — Make payment** by card, digital wallet or bank transfer.
- **Step 9 — Verify your e-mail address.** ICANN requires the registrant e-mail to be confirmed within 15 days, otherwise the domain is suspended.
- **Step 10 — Point the domain at your hosting** by setting the name-server (NS) records supplied by your hosting provider, or by creating A records that point to the server IP address. Propagation across the global DNS takes anywhere from a few minutes to 48 hours.
- **Step 11 — Renew before expiry.** A lapsed domain enters a redemption period with a heavy restoration fee, and may then be released for anyone to buy.

Figure 1.12 Domain name registration workflow

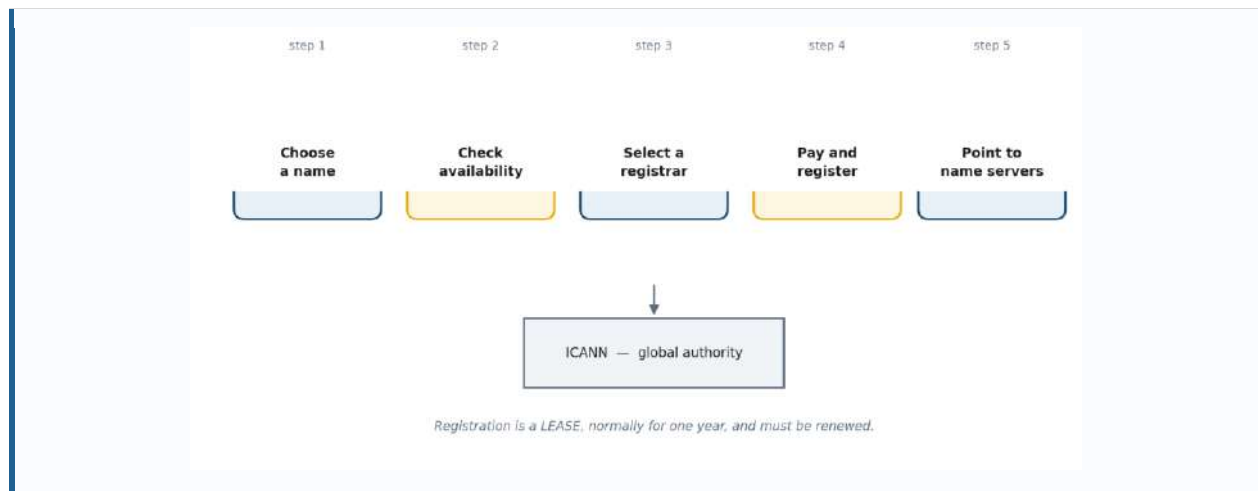


Figure 1.12 — The registration lifecycle from choosing a name to renewal.

Note: Registering a domain does **not** give you a website. A domain name is only an address. You also need **web hosting** — actual disk space on a server that is permanently connected to the Internet.

1.3.5 Web Hosting

Definition

Web hosting is a service that provides storage space, bandwidth and server resources on a computer that is permanently connected to the Internet, so that the files of a website can be stored there and served to visitors on request. The company that provides this service is called a **web hosting provider** or **host**.

What a hosting package normally includes

- Disk space (SSD storage) for HTML, CSS, JavaScript, PHP files, images and the database.
- Monthly bandwidth or data-transfer allowance.
- A web server (Apache, Nginx or LiteSpeed) and a language runtime such as PHP.
- One or more MySQL/MariaDB databases with phpMyAdmin access.
- A control panel — typically cPanel or Plesk — for file management, e-mail and backups.
- E-mail accounts on your own domain, FTP/SFTP accounts, and a free SSL certificate (Let's Encrypt).
- Uptime guarantee, technical support and automatic backups.

Types of web hosting

Type	How it works	Advantages	Disadvantages	Best for
Shared hosting	Hundreds of websites share one physical server and its resources.	Cheapest; no technical skill needed; managed by the provider.	Slow under load; "bad neighbour" effect; very limited control.	Student projects, blogs, small business sites.

Type	How it works	Advantages	Disadvantages	Best for
VPS hosting	One physical server is divided into isolated virtual machines with guaranteed resources.	Dedicated RAM and CPU; root access; good value.	Costs more; needs basic server administration knowledge.	Growing sites, small applications.
Dedicated hosting	An entire physical server is rented by one customer.	Maximum performance, full control and security.	Most expensive; you manage everything.	Large e-commerce, high-traffic portals.
Cloud hosting	The site runs on a cluster of servers; resources scale on demand.	Highly scalable and fault tolerant; pay for what you use.	Variable monthly cost; more complex to configure.	Applications with unpredictable traffic.
Managed WordPress	Cloud or shared hosting tuned specifically for WordPress.	Automatic updates, caching and security.	Only for WordPress; limited plugin freedom.	Content-driven sites.
Free / self hosting	Free subdomain services, or running a server on your own premises.	No cost, or complete control.	Adverts, no support, unreliable; self-hosting needs a static IP and constant power.	Learning and experimentation only.

How to choose a hosting provider — evaluation checklist

- **Uptime guarantee** — look for 99.9 % or better with a service credit policy.
- **Server location** — a server nearer your audience gives lower latency; for Nepali users, servers in Singapore or India are usually fastest.
- **Storage and bandwidth** — check whether "unlimited" is subject to a fair-use policy.
- **Supported technology** — the PHP version, MySQL version, number of databases, and whether SSH is allowed.
- **Free SSL certificate**, daily backups and one-click restore.
- **Support quality** — 24×7 live chat and ticketing, with real response times.
- **Renewal price**, not just the promotional first-year price.
- **Migration policy and refund window** in case you need to leave.

Deploying a website — the complete picture

Figure 1.13 From localhost to a live website

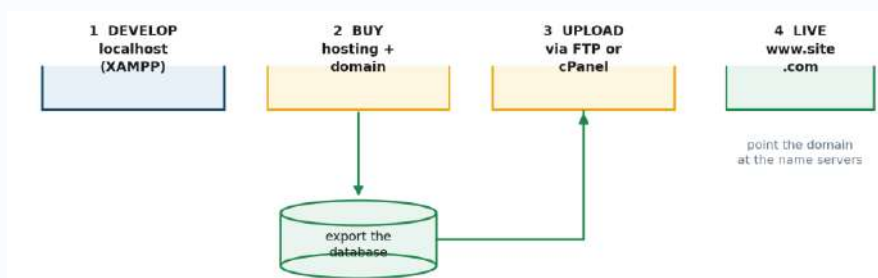


Figure 1.13 — Files are uploaded by FTP, the database is imported, and DNS points the domain to the server.

Domain name versus web hosting

Basis	Domain Name	Web Hosting
Meaning	The address of your website.	The space where the website files actually live.
Analogy	Your postal address.	The house at that address.
Provided by	Domain registrar.	Hosting provider.
Payment	Annual registration fee.	Monthly or annual rental.
Can exist alone?	Yes — a registered domain may point nowhere.	Yes — hosting is accessible by IP address alone.
Example	gandakiuniversity.edu.np	10 GB SSD, 100 GB bandwidth on an Apache server

1.4 Exam-Based Questions — Chapter 1**Group A — Very Short Answer Questions (2 marks each)**

1. Define the Internet. Write any two of its uses.
2. What is the World Wide Web? Who invented it and in which year?
3. Define a web browser and name any four popular web browsers.
4. What is a web server? Write the names of any two web server software packages.
5. Define URL and write its general syntax.
6. What is a protocol? Why is it necessary in computer networks?
7. Write the full form of HTTP, HTTPS, SMTP, POP3, FTP and WAP.
8. Write the default port numbers of HTTP, HTTPS, FTP, SMTP, POP3 and DNS.
9. What does it mean to say that HTTP is a stateless protocol?
10. Define DNS. Which port does it normally use?
11. What is a TLD? Give two examples of a gTLD and two of a ccTLD.
12. Define web hosting. Name any two types of web hosting.
13. Differentiate between an absolute URL and a relative URL with one example each.
14. What is the difference between a static and a dynamic web page?
15. What is a WAP gateway? State its function in one sentence.
16. What is meant by URL encoding? Encode the string `web tech&php`.

Group B — Short Answer Questions (5 marks each)

1. Differentiate between the Internet and the World Wide Web with at least five points.
2. Explain the client–server architecture of the web with a suitable block diagram.
3. Describe the internal components of a web browser with a diagram.
4. Explain the parts of a URL with a labelled diagram and a suitable example.

5. Differentiate between HTTP and HTTPS. Why is HTTPS considered secure?
6. Explain the working of SMTP and POP3 with a diagram showing the path of an e-mail.
7. Differentiate between POP3 and IMAP.
8. What is FTP? Explain its active and passive modes and compare FTP with SFTP.
9. Explain the domain name hierarchy with a neat labelled diagram.
10. Describe the DNS name-resolution process step by step.
11. Explain any five DNS record types with their purpose.
12. Explain the domain name registration process.
13. Compare shared hosting, VPS hosting, dedicated hosting and cloud hosting.
14. Explain the HTTP request and response message structure with an example of each.
15. Explain the different classes of HTTP status codes with two examples from each class.
16. Differentiate between a domain name and web hosting. Can one exist without the other?
17. What is WAP? Explain the WAP protocol stack with a diagram.

Group C — Long Answer Questions (7.5 marks each)

1. Explain in detail how a web page is delivered from a web server to a browser. Your answer must include the DNS lookup, the TCP connection, the HTTP request and response, and the rendering process, supported by a block diagram.
2. Define a web browser and a web server. Explain the architecture of a web browser with a labelled diagram, describe the function of each component, and explain the critical rendering path.
3. Explain the following protocols in detail with their purpose, working, default port and one practical use: HTTP, HTTPS, SMTP, POP3, FTP and WAP. Present a comparison table at the end.
4. What is DNS? Explain the domain name hierarchy with a diagram, describe the complete resolution process, and list any five DNS record types with their function.
5. Describe the complete process of taking a website from your local computer to the live Internet. Cover domain registration, choosing web hosting, uploading files and DNS configuration, with a suitable diagram.
6. Explain URL in detail. Describe each of its components with an example, differentiate between URL, URI and URN, and explain absolute versus relative URLs and URL encoding.

Laboratory / Practical Exercises — Chapter 1

1. Use the browser Network panel to open any website. Record the request method, status code, content type and total transfer size of the main document, and write a short report.
2. Run `nslookup` on five different websites and record their A records, MX records and name servers. Present the results in a table.
3. Perform a WHOIS lookup on `gandakiuniversity.edu.np` and on `google.com`. Compare the registrar, creation date and expiry date.
4. Take any three URLs of your choice and break each into scheme, host, port, path, query string and fragment in a tabular form.
5. Compare the pricing, storage, bandwidth and features of three real hosting providers and recommend one for a college website, justifying your choice.

Chapter 2: HTML and XHTML

Allotted time: 8 hours

Note: This is the foundation chapter of practical web development. Everything you build for the rest of the course — styled with CSS in Chapter 3, made interactive with JavaScript in Chapter 4 and connected to a database with PHP in Chapter 5 — is built on the markup you learn here. Master it thoroughly.

Chapter Learning Outcomes

After completing this chapter you will be able to:

- Explain what HTML and XHTML are, and write a valid HTML5 document from memory.
- Distinguish tags, elements and attributes, and use block and inline elements correctly.
- Apply text formatting using presentational and phrase (semantic) elements.
- Create unordered, ordered and definition lists, including nested lists and menus.
- Create hyperlinks of every kind and use images, audio and video in a page.
- Build tables using `colspan` and `rowspan` and know when tables should not be used.
- Build complete, accessible forms using every standard form control.

2.1 Structuring Documents for the Web: Introducing HTML and XHTML, Tags and Elements, Basic Syntax

2.1.1 What is HTML?

Definition

HTML (HyperText Markup Language) is the standard markup language used to create and structure the content of documents that are displayed in a web browser. It uses a system of **tags** to describe the meaning and structure of each part of a document — headings, paragraphs, lists, links, images, tables and forms.

Understanding the name

Word	Meaning
HyperText	Text that contains links (hyperlinks) to other text or documents. Clicking a link moves you non-sequentially to another resource — this is what makes the Web a "web".
Markup	The practice of "marking up" plain text with annotations that describe its structure and meaning, rather than writing programming instructions.
Language	A defined vocabulary of elements and a formal grammar (syntax) that browsers know how to interpret.

Important characteristics of HTML

- HTML is a **markup language, not a programming language**. It has no variables, no conditionals and no loops; it cannot perform calculations or take decisions.
- It is **platform independent** — the same file renders on Windows, macOS, Linux, Android and iOS.
- It is **case insensitive** (`<P>` and `<p>` are equivalent), although lower case is the universal convention.
- It is **forgiving** — browsers try to render a page even when tags are missing or wrongly nested. This is convenient but hides mistakes, so always validate your markup.
- It is **free and open** — maintained as a living standard by the **WHATWG**, in cooperation with the **W3C**.
- HTML files are saved with the extension `.html` or `.htm` and are plain text, so they can be written in any text editor.

Versions of HTML

Version	Year	Key contribution
HTML 1.0	1991	The original specification: about 18 elements, text and links only.
HTML 2.0	1995	First formal standard (RFC 1866); introduced forms.
HTML 3.2	1997	Tables, applets, text flow around images, and presentational tags such as <code></code> .
HTML 4.01	1999	Separation of structure from presentation; strong support for style sheets and scripting; three DTDs (Strict, Transitional, Frameset).
XHTML 1.0	2000	HTML 4.01 reformulated as strict XML.
HTML5	2014	Semantic elements, native audio and video, canvas, new form input types, local storage, geolocation. Removed the need for plug-ins such as Flash.
HTML Living Standard	2016 →	Continuous versionless updates maintained by WHATWG. This is what browsers implement today.

2.1.2 Tags, Elements and Attributes

Tag

A **tag** is a keyword enclosed in angle brackets that tells the browser how to treat the content that follows. Most tags occur in pairs: an **opening tag** `<p>` and a **closing tag** `</p>` in which the tag name is preceded by a forward slash.

Element

An **element** is the complete unit consisting of the opening tag, the content, and the closing tag. Element = opening tag + content + closing tag.

Attribute

An **attribute** is extra information written inside the opening tag as a `name="value"` pair. Attributes modify the behaviour or provide additional data about the element.

Figure 2.1 Anatomy of an HTML element

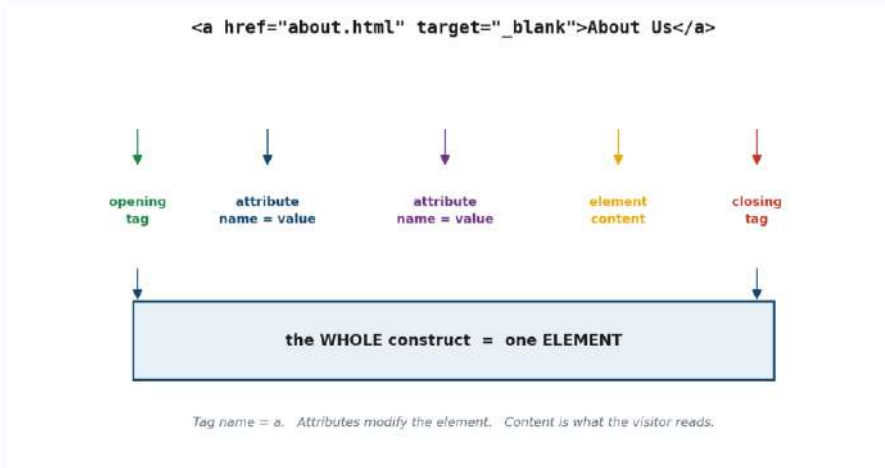


Figure 2.1 — Tag, element, attribute name, attribute value and content.

Paired versus empty elements

Type	Description	Examples
Paired (container) elements	Have both an opening and a closing tag and wrap content.	<code><p>...</p></code> , <code><h1>...</h1></code> , <code><div>...</div></code> , <code><a>...</code> , <code><table>...</table></code>
Empty (void / self-closing) elements	Have no content and therefore no closing tag. In XHTML they must be written with a trailing slash.	<code>
</code> , <code><hr></code> , <code></code> , <code><input></code> , <code><meta></code> , <code><link></code>

Global attributes — usable on almost every element

Attribute	Purpose	Example
<code>id</code>	A unique identifier for one element in the whole document. Used by CSS, JavaScript and internal links.	<code><div id="header"></code>
<code>class</code>	A group name that may be shared by many elements. The main hook for CSS styling.	<code><p class="note lead"></code>
<code>style</code>	Inline CSS applied to this element only.	<code><p style="color:red"></code>
<code>title</code>	Advisory text shown as a tooltip on hover.	<code><abbr title="World Wide Web"></code>
<code>lang</code>	The natural language of the content; important for screen readers.	<code><html lang="en"></code>
<code>dir</code>	Text direction, <code>ltr</code> or <code>rtl</code> .	<code><p dir="rtl"></code>

Attribute	Purpose	Example
<code>hidden</code>	Hides the element from rendering.	<code><div hidden></code>
<code>data-*</code>	Custom data attributes read by JavaScript.	<code><li data-price="450"></code>

Common Mistake: An `id` value must be unique in the page and must not contain spaces. A `class` value may be reused, and an element may carry several class names separated by spaces. Confusing the two is one of the most common beginner errors and it breaks both CSS and JavaScript.

2.1.3 Basic Syntax and Rules of HTML

- Every document begins with a **document type declaration**: `<!DOCTYPE html>`.
- Tags are enclosed in angle brackets and are conventionally written in lower case.
- Elements must be **properly nested**. `<b><i>text</i></b>` is correct; `<b><i>text</b></i>` is wrong.
- Attribute values should always be enclosed in double quotation marks.
- Multiple consecutive spaces, tabs and newlines in the source **collapse into a single space** when rendered. This is called whitespace collapsing.
- Comments are written as `<!-- this is a comment -->` and are not displayed by the browser.
- Indent nested elements by two spaces to keep the document readable.

2.1.4 Structure of an HTML Document

Figure 2.2 Skeleton structure of an HTML5 document

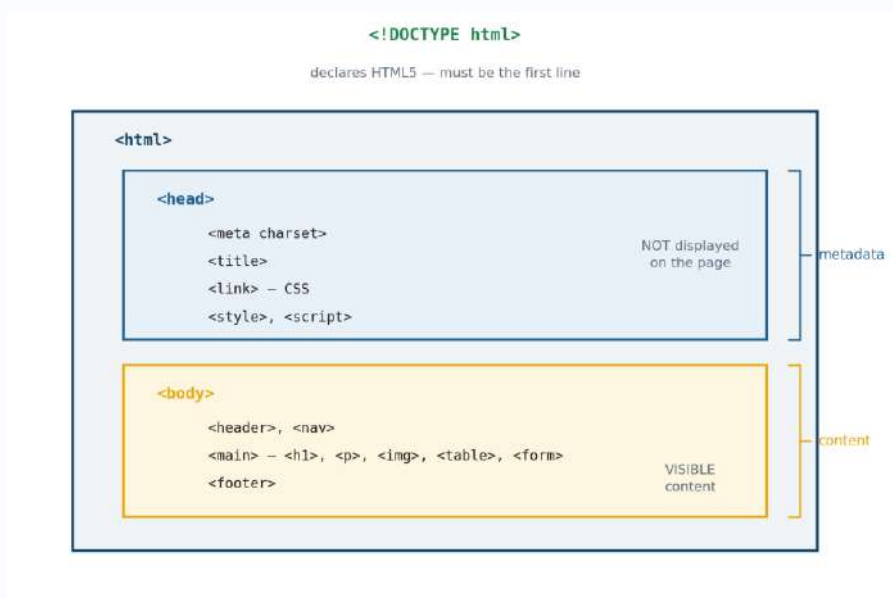


Figure 2.2 — The head holds information about the page; the body holds what the visitor sees.

Complete minimal HTML5 document

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Notes on Web Technology I for BIT students">
  <meta name="author" content="Dammar Khadayat">
  <title>My First Web Page</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <h1>Welcome to Gandaki University</h1>
  <p>This is my first web page written in <strong>HTML5</strong>.</p>
</body>
</html>
```

Line / Element	Purpose
<code><!DOCTYPE html></code>	Not a tag but a declaration. Tells the browser to use HTML5 standards mode. If omitted, the browser falls back to "quirks mode" and layout may break.
<code><html lang="en"></code>	The root element that contains everything. The <code>lang</code> attribute helps screen readers and search engines.
<code><head></code>	Container for metadata — information about the document that is not displayed in the page area.
<code><meta charset="UTF-8"></code>	Declares the character encoding. UTF-8 supports Devanagari, symbols and emoji. Always include it first.
<code><meta name="viewport" ...></code>	Makes the page scale correctly on mobile devices. Essential for responsive design (Chapter 3).
<code><title></code>	The text shown on the browser tab, in bookmarks and in search-engine results. Compulsory in a valid document.
<code><link rel="stylesheet"></code>	Attaches an external CSS file.
<code><body></code>	Contains all visible content: text, images, links, tables, forms and scripts.

2.1.5 Block-level and Inline Elements

Every HTML element has a default display behaviour that determines how it occupies space on the page.

Figure 2.3 Block versus inline flow

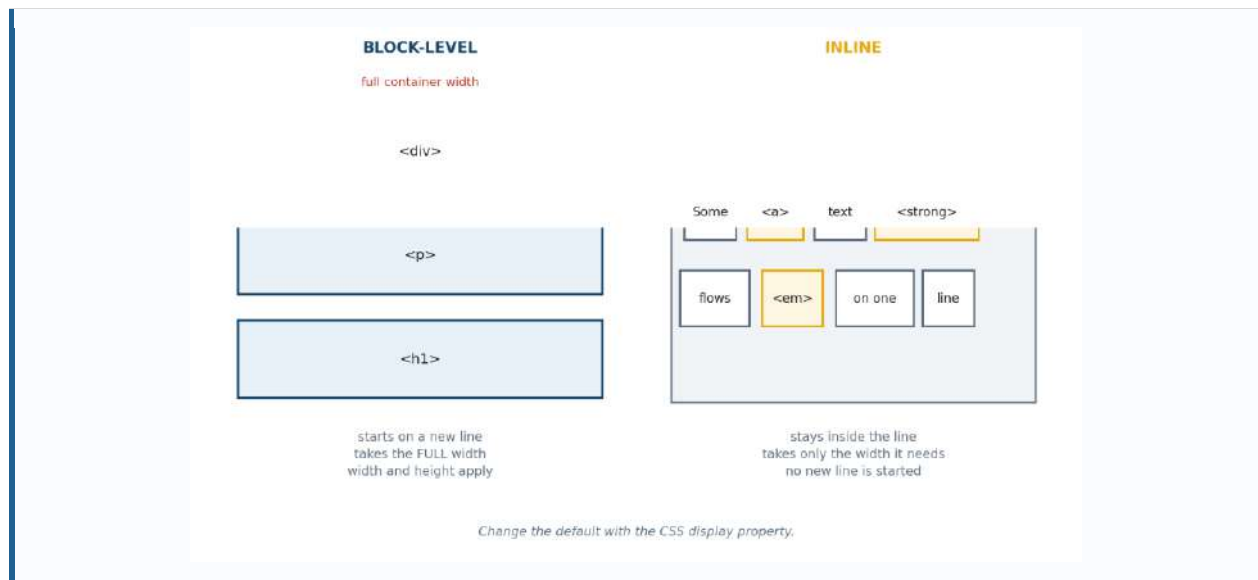


Figure 2.3 — Block elements stack vertically; inline elements flow horizontally within text.

Basis	Block-level element	Inline element
Line behaviour	Always begins on a new line.	Continues on the same line.
Width occupied	The full width of its container by default.	Only as wide as its content.
Width / height	Can be set with CSS.	Cannot be set (unless changed to inline-block).
Margin / padding	All four sides take effect.	Left and right work; top and bottom do not push other content.
Can contain	Both block and inline elements.	Only text and other inline elements.
Examples	<code><div></code> <code><p></code> <code><h1></code> – <code><h6></code> <code></code> <code></code> <code></code> <code><table></code> <code><form></code> <code><header></code> <code><footer></code> <code><section></code>	<code></code> <code><a></code> <code></code> <code></code> <code></code> <code></code> <code><i></code> <code><input></code> <code><label></code> <code>
</code> <code><code></code>

2.1.6 Semantic Elements in HTML5

Definition

A **semantic element** is one whose name clearly describes the *meaning* of its content to both the browser and the developer, rather than merely describing how it should look. `<header>` is semantic because it says what the content *is*; `<div class="header">` is non-semantic because `<div>` says nothing about meaning.

Why semantic markup matters

- **Accessibility** — screen readers can jump directly to the navigation or the main content, which greatly helps visually impaired users.
- **Search engine optimisation** — search engines understand the page structure and rank the content more accurately.

- **Maintainability** — code is self-documenting and easier for a team to read.
- **Future-proofing** — devices and tools that did not exist when the page was written can still interpret its structure.

Figure 2.4 Semantic page layout in HTML5

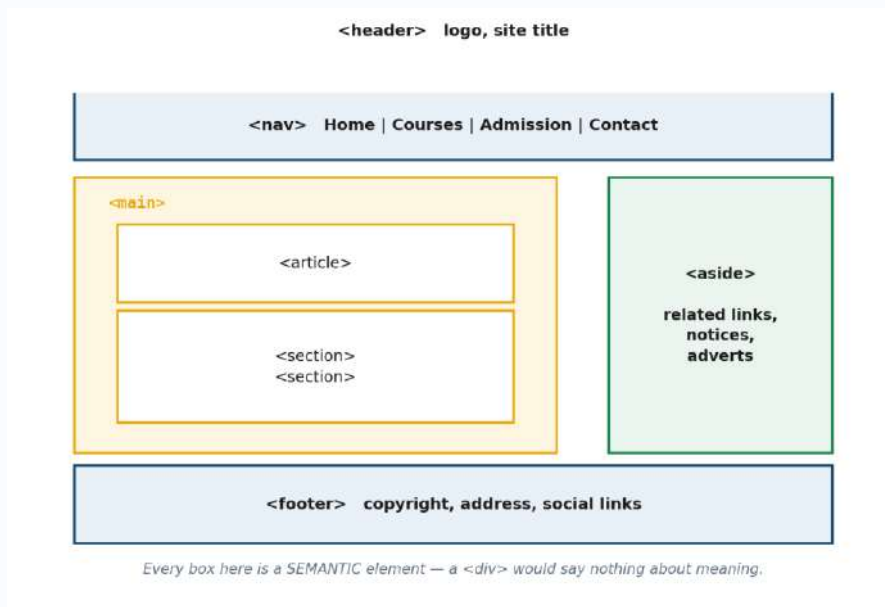


Figure 2.4 — A typical page built entirely from semantic elements instead of generic divisions.

Element	Meaning and correct use
<header>	Introductory content for the page or for a section: logo, site title, search box.
<nav>	A block of major navigation links. Use it only for primary navigation, not every group of links.
<main>	The dominant, unique content of the document. There must be exactly one per page.
<section>	A thematic grouping of content, normally with its own heading.
<article>	Self-contained content that would still make sense if published on its own: a blog post, a news item, a product card.
<aside>	Content tangentially related to the surrounding content: a sidebar, a pull quote, an advertisement.
<footer>	Closing information for the page or a section: copyright, contact details, related links.
<figure> / <figcaption>	An image, diagram or code listing together with its caption.
<time>	A machine-readable date or time: <time datetime="2026-07-18">18 July</time>.
<mark>	Text highlighted for reference, such as a search hit.
<div> / 	Non-semantic generic containers. Use them only when no semantic element fits.

2.1.7 XHTML — Extensible HyperText Markup Language

Definition

XHTML is a reformulation of HTML as an application of **XML (Extensible Markup Language)**. It contains the same elements and attributes as HTML 4.01 but enforces the strict, unforgiving syntax rules of XML. It was published by the W3C in 2000 with the aim of making web documents machine-parsable and error-free.

The strict rules of XHTML

Rule	Wrong (allowed in HTML)	Correct in XHTML
A DOCTYPE is mandatory	no doctype	<code><!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" ...></code>
Documents must be well-formed	unclosed tags	Every element opened must be closed.
Tag and attribute names must be lower case	<code><BODY></code> , <code></code>	<code><body></code> , <code></code>
Elements must be properly nested	<code><i>hi</i></code>	<code><i>hi</i></code>
Empty elements must be self-closed	<code>
</code> , <code></code>	<code>
</code> , <code></code>
Attribute values must be quoted	<code><table width=100></code>	<code><table width="100"></code>
Attribute minimisation is forbidden	<code><input checked></code>	<code><input checked="checked" /></code>
A root element with a namespace is required	<code><html></code>	<code><html xmlns="http://www.w3.org/1999/xhtml"></code>

Example of a valid XHTML 1.0 Strict document

```
XHTML
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">
<head>
<title>XHTML Example</title>
</head>
<body>
<p>All tags are closed, lower case and properly nested.</p>
<hr />

<br />
<input type="checkbox" checked="checked" />
</body>
</html>
```

Difference between HTML and XHTML

Basis	HTML	XHTML
Full form	HyperText Markup Language	Extensible HyperText Markup Language
Based on	SGML (Standard Generalized Markup Language)	XML (Extensible Markup Language)
Case sensitivity	Not case sensitive; <code>
</code> = <code>
</code>	Strictly lower case for tags and attributes
Closing tags	Some closing tags may be omitted, e.g. <code></p></code> , <code></code>	Every element must be closed without exception
Empty elements	Written as <code>
</code>	Must be self-closed as <code>
</code>
Attribute quotes	Optional	Compulsory
Attribute minimisation	Allowed, e.g. <code>checked</code>	Not allowed; must write <code>checked="checked"</code>
Root element	<code><html></code>	<code><html xmlns="http://www.w3.org/1999/xhtml"></code>
Error handling	Very forgiving; the browser guesses and renders anyway	Strict; a single error can stop the page from displaying when served as XML
MIME type	<code>text/html</code>	<code>application/xhtml+xml</code> (or <code>text/html</code> for compatibility)
Parsing	HTML parser	XML parser — faster and simpler but unforgiving
Current status	HTML5 living standard is the modern norm	Largely superseded; still used where XML tooling is required

Exam Tip: "Differentiate between HTML and XHTML" is a guaranteed favourite for 5 marks. Learn at least six points, and always mention the two headline facts: XHTML is based on XML while HTML is based on SGML, and XHTML is strict while HTML is forgiving.

Difference between HTML4 and HTML5

Basis	HTML 4.01	HTML5
DOCTYPE	Long and complex, referring to a DTD	Simply <code><!DOCTYPE html></code>
Character encoding	<code><meta http-equiv="content-type" content="text/html; charset=UTF-8"></code>	<code><meta charset="UTF-8"></code>
Multimedia	Required plug-ins such as Flash for audio and video	Native <code><audio></code> and <code><video></code> elements
Graphics	Not supported natively	<code><canvas></code> and inline SVG
Semantic elements	Not available; everything was a <code><div></code>	<code><header></code> , <code><nav></code> , <code><main></code> , <code><section></code> , <code><article></code> , <code><aside></code> , <code><footer></code>

Basis	HTML 4.01	HTML5
Form controls	Limited input types	email, url, number, date, range, color, search, tel, with built-in validation
Storage	Cookies only, about 4 KB	localStorage and sessionStorage, several megabytes
Other APIs	None	Geolocation, drag and drop, web workers, web sockets
Deprecated tags	<code></code> , <code><center></code> , <code><big></code> , <code><strike></code> were in use	Removed in favour of CSS

Note: Deprecated elements you should recognise but never use: `<font>`, `<center>`, `<big>`, `<strike>`, `<u>` (as a purely visual tag), `<frame>`, `<frameset>`, `<applet>`, `<marquee>`. All presentational effects belong in CSS.

2.2 Basic Text Formatting, Presentational Elements, Phrase Elements

2.2.1 Headings

HTML provides six levels of heading, `<h1>` (most important) down to `<h6>` (least important). Headings create the document outline that screen readers and search engines rely upon.

HTML

```
<h1>Gandaki University</h1>      <!-- page title, only ONE per page -->
<h2>Faculty of Science and Technology</h2>
<h3>Bachelor of Information Technology</h3>
<h4>Semester II</h4>
<h5>Web Technology I</h5>
<h6>Chapter 2</h6>
```

Common Mistake: Do not choose a heading level because of its size. Choose it because of its position in the outline, and control the size with CSS. Never skip levels — do not jump from `<h1>` to `<h4>`.

2.2.2 Paragraphs, Line Breaks and Horizontal Rules

HTML

```
<p>This is a paragraph. The browser adds space above and below it.</p>
<p>Second paragraph.<br>This sentence starts on a new line inside the
  same paragraph.</p>
<hr>      <!-- thematic break: a horizontal line -->
<pre>
  Text inside PRE keeps its
  spacing and line breaks exactly as typed.
</pre>
```

Element	Type	Purpose
<code><p></code>	Paired, block	Defines a paragraph of text with automatic spacing above and below.

Element	Type	Purpose
<code>
</code>	Empty, inline	Forces a line break within a block, for example inside an address or a poem.
<code><hr></code>	Empty, block	A thematic break between sections, rendered as a horizontal line.
<code><pre></code>	Paired, block	Preformatted text: preserves whitespace and line breaks and uses a monospace font.

Note: Whitespace collapsing: any run of spaces, tabs or newlines in your source code is displayed as a single space. To display several spaces you must use the entity ` ` or the `<pre>` element, or control spacing through CSS.

2.2.3 Character Entities

Some characters have special meaning in HTML and cannot be typed directly. They are written as **entities**, which begin with `&` and end with `;`.

Character	Entity name	Entity number	Meaning
<code><</code>	<code>&lt;</code>	<code>&#60;</code>	Less than — needed to display a tag as text
<code>></code>	<code>&gt;</code>	<code>&#62;</code>	Greater than
<code>&</code>	<code>&amp;</code>	<code>&#38;</code>	Ampersand
<code>"</code>	<code>&quot;</code>	<code>&#34;</code>	Double quotation mark
(space)	<code>&nbsp;</code>	<code>&#160;</code>	Non-breaking space
©	<code>&copy;</code>	<code>&#169;</code>	Copyright symbol
®	<code>&reg;</code>	<code>&#174;</code>	Registered trademark
₹ / ₹	<code>&#8377;</code>	<code>&#8377;</code>	Currency symbols
—	<code>&mdash;</code>	<code>&#8212;</code>	Em dash

2.2.4 Presentational Elements

Definition

Presentational elements describe only *how text should look*. They carry no meaning about the content. They exist for historical reasons; modern practice is to achieve these effects with CSS or to use semantic phrase elements instead.

Element	Visual effect	Status	Modern replacement
<code></code>	Bold	Valid in HTML5 but presentational	<code></code> for importance, or CSS <code>font-weight</code>
<code><i></code>	Italic	Valid but presentational	<code></code> for emphasis, or CSS <code>font-style</code>

Element	Visual effect	Status	Modern replacement
<code><u></code>	Underline	Discouraged (looks like a link)	CSS <code>text-decoration: underline</code>
<code><s></code> / <code><strike></code>	Strike-through	<code><strike></code> removed in HTML5	<code></code> or CSS <code>text-decoration: line-through</code>
<code><big></code>	Larger text	Removed in HTML5	CSS <code>font-size</code>
<code><small></code>	Smaller text	Redefined in HTML5 as "side comment"	Use for fine print and legal notes
<code><sup></code>	Superscript	Valid	Keep — x ² , footnote markers
<code><sub></code>	Subscript	Valid	Keep — H ₂ O, chemical formulae
<code></code>	Colour, size, face	Removed in HTML5	CSS <code>color</code> , <code>font-size</code> , <code>font-family</code>
<code><center></code>	Centres content	Removed in HTML5	CSS <code>text-align: center</code> or <code>margin: auto</code>

HTML

```

<p>Water is H<sub>2</sub>O and Einstein wrote E = mc<sup>2</sup>.</p>
<p><b>Bold looking</b> and <i>italic looking</i> text.</p>
<p><s>Rs. 1200</s> Rs. 950 only</p>
<p><small>Terms and conditions apply.</small></p>

```

2.2.5 Phrase (Semantic) Elements

Definition

Phrase elements mark up a fragment of text according to its **meaning** rather than its appearance. A browser gives each a default style, but the real value is that assistive technology, search engines and other software understand what the text represents.

Element	Meaning	Default rendering	Example
<code></code>	Strong importance, seriousness or urgency	Bold	<code>Warning!</code>
<code></code>	Stress emphasis that changes the sense of the sentence	Italic	I <code>never</code> said that
<code><abbr></code>	An abbreviation or acronym; use <code>title</code> for the expansion	Dotted underline	<code><abbr title="HyperText Markup Language">HTML</abbr></code>
<code><cite></code>	The title of a creative work	Italic	<code><cite>The Alchemist</cite></code>
<code><q></code>	A short inline quotation	Automatic quotation marks	He said <code><q>be brief</q></code> .
<code><blockquote></code>	A long, block-level quotation; <code>cite</code> gives the source URL	Indented block	<code><blockquote cite="url">...</blockquote></code>
<code><code></code>	A fragment of computer code	Monospace	<code><code>echo "hi";</code></code>
<code><kbd></code>	Keyboard input by the user	Monospace	Press <code><kbd>Ctrl</kbd>+<kbd>S</kbd></code>
<code><samp></code>	Sample output from a program	Monospace	<code><samp>File not found</samp></code>

Element	Meaning	Default rendering	Example
<code><var></code>	A variable in mathematics or programming	Italic	<code><var>x</var> = 5</code>
<code><dfn></code>	The defining instance of a term	Italic	A <code><dfn>tag</dfn></code> is ...
<code><address></code>	Contact information for the author or owner	Italic block	<code><address>Pokhara, Nepal</address></code>
<code></code>	Text deleted from the document	Strike-through	<code>Rs. 1200</code>
<code><ins></code>	Text inserted into the document	Underline	<code><ins>Rs. 950</ins></code>
<code><mark></code>	Text highlighted for reference	Yellow background	<code><mark>important</mark></code>

Complete worked example

HTML

```

<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Text Formatting Demo</title></head>
<body>
  <h1>Text Formatting in HTML</h1>

  <p><strong>Attention:</strong> the assignment is due on Sunday.</p>
  <p>You must <em>never</em> submit code you have not tested.</p>

  <p><abbr title="Cascading Style Sheets">CSS</abbr> controls presentation.</p>

  <blockquote cite="https://www.w3.org/">
    <p>The power of the Web is in its universality. Access by everyone
      regardless of disability is an essential aspect.</p>
  </blockquote>
  <p>— <cite>Tim Berners-Lee</cite></p>

  <p>To save a file press <kbd>Ctrl</kbd> + <kbd>S</kbd>.</p>
  <p>The statement <code>document.write("Hi");</code> prints
    <samp>Hi</samp> on the page.</p>
  <p>If <var>r</var> = 7 then area = &pi;<var>r</var><sup>2</sup>.</p>

  <p>Price: <del>Rs. 1200</del> <ins>Rs. 950</ins> only.</p>
  <p>Search result: web <mark>technology</mark> notes.</p>

  <address>
    Department of IT, Gandaki University<br>
    Pokhara, Nepal
  </address>
</body>
</html>

```

Difference between `<b>` and `<strong>`, and `<i>` and `<em>`

Basis	<code></code> and <code><i></code>	<code></code> and <code></code>
Category	Presentational — describe appearance	Semantic — describe meaning

Basis	 and <i>	 and
Purpose of / 	 simply draws attention without extra importance, e.g. a keyword or product name	 marks content of strong importance, seriousness or urgency
Purpose of <i> / 	<i> marks text in an alternate voice: a technical term, a foreign phrase, a thought	 marks stress emphasis that changes the meaning of the sentence
Screen reader	Read in a normal voice	Read with vocal stress or emphasis
SEO value	Little or none	Considered by search engines
Recommendation	Use only when no meaning is implied	Preferred in modern, accessible HTML

2.3 Lists, Creating Links, Images, Audio and Video

2.3.1 Lists

HTML provides three kinds of list. Lists are also the standard way to build navigation menus.

List type	Tag	Item tag	Use
Unordered list			Items whose order does not matter; rendered with bullets.
Ordered list			Items in a definite sequence; rendered with numbers or letters.
Description list	<dl>	<dt> term, <dd> description	Term-and-definition pairs such as a glossary or FAQ.

Syntax and example

HTML

```

<!-- 1. UNORDERED LIST -->
<h3>Skills required</h3>
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

<!-- 2. ORDERED LIST: type = 1 | A | a | I | i , start, reversed -->
<h3>Steps to create a web page</h3>
<ol type="1" start="1">
  <li>Open VS Code</li>
  <li>Create index.html</li>
  <li>Write the HTML skeleton</li>
  <li>Open with Live Server</li>
</ol>

<!-- 3. DESCRIPTION LIST -->
<h3>Glossary</h3>
<dl>
  <dt>HTML</dt>

```

```

    <dd>The markup language used to structure a web page.</dd>
  <dt>CSS</dt>
    <dd>The style sheet language used to present a web page.</dd>
</dl>

```

<!-- 4. NESTED LIST: a placed INSIDE an -->

```

<h3>Course structure</h3>
<ul>
  <li>Semester I
    <ul>
      <li>Programming in C</li>
      <li>Digital Logic</li>
    </ul>
  </li>
  <li>Semester II
    <ul>
      <li>Web Technology I
        <ol>
          <li>Basic Concept</li>
          <li>HTML and XHTML</li>
        </ol>
      </li>
      <li>Data Structures</li>
    </ul>
  </li>
</ul>

```

Common Mistake: A nested list must be placed **inside** an `<li>` element, before that `<li>` is closed. Placing a `<ul>` directly inside another `<ul>` is invalid markup.

Attributes of `<ol>`

Attribute	Values	Effect
<code>type</code>	1, A, a, I, i	Numbers, upper-case letters, lower-case letters, upper-case Roman, lower-case Roman
<code>start</code>	any integer	The number at which the list begins, e.g. <code>start="5"</code>
<code>reversed</code>	boolean	Counts downwards instead of upwards

Building a navigation menu from a list

HTML

```

<nav>
  <ul class="menu">
    <li><a href="index.html">Home</a></li>
    <li><a href="about.html">About</a></li>
    <li><a href="courses.html">Courses</a>
      <ul class="submenu">
        <li><a href="bit.html">BIT</a></li>
        <li><a href="bca.html">BCA</a></li>
      </ul>
    </li>
    <li><a href="contact.html">Contact</a></li>
  </ul>

```

```
</nav>
```

2.3.2 Creating Links (Hyperlinks)

Definition

A **hyperlink** is a reference from one document to another resource, created with the **anchor element** `<a>`. Clicking it navigates the browser to the target given in the `href` attribute. Hyperlinks are what turn a set of separate documents into an interconnected web.

Syntax

SYNTAX

```
<a href="URL" target="_blank" title="tooltip" rel="noopener">Link text</a>
```

Attribute	Purpose	Common values
<code>href</code>	The destination of the link (Hypertext REFERENCE).	A URL, a file path, <code>#id</code> , <code>mailto:</code> , <code>tel:</code>
<code>target</code>	Where to open the destination.	<code>_self</code> (default, same tab), <code>_blank</code> (new tab), <code>_parent</code> , <code>_top</code>
<code>title</code>	Advisory tooltip shown on hover.	any text
<code>rel</code>	Relationship between the current page and the target.	<code>noopener</code> , <code>noreferrer</code> , <code>nofollow</code> , <code>stylesheet</code>
<code>download</code>	Downloads the target instead of navigating to it.	optional file name

Types of links — complete example

HTML

```
<!-- 1. External link to another website (absolute URL) -->
<a href="https://www.w3schools.com" target="_blank" rel="noopener">
  Learn more at W3Schools</a>

<!-- 2. Internal link to another page of the same site (relative URL) -->
<a href="about.html">About Us</a>
<a href="pages/courses.html">Courses</a>    <!-- one folder down -->
<a href="../index.html">Home</a>            <!-- one folder up -->
<a href="/index.html">Site root</a>         <!-- root relative -->

<!-- 3. Link to a section within the SAME page (fragment / bookmark) -->
<a href="#chapter3">Go to Chapter 3</a>
...
<h2 id="chapter3">Chapter 3: CSS</h2>

<!-- 4. Link to a section of ANOTHER page -->
<a href="notes.html#unit5">Unit 5 of the notes</a>

<!-- 5. E-mail link, with optional subject and body -->
<a href="mailto:info@gandakiuniversity.edu.np?subject=Admission%20Query">
```

Email the office

<!-- 6. Telephone link (works on mobile devices) -->

Call us

<!-- 7. Download link -->

Download the syllabus

<!-- 8. IMAGE USED AS A LINK -->

Relative path notation

Notation	Meaning	Example
<code>file.html</code>	A file in the same folder as the current page.	<code>about.html</code>
<code>folder/file.html</code>	A file inside a sub-folder.	<code>pages/contact.html</code>
<code>../file.html</code>	Go up one folder level, then find the file.	<code>../index.html</code>
<code>../../file.html</code>	Go up two folder levels.	<code>../../home.html</code>
<code>/file.html</code>	Start from the root of the website.	<code>/css/style.css</code>

Link states used by CSS

The browser tracks four states for every link. They are styled in this order, which is remembered by the mnemonic **LoVe HAtE**: `:link`, `:visited`, `:hover`, `:active`.

CSS

```
a:link { color: #0645ad; text-decoration: none; } /* not yet visited */
a:visited { color: #6b3fa0; } /* already visited */
a:hover { color: #d14; text-decoration: underline; } /* mouse over it */
a:active { color: #ff6600; } /* being clicked */
```

Exam Tip: Always add `rel="noopener"` when using `target="_blank"`. Without it, the newly opened page can access `window.opener` and manipulate your page — a real security risk known as tab-nabbing.

2.3.3 Images

Definition and syntax

The `<img>` element embeds an image into a page. It is an **empty, inline** element: it has no closing tag and no content of its own.

SYNTAX

```

```

```
title="tooltip text" loading="lazy">
```

Attribute	Required?	Purpose
<code>src</code>	Yes	The path or URL of the image file (SouRCe).
<code>alt</code>	Yes	Alternative text read by screen readers and displayed if the image fails to load. Essential for accessibility, and legally required in many countries.
<code>width</code> / <code>height</code>	Recommended	Dimensions in pixels. Supplying them reserves space and prevents the page from jumping while images load.
<code>title</code>	No	Tooltip on hover.
<code>loading</code>	No	<code>lazy</code> defers loading until the image is near the viewport, which speeds up the page.

Image file formats

Format	Full form	Best used for	Transparency	Animation
JPEG / JPG	Joint Photographic Experts Group	Photographs, images with many colours	No	No
PNG	Portable Network Graphics	Logos, icons, screenshots, sharp edges	Yes	No
GIF	Graphics Interchange Format	Simple animations, 256-colour graphics	Yes (1-bit)	Yes
SVG	Scalable Vector Graphics	Logos, icons, diagrams — scales to any size without blurring	Yes	Yes
WebP	Web Picture format	Modern all-purpose replacement, smaller files	Yes	Yes

Worked example — image, figure and photo gallery

HTML

```
<!-- basic image -->


<!-- image with a semantic caption -->
<figure>
  
  <figcaption>Figure 1: The computer laboratory, block B.</figcaption>
</figure>

<!-- a simple photo gallery built from a list of linked thumbnails -->
<h2>Photo Gallery</h2>
<ul class="gallery">
  <li><a href="images/big1.jpg" target="_blank">
    </a></li>
  <li><a href="images/big2.jpg" target="_blank">
    </a></li>
```

```
<li><a href="images/big3.jpg" target="_blank">
  </a></li>
</ul>
```

Image maps — one image, several clickable regions

HTML

```


<map name="campusmap">
  <area shape="rect" coords="10,10,150,120" href="library.html"
    alt="Library">
  <area shape="circle" coords="300,200,60" href="canteen.html"
    alt="Canteen">
  <area shape="poly" coords="400,50,500,90,470,180,390,150"
    href="hostel.html" alt="Hostel">
</map>
```

Common Mistake: Never leave `alt` out. If an image is purely decorative write `alt=""` (empty), which tells a screen reader to skip it. Writing no `alt` attribute at all makes the screen reader announce the file name, which is meaningless to the user.

2.3.4 Audio and Video

Before HTML5 — the Flash era

Until about 2010, playing audio or video in a browser required a third-party plug-in, almost always **Adobe Flash Player**, embedded with the `<object>` or `<embed>` element. Flash had serious drawbacks: it needed a separate installation, consumed battery, had frequent security vulnerabilities, was not supported on iOS, and was invisible to search engines. **Adobe ended support for Flash on 31 December 2020** and all major browsers removed it. HTML5 replaced it with the native `<audio>` and `<video>` elements.

The ``<audio>`` element

HTML

```
<audio controls loop preload="metadata">
  <source src="media/anthem.mp3" type="audio/mpeg">
  <source src="media/anthem.ogg" type="audio/ogg">
  Your browser does not support the audio element.
</audio>
```

The ``<video>`` element

HTML

```
<video width="640" height="360" controls poster="images/preview.jpg"
  preload="metadata">
  <source src="media/lecture.mp4" type="video/mp4">
```

```
<source src="media/lecture.webm" type="video/webm">
<track src="media/captions-en.vtt" kind="subtitles" srclang="en"
  label="English" default>
Your browser does not support the video tag.
<a href="media/lecture.mp4">Download the video instead.</a>
</video>
```

Attribute	Applies to	Effect
controls	audio, video	Displays the browser's play, pause, volume and seek controls.
autoplay	audio, video	Starts playing automatically. Browsers block it unless muted is also present.
muted	audio, video	Starts with the sound off.
loop	audio, video	Repeats indefinitely.
preload	audio, video	none , metadata or auto — how much to download before playback.
poster	video only	An image shown before playback begins.
width / height	video only	The display size of the player.
<source>	both	Offers several formats; the browser plays the first one it supports.
<track>	video	Adds subtitles or captions from a WebVTT file — important for accessibility.

Embedding an external video with `<iframe>`

HTML

```
<iframe width="640" height="360"
  src="https://www.youtube.com/embed/VIDEO_ID"
  title="Lecture recording"
  frameborder="0"
  allow="accelerometer; autoplay; encrypted-media; picture-in-picture"
  allowfullscreen>
</iframe>
```

Note: An **<iframe>** (inline frame) embeds one HTML document inside another. It is used for YouTube videos, Google Maps and payment widgets. Because it loads third-party content, always set the **title** attribute for accessibility and consider the **sandbox** attribute for untrusted sources.

Difference between `<audio>`/`<video>` and Flash

Basis	HTML5 <audio> / <video>	Adobe Flash
Plug-in needed	No — built into the browser	Yes — Flash Player had to be installed
Mobile support	Full support on Android and iOS	Never supported on iOS
Security	No known plug-in attack surface	Frequent critical vulnerabilities

Basis	HTML5 <code><audio></code> / <code><video></code>	Adobe Flash
Accessibility	Supports <code><track></code> captions and keyboard control	Poor
SEO	Content is visible to search engines	Content was invisible
Control	Fully scriptable through the JavaScript DOM	Required ActionScript
Status	The current standard	Discontinued on 31 December 2020

2.4 Tables: Basic Table Elements and Attributes, Spanning Columns and Rows

2.4.1 Definition and Purpose

An **HTML table** displays data arranged in rows and columns. Tables must be used only for **tabular data** — information that genuinely has a row-and-column relationship, such as a timetable, a mark sheet or a price list. They must never be used to lay out the visual structure of a page; that is the job of CSS.

2.4.2 Basic Table Elements

Element	Full name	Purpose
<code><table></code>	Table	The container for the whole table.
<code><caption></code>	Caption	A title for the table. Must be the first child of <code><table></code> . Important for accessibility.
<code><tr></code>	Table Row	Defines one horizontal row.
<code><th></code>	Table Header cell	A heading cell. Bold and centred by default. Use <code>scope</code> to say what it heads.
<code><td></code>	Table Data cell	An ordinary data cell.
<code><thead></code>	Table Head	Groups the header rows. Repeated on each page when printing.
<code><tbody></code>	Table Body	Groups the main data rows.
<code><tfoot></code>	Table Foot	Groups summary rows such as totals.
<code><colgroup></code> / <code><col></code>	Column group	Lets you apply styling to whole columns at once.

Figure 2.5 The structure of an HTML table

The diagram illustrates the structure of an HTML table. At the top, the `<table>` tag is shown. Below it, the `<caption> Semester II Result </caption>` tag is shown. The table is divided into three main sections: `<thead>`, `<tbody>`, and `<tfoot>`. The `<thead>` section contains a single row with three header cells: `<th>` (Roll), `<th>` (Name), and `<th>` (Marks). The `<tbody>` section contains two rows of data cells: `<td>` (01), `<td>` (Sita), `<td>` (78) in the first row, and `<td>` (02), `<td>` (Ram), `<td>` (85) in the second row. The `<tfoot>` section contains a single row with three footer cells: `<td>` (Average), `<td>` (), and `<td>` (81.5). A note indicates that `<th>` is a header cell (bold, centred) and `<td>` is an ordinary data cell. A vertical arrow points from the `<tbody>` tag to the first row of data cells, with the text 'one <tr> per row'.

Roll	Name	Marks
01	Sita	78
02	Ram	85
Average		81.5

`<th>` = header cell (bold, centred). `<td>` = ordinary data cell.

Figure 2.5 — Rows are created by `<tr>`; cells inside a row are `<th>` or `<td>`.

Basic table example

HTML

```
<table border="1">
  <caption>BIT Semester II — Course List</caption>
  <thead>
    <tr>
      <th scope="col">S.N.</th>
      <th scope="col">Course Code</th>
      <th scope="col">Course Title</th>
      <th scope="col">Credit</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>1</td><td>CIT123</td><td>Web Technology I</td><td>3</td>
    </tr>
    <tr>
      <td>2</td><td>CIT124</td><td>Data Structures</td><td>3</td>
    </tr>
    <tr>
      <td>3</td><td>MTH121</td><td>Discrete Mathematics</td><td>3</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td colspan="3">Total Credits</td><td>9</td>
    </tr>
  </tfoot>
</table>
```

2.4.3 Table Attributes

Attribute	Applies to	Purpose	Status
<code>border</code>	<code><table></code>	Draws a border of the given pixel width.	Deprecated — use CSS <code>border</code>
<code>cellspacing</code>	<code><table></code>	Space between adjacent cells.	Deprecated — use CSS <code>border-spacing</code>
<code>cellpadding</code>	<code><table></code>	Space between the cell border and its content.	Deprecated — use CSS <code>padding</code>
<code>width / height</code>	<code><table></code> , <code><td></code>	Size of the table or cell.	Deprecated — use CSS
<code>align / valign</code>	<code><tr></code> , <code><td></code>	Horizontal and vertical alignment.	Deprecated — use CSS <code>text-align</code> , <code>vertical-align</code>
<code>bgcolor</code>	<code><table></code> , <code><td></code>	Background colour.	Deprecated — use CSS <code>background-color</code>
<code>colspan</code>	<code><td></code> , <code><th></code>	Makes a cell span several columns .	Current and valid
<code>rowspan</code>	<code><td></code> , <code><th></code>	Makes a cell span several rows .	Current and valid
<code>scope</code>	<code><th></code>	<code>col</code> , <code>row</code> , <code>colgroup</code> or <code>rowgroup</code> — tells assistive technology what the header applies to.	Current and recommended

2.4.4 Spanning Columns with `colspan`

The `colspan` attribute makes a single cell occupy the horizontal space of two or more columns.

Figure 2.6 How colspan works

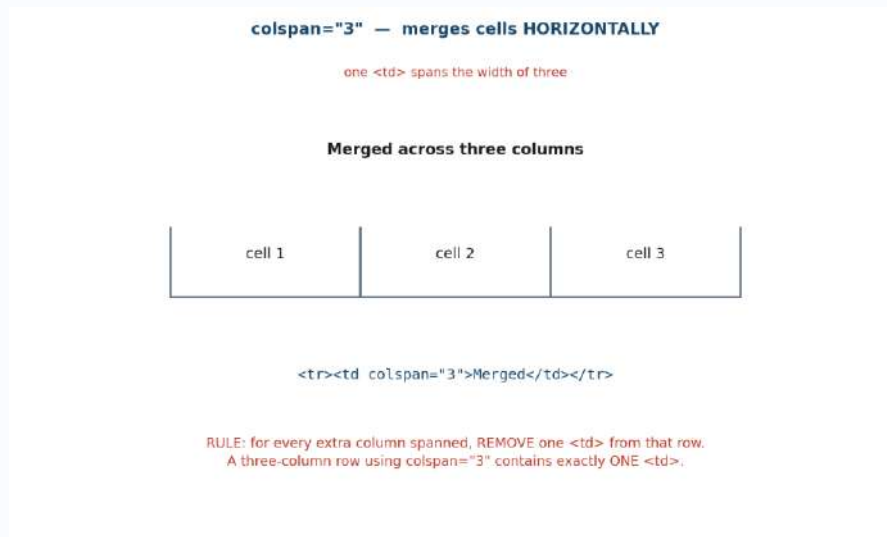


Figure 2.6 — `colspan` merges cells horizontally.

HTML

```
<table border="1">
  <caption>Class Routine</caption>
  <tr>
    <th colspan="4">BIT Second Semester — Weekly Routine</th>
  </tr>
```

```

<tr>
  <th>Day</th><th>10:00 - 11:00</th><th>11:00 - 12:00</th><th>12:00 - 1:00</th>
</tr>
<tr>
  <td>Sunday</td><td>Web Tech</td><td>Data Structure</td><td>Maths</td>
</tr>
<tr>
  <td>Monday</td><td>Web Tech Lab</td><td colspan="2">Project Work</td>
</tr>
</table>

```

2.4.5 Spanning Rows with `rowspan`

The **rowspan** attribute makes a single cell occupy the vertical space of two or more rows.

Figure 2.7 How rowspan works

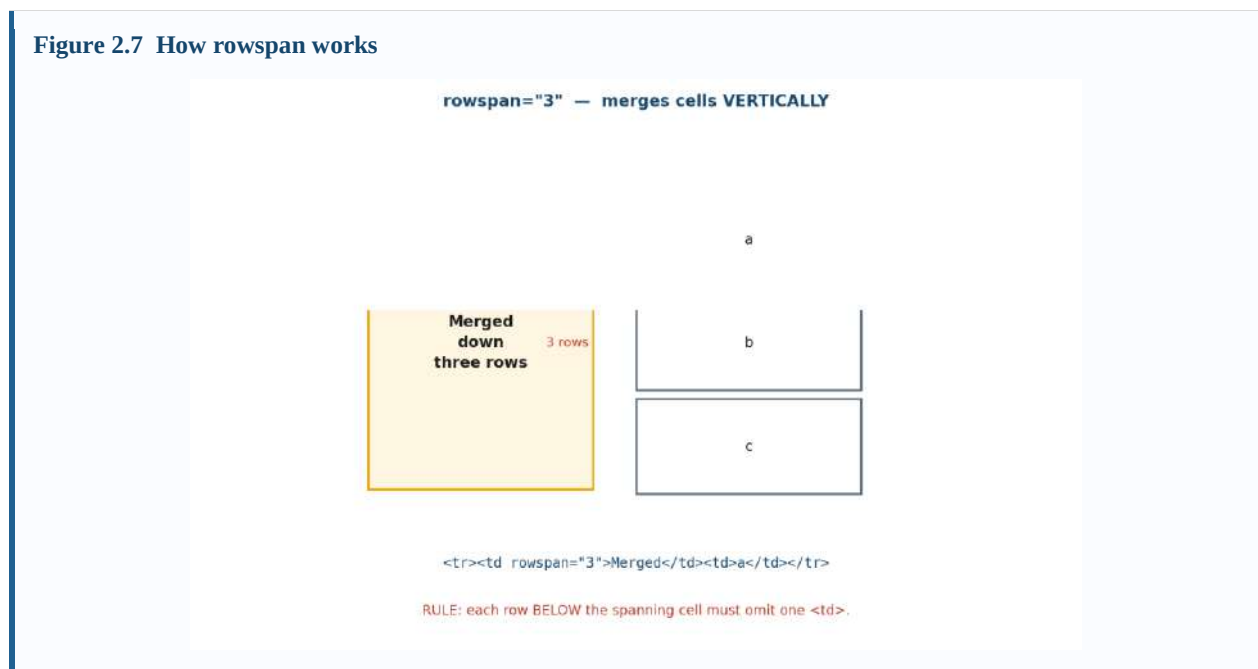


Figure 2.7 — rowspan merges cells vertically.

HTML

```

<table border="1">
  <caption>Faculty Time Allocation</caption>
  <tr>
    <th>Faculty</th><th>Course</th><th>Hours</th>
  </tr>
  <tr>
    <td rowspan="2">Mr. D. Khadayat</td>
    <td>Web Technology I</td><td>3</td>
  </tr>
  <tr>
    <!-- first <td> omitted: it is covered by the rowspan above -->
    <td>Web Technology Lab</td><td>2</td>
  </tr>
  <tr>
    <td>Ms. S. Sharma</td><td>Data Structures</td><td>3</td>
  </tr>
</table>

```

Combined `colspan` and `rowspan` — complete mark sheet**HTML**

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Mark Sheet</title>
  <style>
    table { border-collapse: collapse; width: 100%; font-family: Arial; }
    th, td { border: 1px solid #555; padding: 8px; text-align: center; }
    thead th { background: #14456b; color: #fff; }
    caption { font-weight: bold; margin-bottom: 8px; font-size: 18px; }
    tbody tr:nth-child(even) { background: #f2f6fa; }
  </style>
</head>
<body>

<table>
  <caption>Internal Assessment — Web Technology I (CIT123)</caption>
  <thead>
    <tr>
      <th rowspan="2">Roll No.</th>
      <th rowspan="2">Student Name</th>
      <th colspan="3">Marks Obtained</th>
      <th rowspan="2">Total</th>
    </tr>
    <tr>
      <th>Theory (20)</th>
      <th>Lab (15)</th>
      <th>Project (5)</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>01</td>
      <td>Sita Gurung</td>
      <td>18</td>
      <td>14</td>
      <td>5</td>
      <td>37</td>
    </tr>
    <tr>
      <td>02</td>
      <td>Ram Thapa</td>
      <td>16</td>
      <td>13</td>
      <td>4</td>
      <td>33</td>
    </tr>
    <tr>
      <td>03</td>
      <td>Bishal K.C.</td>
      <td>19</td>
      <td>15</td>
      <td>5</td>
      <td>39</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td colspan="5">Class Average</td>
      <td>36.33</td>
    </tr>
  </tfoot>
</table>

</body>
</html>

```

Difference between `colspan` and `rowspan`

Basis	colspan	rowspan
Direction of merge	Horizontal — across columns	Vertical — down rows
Written on	<td> or <th>	<td> or <th>
Effect on markup	Remove (n - 1) cells from the same row	Remove one cell from each of the (n - 1) rows below

Basis	colspan	rowspan
Typical use	A heading that covers several columns; a totals row	A category name repeated down several rows
Example	<code><td colspan="3">Total</td></code>	<code><td rowspan="2">Semester II</td></code>

Common Mistake: The most common table error is a mismatched cell count. After using `colspan="n"`, remove $n - 1$ cells from that row. After using `rowspan="n"`, remove one cell from each of the following $n - 1$ rows. Otherwise the table becomes crooked.

Note: Use `border-collapse: collapse;` in CSS so that adjacent cell borders merge into a single clean line instead of producing a double line.

2.5 Forms and Form Controls

2.5.1 Introduction to Forms

Definition

An **HTML form** is a section of a document that contains interactive controls through which a user submits information to a web server. Forms are the primary means of collecting input on the Web — registration, login, search, feedback, payment and file upload all use forms.

Figure 2.8 The complete life-cycle of a form submission

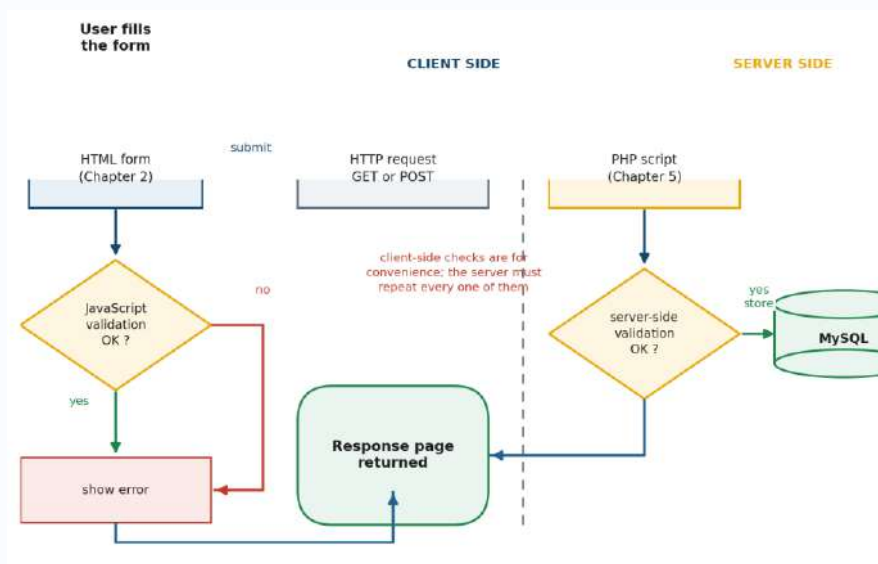


Figure 2.8 — The form is the bridge between Chapter 2 (HTML), Chapter 4 (JavaScript validation) and Chapter 5 (PHP processing).

2.5.2 Creating a Form — the `` Element

Syntax

SYNTAX

```
<form action="process.php" method="post" name="regForm" id="regForm"
  enctype="multipart/form-data" target="_self"
  accept-charset="UTF-8" novalidate
  onsubmit="return validateForm()" onreset="return confirm('Clear?')">
  ... form controls go here ...
</form>
```

Attributes of the `` element

Attribute	Purpose	Typical values
<code>action</code>	The URL of the server-side program that will receive and process the data. If omitted, the form submits to the same page.	<code>process.php</code> , <code>login.php</code>
<code>method</code>	The HTTP method used to send the data.	<code>get</code> (default) or <code>post</code>
<code>name</code>	A name for the form, used by older JavaScript as <code>document.regForm</code> .	any identifier
<code>id</code>	A unique identifier used by CSS and by <code>document.getElementById()</code> .	any identifier
<code>target</code>	Where to display the response.	<code>_self</code> , <code>_blank</code> , <code>_parent</code> , <code>_top</code>
<code>enctype</code>	How the form data is encoded before sending. Only relevant when <code>method="post"</code> .	<code>application/x-www-form-urlencoded</code> (default), <code>multipart/form-data</code> (required for file upload), <code>text/plain</code>
<code>accept-charset</code>	The character encoding the server should use to interpret the data.	<code>UTF-8</code>
<code>accept</code>	A hint about the file types accepted (now normally placed on the <code><input type="file"></code> instead).	<code>image/*</code> , <code>.pdf</code>
<code>novalidate</code>	Switches off the browser's built-in HTML5 validation.	boolean
<code>autocomplete</code>	Whether the browser may auto-fill the fields.	<code>on</code> / <code>off</code>
<code>onsubmit</code>	JavaScript that runs when the form is submitted. Returning <code>false</code> cancels the submission — this is how client-side validation works.	<code>return validateForm()</code>
<code>onreset</code>	JavaScript that runs when the form is reset.	<code>return confirm("Clear?")</code>

Difference between the GET and POST methods

Basis	GET	POST
Data location	Appended to the URL as a query string after <code>?</code>	Sent inside the body of the HTTP request
Visibility	Visible in the address bar, browser history and server logs	Not visible in the address bar
Security	Insecure — never use for passwords or personal data	More secure, though still needs HTTPS to be truly safe

Basis	GET	POST
Data length limit	Limited by the maximum URL length, about 2048 characters	Practically unlimited (server configurable)
Data type	ASCII text only	Text and binary; supports file upload
Bookmark / share	The result can be bookmarked and shared	Cannot be bookmarked
Caching	Can be cached by the browser	Not cached
Browser reload	Re-executes harmlessly	The browser warns about re-submitting data
Idempotent	Yes — repeating it does not change the server state	No — repeating it may create duplicate records
Read in PHP by	<code>\$_GET["fieldname"]</code>	<code>\$_POST["fieldname"]</code>
Best used for	Search queries, filters, pagination — anything retrieving data	Login, registration, payment, file upload — anything changing data

Exam Tip: A reliable one-line rule for the examination: ****use GET to *get* data, use POST to *post* (change) data.****

2.5.3 The `` Element and Its Types

`<input>` is an empty inline element whose behaviour is completely determined by its `type` attribute.

<code>type</code> value	Control produced	Notes / example
text	Single-line text box	The default. <code><input type="text" name="fname"></code>
password	Text box that masks characters	Masking is visual only; use HTTPS for real security.
radio	Radio button	Only one option can be selected in a group. All buttons in the group must share the same `name` and have different `value`'s .
checkbox	Check box	Zero or more options may be selected. Use <code>name="hobby[]"</code> to receive an array in PHP.
file	File selection box	Requires <code>enctype="multipart/form-data"</code> on the form.
hidden	Invisible field	Carries data the user should not edit, such as a record id or a CSRF token.
submit	Submit button	Sends the form to the action URL.
reset	Reset button	Restores every control to its initial value.
button	Generic button	Does nothing unless JavaScript is attached.
image	Image used as a submit button	<code><input type="image" src="go.png" alt="Go"></code>
email	E-mail box	HTML5 — the browser checks the format automatically.
url	URL box	HTML5 — checks for a valid web address.

type value	Control produced	Notes / example
number	Numeric spinner	Supports <code>min</code> , <code>max</code> and <code>step</code> .
tel	Telephone box	Shows a numeric keypad on mobile devices.
date, time, datetime-local, month, week	Date and time pickers	HTML5 — a native calendar is displayed.
range	Slider	<code><input type="range" min="0" max="100" step="5"></code>
color	Colour picker	Returns a hexadecimal value such as <code>#ff0000</code> .
search	Search box	Displays a clear (×) button in most browsers.

Common attributes of form controls

Attribute	Purpose
name	The key under which the value is sent to the server. A control with no `name` is not submitted at all. This is the single most important attribute.
value	The initial value, or for radio buttons and check boxes the value sent when the control is selected.
id	Unique identifier; links a <code><label></code> to its control and is used by JavaScript.
placeholder	Grey hint text shown inside an empty field. It is a hint, not a substitute for a <code><label></code> .
required	The form cannot be submitted while the field is empty (HTML5 validation).
readonly	The value can be seen and submitted but not edited.
disabled	The control is greyed out and its value is not submitted.
maxlength / minlength	Maximum and minimum number of characters.
size	The visible width of a text box in characters.
pattern	A regular expression the value must match, e.g. <code>pattern="[0-9]{10}"</code> for a ten-digit phone number.
min / max / step	Range constraints for numbers, dates and sliders.
checked	Pre-selects a radio button or check box.
selected	Pre-selects an <code><option></code> inside a <code><select></code> .
multiple	Allows several values — several files, several e-mail addresses, several list options.
autofocus	Places the cursor in this control when the page loads.
accept	Restricts the file types offered in a file picker, e.g. <code>accept=".pdf,image/*"</code> .

2.5.4 Other Form Controls

`<label>` — the accessibility essential

HTML

```

<!-- Method 1: explicit association (recommended) -->
<label for="email">Email address:</label>
<input type="email" id="email" name="email">

<!-- Method 2: implicit, by wrapping the control -->
<label>Email address: <input type="email" name="email"></label>

```

The **for** attribute of the label must equal the **id** of the control. This lets a screen reader announce the correct prompt, and lets a sighted user click the text to focus the field — which is especially valuable for small radio buttons and check boxes.

`<textarea>` — multi-line text

HTML

```

<label for="msg">Your message:</label>
<textarea id="msg" name="message" rows="6" cols="45"
  placeholder="Type your feedback here..."
  maxlength="500"></textarea>

```

Note: `<textarea>` is a paired element and its initial value goes between the tags, not in a **value** attribute. Never put whitespace between `<textarea>` and `</textarea>` unless you want that whitespace to appear in the box.

`<select>`, ``<option>`` and ``<optgroup>`` — drop-down lists

HTML

```

<!-- single-selection drop-down -->
<label for="prog">Choose a programme:</label>
<select id="prog" name="programme" required>
  <option value="">-- Select --</option>
  <option value="bit" selected>BIT</option>
  <option value="bca">BCA</option>
  <option value="bba">BBA</option>
</select>

<!-- grouped and multiple-selection list box -->
<label for="subj">Elective subjects (hold Ctrl to pick more):</label>
<select id="subj" name="subjects[]" multiple size="6">
  <optgroup label="Programming">
    <option value="web">Web Technology</option>
    <option value="java">Java Programming</option>
  </optgroup>
  <optgroup label="Management">
    <option value="mis">Management Information System</option>
    <option value="ent">Entrepreneurship</option>
  </optgroup>
</select>

```

`<fieldset>` and ``<legend>`` — grouping related controls

HTML

```

<fieldset>
  <legend>Personal Information</legend>

```

```
<label for="fn">First name:</label>
<input type="text" id="fn" name="fname"><br><br>
<label for="ln">Last name:</label>
<input type="text" id="ln" name="lname">
</fieldset>
```

`<datalist>` — a text box with suggestions

HTML

```
<label for="city">City:</label>
<input list="cities" id="city" name="city">
<datalist id="cities">
  <option value="Pokhara">
  <option value="Kathmandu">
  <option value="Butwal">
  <option value="Biratnagar">
</datalist>
```

2.5.5 Complete Worked Example — Student Registration Form

This single example uses every control required by the syllabus: text and password inputs, radio buttons, check boxes, a select box, a file select box, hidden controls, a text area and buttons.

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Registration — Gandaki University</title>
</style>
  body { font-family: Arial, sans-serif; background:#f4f6f8; padding:20px; }
  form { max-width:640px; margin:auto; background:#fff; padding:24px;
    border-radius:8px; box-shadow:0 2px 8px rgba(0,0,0,.12); }
  h2 { color:#14456b; margin-top:0; }
  fieldset { border:1px solid #cfd8e3; border-radius:6px;
    margin-bottom:18px; padding:14px; }
  legend { font-weight:bold; color:#1f6091; padding:0 6px; }
  label { display:inline-block; width:150px; vertical-align:top;
    margin:8px 0; }
  input[type=text], input[type=email], input[type=password],
  input[type=tel], input[type=date], select, textarea {
    width:300px; padding:7px; border:1px solid #b9c6d4;
    border-radius:4px; }
  .btn { background:#14456b; color:#fff; border:none; padding:10px 22px;
    border-radius:4px; cursor:pointer; margin-right:8px; }
  .btn:hover { background:#1f6091; }
  .req { color:#c0392b; }
</style>
</head>
<body>

<form action="register.php" method="post"
  enctype="multipart/form-data" accept-charset="UTF-8"
  name="regForm" id="regForm" onsubmit="return validateForm()"
  onreset="return confirm('Clear all the fields?')">
```

<h2>Student Registration Form</h2>

<!-- HIDDEN CONTROL: sent to the server but never shown -->

<input type="hidden" name="form_id" value="REG-2026-BIT">

<fieldset>

<legend>Personal Details</legend>

<label for="fullname">Full Name *</label>

<input type="text" id="fullname" name="fullname" required
placeholder="e.g. Sita Gurung" maxlength="50" autofocus>

<label for="dob">Date of Birth</label>

<input type="date" id="dob" name="dob" max="2010-12-31">

<label>Gender *</label>

<input type="radio" id="m" name="gender" value="male" required>

<label for="m" style="width:auto">Male</label>

<input type="radio" id="f" name="gender" value="female">

<label for="f" style="width:auto">Female</label>

<input type="radio" id="o" name="gender" value="other">

<label for="o" style="width:auto">Other</label>

<label for="photo">Passport Photo</label>

<input type="file" id="photo" name="photo" accept="image/*">

</fieldset>

<fieldset>

<legend>Contact and Account</legend>

<label for="email">Email *</label>

<input type="email" id="email" name="email" required
placeholder="you@example.com">

<label for="phone">Mobile Number</label>

<input type="tel" id="phone" name="phone" pattern="[0-9]{10}"
placeholder="10 digits" title="Enter exactly 10 digits">

<label for="pwd">Password *</label>

<input type="password" id="pwd" name="password" required
minlength="8" placeholder="At least 8 characters">

<label for="cpwd">Confirm Password *</label>

<input type="password" id="cpwd" name="cpassword" required>

</fieldset>

<fieldset>

<legend>Academic Information</legend>

<label for="prog">Programme *</label>

<select id="prog" name="programme" required>

<option value="">-- Select a programme --</option>

<option value="bit">BIT</option>

<option value="bca">BCA</option>

<option value="bba">BBA</option>

</select>

<label for="sem">Semester</label>

<select id="sem" name="semester" size="1">

<option value="1">First</option>

<option value="2" selected>Second</option>

```

    <option value="3">Third</option>
</select><br>

<label>Interests</label>
<input type="checkbox" id="c1" name="interest[]" value="web" checked>
  <label for="c1" style="width:auto">Web Development</label>
<input type="checkbox" id="c2" name="interest[]" value="ai">
  <label for="c2" style="width:auto">AI</label>
<input type="checkbox" id="c3" name="interest[]" value="net">
  <label for="c3" style="width:auto">Networking</label><br>

<label for="addr">Address</label>
<textarea id="addr" name="address" rows="4" cols="40"
  placeholder="Ward, Municipality, District"></textarea><br>

<label for="roll">Roll Number</label>
<input type="text" id="roll" name="roll" value="BIT-2026-001" readonly>
</fieldset>

<fieldset>
  <legend>Declaration</legend>
  <input type="checkbox" id="agree" name="agree" value="yes" required>
  <label for="agree" style="width:auto">
    I declare that the information given above is true.</label>
</fieldset>

<button type="submit" class="btn">Register</button>
<button type="reset" class="btn" style="background:#7f8c8d">Reset</button>

</form>
</body>
</html>

```

Note: The JavaScript function `validateForm()` referenced in `onsubmit` is written in Chapter 4, and the server-side script `register.php` that stores this data in a MySQL database is written in Chapter 5. The three chapters together build one complete application.

2.5.6 Forms and Accessibility — a Short Checklist

- Give every control a `<label>` whose `for` matches the control's `id`.
- Group related radio buttons and check boxes inside a `<fieldset>` with a `<legend>`.
- Mark compulsory fields both visually (an asterisk) and programmatically (`required`).
- Do not rely on `placeholder` alone as a label — it disappears as soon as the user types.
- Keep the tab order logical; the source order is the tab order.
- Give error messages in text, not by colour alone.

2.6 Exam-Based Questions — Chapter 2

Group A — Very Short Answer Questions (2 marks each)

1. Define HTML. Write its full form and the file extension used.
2. What is the difference between a tag and an element? Give one example of each.

3. What is an attribute? Write any two global attributes with their use.
4. What is the purpose of the `<!DOCTYPE html>` declaration?
5. Differentiate between the `<head>` and the `<body>` section of an HTML document.
6. What is an empty element? Give any four examples.
7. Differentiate between `id` and `class` attributes.
8. What are semantic elements? Write any four with their purpose.
9. Differentiate between `<b>` and `<strong>`.
10. Write the HTML code to create an ordered list of any three items using Roman numerals.
11. Write the syntax of the anchor element and explain the `target` attribute.
12. What is the use of the `alt` attribute in the `<img>` tag? Why is it important?
13. Differentiate between `colspan` and `rowspan`.
14. What is the use of the `enctype` attribute? Which value is required for file upload?
15. Differentiate between radio buttons and check boxes.
16. What is the purpose of the `<label>` element in a form?
17. What is a hidden control? Where is it used?
18. Write the code to embed an audio file with playback controls.
19. What is whitespace collapsing in HTML? How can several spaces be displayed?
20. Write any four character entities with their meaning.

Group B — Short Answer Questions (5 marks each)

1. Explain the structure of an HTML5 document with a diagram and write a complete minimal example.
2. Differentiate between HTML and XHTML with at least six points.
3. Differentiate between HTML4 and HTML5 with suitable examples.
4. Explain block-level and inline elements with examples and a diagram.
5. What are semantic elements? Explain any six with a diagram of a semantic page layout.
6. Explain presentational elements and phrase elements with at least four examples of each.
7. Explain the three types of HTML list with syntax and a code example of each.
8. Write an HTML program to create a nested list showing at least two levels.
9. Explain the different types of hyperlink (external, internal, fragment, e-mail, telephone and download) with a code example of each.
10. Explain the `<img>` element with all its important attributes. Write code to use an image as a hyperlink.
11. Explain the HTML5 `<audio>` and `<video>` elements with their attributes and a code example.
12. Explain the basic elements of an HTML table with a code example that uses `<thead>`, `<tbody>` and `<tfoot>`.
13. Write an HTML program to create a table using both `colspan` and `rowspan`.
14. Differentiate between the GET and POST methods of form submission.
15. Explain any eight attributes of the `<form>` element.
16. Explain any eight input types available in HTML5 with syntax.

17. Explain `<select>`, `<option>`, `<optgroup>` and `<datalist>` with a code example.
18. Write the HTML code to create a photo gallery of four images in which each thumbnail links to the full-size picture.
19. Explain the use of `<iframe>` and write code to embed a YouTube video.
20. What is a description list? Compare it with an unordered list and give a code example.

Group C — Long Answer Questions (7.5 marks each)

1. Explain the complete structure of an HTML5 document. Describe the purpose of each element in the `<head>` section, differentiate between block-level and inline elements, and write a complete web page that uses at least eight different semantic elements.
2. What is XHTML? Explain its rules in detail, differentiate it from HTML with at least eight points, and write a valid XHTML 1.0 Strict document.
3. Design a complete web page for a college that contains a heading, a navigation menu built from a list, an image gallery of at least three images, internal and external links, and a footer. Write the full HTML code with comments.
4. Explain HTML tables in detail. Describe every table element and attribute, explain `colspan` and `rowspan` with diagrams, and write the complete code for a mark sheet that uses both.
5. Explain HTML forms in detail. Describe the `<form>` element and its attributes, differentiate between GET and POST, explain at least ten input types, and write a complete registration form that uses text, password, radio, check box, select, file, hidden and text-area controls.
6. Explain how multimedia is included in a web page. Compare the HTML5 approach with the older Flash approach, describe the `<audio>`, `<video>`, `<source>` and `<track>` elements with all their attributes, and write complete code for a page containing an audio player, a video player and an embedded YouTube video.
7. Explain the different types of link and path notation in HTML. Write a complete multi-page website structure (at least three pages in different folders) and show the exact relative links needed to navigate between them.

Laboratory / Practical Exercises — Chapter 2

1. Develop a web page containing an unordered list, an ordered list, a description list and a nested list, together with a horizontal navigation menu built from a list.
2. Develop a web page with a different look and feel using divisions (`<div>`) and semantic sectioning elements.
3. Create a photo gallery page with at least six thumbnail images where each image links to its full-size version.
4. Create a set of three pages that link to one another (internal links) and also link to at least two external websites. Include fragment links within a long page.
5. Design a user interface with different controls to take user data: a complete registration form using text, password, e-mail, telephone, date, radio, check box, select, multiple select, file, text area, hidden field, submit and reset.
6. Create your class routine as an HTML table using `colspan` and `rowspan`, with `<caption>`, `<thead>`, `<tbody>` and `<tfoot>`.

7. Build a page that plays one audio file and one video file with native controls, and also embeds one YouTube video through an `<iframe>`.
8. Validate all the pages you created using the W3C Markup Validation Service and correct every reported error. Submit a screenshot of the "Document checking completed. No errors" message.

Chapter 3: Introducing Cascading Style Sheets

Allotted time: 8 hours

Note: HTML gives a page its structure and meaning; CSS gives it its appearance. This chapter teaches you to separate the two, which is the single most important principle of professional web development.

Chapter Learning Outcomes

After completing this chapter you will be able to:

- Explain what CSS is and why the separation of content from presentation matters.
- Write CSS using all three levels — inline, internal and external — and choose the right one.
- Use every category of selector and calculate specificity to predict which rule wins.
- Apply colour, text and font properties confidently.
- Draw and apply the CSS box model, including `box-sizing`.
- Use the `:focus` and `:active` pseudo-classes and other pseudo-classes and pseudo-elements.
- Position elements using `static`, `relative`, `absolute`, `fixed` and `sticky`, and lay out pages with floats, Flexbox and Grid.
- Make a page responsive with media queries.

3.1 Introduction, Where You Can Add CSS Rules, Levels of Style Sheets

3.1.1 What is CSS?

Definition

CSS (Cascading Style Sheets) is a style-sheet language used to describe the presentation of a document written in a markup language such as HTML. It controls colour, font, spacing, size, layout, position and responsiveness — everything about how a page **looks**, as opposed to what the page **contains**.

The word "**Cascading**" refers to the algorithm the browser uses to decide which rule applies when several rules target the same element. The rules cascade down, and the winner is chosen by origin, importance, specificity and source order.

Why CSS was created — the problem it solved

Before CSS, appearance was written directly into the HTML using elements such as `<font>`, `<center>` and attributes such as `bgcolor` and `align`. If a site had 200 pages and the heading colour had to change, all 200 files had to be edited. CSS removed this problem by allowing one style sheet to control an unlimited number of pages.

Figure 3.1 Why separation of content and presentation matters

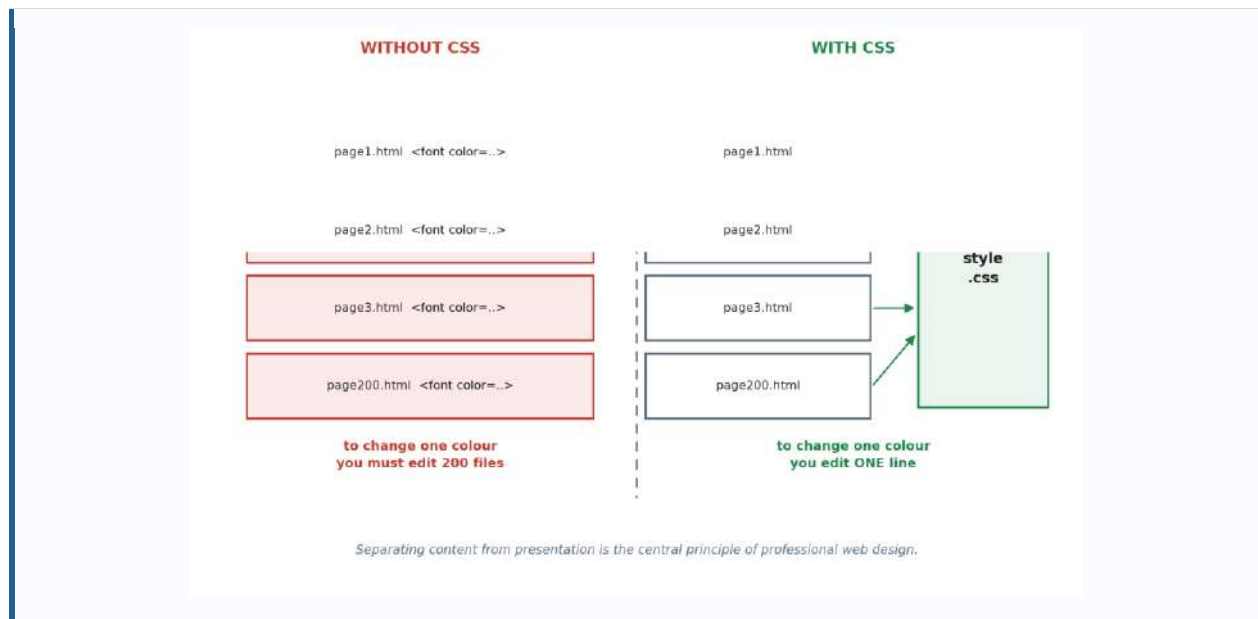


Figure 3.1 — One style sheet controls every page of a website.

Advantages of CSS

- **Separation of concerns** — structure lives in HTML, presentation lives in CSS. Each can be edited by a different person.
- **Consistency** — one style sheet gives every page an identical look.
- **Maintainability** — a single edit updates the whole site.
- **Faster page loading** — the HTML file becomes smaller and the CSS file is downloaded once and then cached by the browser.
- **Greater control** — CSS can do things HTML never could: shadows, gradients, animations, transforms, grid layout.
- **Responsive design** — media queries adapt the same HTML to mobile, tablet and desktop screens.
- **Accessibility** — content order stays logical for screen readers while the visual order can differ.
- **Multiple output formats** — separate style sheets can be supplied for screen, print and speech.

Limitations of CSS

- Browser inconsistencies still exist for newer features, so testing in more than one browser is necessary.
- CSS cannot read from a database or perform logic; it is not a programming language.
- There is no parent selector in the classic sense (although `:has()` now provides one in modern browsers).
- Large projects can accumulate unused and conflicting rules unless a naming convention is followed.

Versions of CSS

Version	Year	Contribution
CSS1	1996	Fonts, colour, text, margins, the box model.
CSS2 / CSS 2.1	1998 / 2011	Positioning, z-index, media types, tables, generated content.
CSS3	2011 →	Split into independent modules: selectors, backgrounds, borders, transitions, animations, Flexbox, Grid, media queries, custom properties.
CSS4 modules	continuous	There is no single "CSS4". Individual modules advance separately — for example Selectors Level 4, Grid Level 2 (subgrid), container queries, <code>:has()</code> .

3.1.2 Basic Syntax of CSS

Figure 3.2 Anatomy of a CSS rule

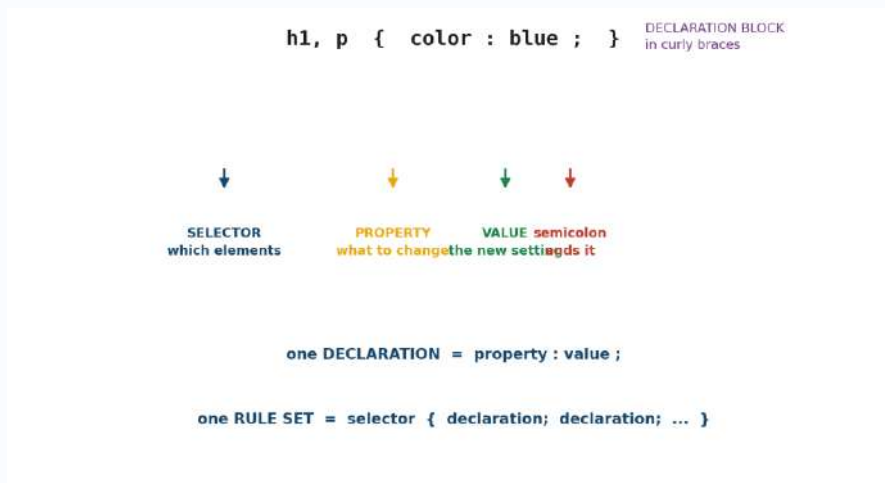


Figure 3.2 — Selector, declaration block, property and value.

CSS

/ This is a CSS comment. It is ignored by the browser. */*

```
h1 {
  color: #14456b;    /* property : value ; */
  font-size: 32px;
  text-align: center;
}
```

/ Grouping: one rule applied to several selectors */*

```
h2, h3, h4 {
  font-family: Arial, Helvetica, sans-serif;
  margin-bottom: 10px;
}
```

3.1.3 Levels of Style Sheets — Where You Can Add CSS

CSS can be attached to an HTML document in three ways, called the three **levels** of style sheet.

Figure 3.3 The three levels of style sheet and their priority

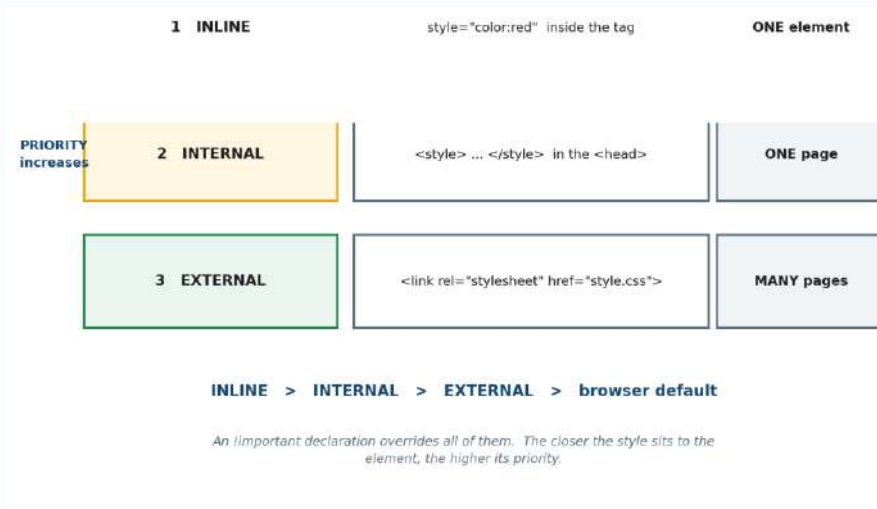


Figure 3.3 — The closer the style is to the element, the higher its priority.

(a) Inline style sheet

HTML

```
<p style="color: red; font-size: 18px; text-align: center;">
  This paragraph is styled inline.
</p>
```

- **Advantage:** highest priority; useful for a quick test or a style generated dynamically by JavaScript or PHP.
- **Disadvantage:** applies to one element only, mixes presentation into the markup, cannot be reused, cannot be cached, and cannot use pseudo-classes or media queries.

(b) Internal (embedded) style sheet

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Internal CSS</title>
  <style>
    body { background-color: #f4f6f8; font-family: Arial, sans-serif; }
    h1 { color: #14456b; text-align: center; }
    p { line-height: 1.6; color: #333; }
  </style>
</head>
<body>
  <h1>Internal Style Sheet</h1>
  <p>These rules apply only to this single page.</p>
</body>
```

```
</html>
```

- **Advantage:** styles the whole page, no extra HTTP request, useful for a one-page document or for critical above-the-fold CSS.
- **Disadvantage:** cannot be shared between pages; the rules are downloaded again with every page.

(c) External style sheet — the recommended method

```
CSS
/* ----- file: css/style.css ----- */
body {
  background-color: #f4f6f8;
  font-family: "Segoe UI", Arial, sans-serif;
  margin: 0;
}
h1 { color: #14456b; text-align: center; }
.card { background:#fff; padding:16px; border-radius:8px; }

HTML
<!-- ----- file: index.html ----- -->
<head>
  <meta charset="UTF-8">
  <title>External CSS</title>
  <link rel="stylesheet" href="css/style.css">
</head>
```

- **Advantage:** one file styles the entire website; the browser caches it so later pages load faster; the HTML stays clean; the designer and the developer can work independently.
- **Disadvantage:** one extra HTTP request; the page may briefly appear unstyled if the file loads slowly.

The `@import` rule — a fourth way

```
CSS
<style>
  @import url("css/base.css");
  @import url("css/theme.css");
</style>
```

@import must be the first rule in a style sheet. It is generally avoided because imports load sequentially rather than in parallel, which makes pages slower than using several `<link>` elements.

Comparison of the three levels

Basis	Inline	Internal (Embedded)	External
Written where	In the <code>style</code> attribute of a tag	Inside <code><style></code> in the <code><head></code>	In a separate <code>.css</code> file
Scope	One element only	One page	Unlimited pages
Priority	Highest	Medium	Lowest
Reusability	None	Within the page only	Across the whole site

Basis	Inline	Internal (Embedded)	External
Caching	Not cached	Not cached separately	Cached by the browser
Effect on file size	Increases HTML size a lot	Increases HTML size	Keeps HTML small
Maintenance	Very difficult	Moderate	Easy
Pseudo-classes / media queries	Not possible	Possible	Possible
Recommended for	Quick tests, JavaScript-generated styles, HTML e-mail	Single-page documents, critical CSS	Almost all real projects

3.1.4 CSS Selectors

A **selector** is the pattern that decides which HTML elements a rule applies to. Selectors are the heart of CSS.

Basic selectors

Selector	Syntax	Selects	Example
Universal	*	Every element in the document	* { margin: 0; padding: 0; }
Type (element)	tagname	All elements of that type	p { color: grey; }
Class	.classname	All elements carrying that class	.warning { color: red; }
ID	#idname	The single element with that id	#header { height: 80px; }
Grouping	s1, s2, s3	All the listed selectors at once	h1, h2, h3 { font-weight: 700; }

Combinator selectors

Name	Syntax	Meaning	Example
Descendant	A B	Every B anywhere inside A, at any depth	div p — every paragraph inside a division
Child	A > B	Only a B that is a direct child of A	ul > li — top-level list items only
Adjacent sibling	A + B	The single B that comes immediately after A	h2 + p — the first paragraph after a heading
General sibling	A ~ B	Every B that follows A and shares the same parent	h2 ~ p — all following paragraphs

Attribute selectors

Syntax	Selects elements whose attribute...
[type]	exists at all
[type="text"]	is exactly equal to the value

Syntax	Selects elements whose attribute...
<code>[class~="btn"]</code>	contains the given word in a space-separated list
<code>[href^="https"]</code>	begins with the value (^ = starts with)
<code>[href\$=".pdf"]</code>	ends with the value (\$ = ends with)
<code>[href*="gandaki"]</code>	contains the value anywhere (* = contains)

Pseudo-classes — selecting by state

Pseudo-class	Selects an element that...
<code>:link</code>	is an unvisited hyperlink
<code>:visited</code>	is a hyperlink the user has already visited
<code>:hover</code>	is currently under the mouse pointer
<code>:focus</code>	currently has keyboard focus — the cursor is in it, or it was reached with the Tab key. Essential for form accessibility.
<code>:active</code>	is being activated at this instant — the mouse button is held down on it
<code>:focus-visible</code>	has focus and the browser judges that a focus ring should be shown (keyboard, not mouse)
<code>:first-child</code> / <code>:last-child</code>	is the first or last child of its parent
<code>:nth-child(n)</code>	is the nth child; accepts odd , even or formulas such as 3n+1
<code>:checked</code>	is a checked radio button or check box
<code>:disabled</code> / <code>:enabled</code>	is a disabled or enabled form control
<code>:required</code> / <code>:optional</code>	is a required or optional form field
<code>:valid</code> / <code>:invalid</code>	currently satisfies or violates its validation constraints
<code>:not(selector)</code>	does not match the given selector

The `:focus` and `:active` pseudo-classes in practice

```

CSS
/* :hover -> mouse is over the element */
/* :focus -> element has keyboard focus (Tab or click) */
/* :active -> element is being pressed right now */

input[type="text"], input[type="email"], textarea {
  border: 1px solid #b9c6d4;
  padding: 8px;
  transition: border-color .2s, box-shadow .2s;
}

input:focus, textarea:focus {
  border-color: #1f6091;
  box-shadow: 0 0 0 3px rgba(31, 96, 145, 0.25);

```

```
outline: none;          /* replaced by the box-shadow above */
}

.btn { background:#14456b; color:#fff; border:none; padding:10px 20px; }
.btn:hover { background:#1f6091; }
.btn:focus { outline: 3px solid #f1c40f; outline-offset: 2px; }
.btn:active { background:#0d2f4a; transform: translateY(1px); }
```

Common Mistake: Never write `outline: none;` on a focusable element without providing a visible replacement. Keyboard users depend on the focus indicator to know where they are on the page, and removing it makes a site unusable for them.

Pseudo-elements — selecting part of an element

Pseudo-element	Selects
<code>::before</code>	A generated box inserted before the element's content. Needs the <code>content</code> property.
<code>::after</code>	A generated box inserted after the element's content.
<code>::first-line</code>	The first rendered line of a block of text.
<code>::first-letter</code>	The first letter — used for drop caps.
<code>::selection</code>	The portion of text currently highlighted by the user.
<code>::placeholder</code>	The placeholder text inside an input or text area.

```
CSS

p::first-letter { font-size: 300%; color: #14456b; float: left;
                  line-height: 1; margin-right: 6px; }
.required::after { content: " *"; color: #c0392b; }
a[href^="https"]::before { content: "\1F517 "; }
::selection { background: #14456b; color: #fff; }
input::placeholder { color: #9aa7b4; font-style: italic; }
```

Note: Use one colon for pseudo-classes (`:hover`) and two colons for pseudo-elements (`::before`). One colon still works on pseudo-elements for backward compatibility, but two is correct.

3.1.5 The Cascade, Specificity and Inheritance

The cascade

When several declarations set the same property on the same element, the browser resolves the conflict in this order:

- **1. Importance and origin** — a declaration marked `!important` wins over a normal one. Author styles beat user styles, which beat browser defaults.
- **2. Specificity** — the more specific selector wins.

- **3. Source order** — if importance and specificity are equal, the rule written **last** wins.

Calculating specificity

Specificity is written as four numbers, **a, b, c, d**, counted as follows:

Position	Counts	Weight
a	Inline style attribute	1000
b	Number of ID selectors (#id)	100 each
c	Number of class, attribute and pseudo-class selectors (.cls , [type] , :hover)	10 each
d	Number of type and pseudo-element selectors (p , ::before)	1 each

Figure 3.4 How CSS specificity is calculated

SPECIFICITY = (a , b , c , d)			
a inline	b #id	c .class, [attr], :hover count	d element, ::before total
* { }		0 0 0 0	0
p { }		0 0 0 1	1
div p { }		0 0 0 2	2
.note { }		0 0 1 0	10
p.note { }		0 0 1 1	11
#main { }		0 1 0 0	100
#main .note p { }		0 1 1 1	111
style="..."		1 0 0 0	1000

HIGHER TOTAL WINS. If two totals tie, the rule written LAST wins.

Figure 3.4 — Specificity decides which conflicting rule is applied.

HTML

```
<!-- Which colour does this paragraph get? -->
<div id="main">
  <p class="note">What colour am I?</p>
</div>
```

CSS

```
p      { color: black; } /* specificity 1 */
.note   { color: green; } /* specificity 10 */
div p.note { color: blue; } /* specificity 12 */
#main .note { color: orange; } /* specificity 110 <-- WINNER */

/* Result: the paragraph is ORANGE. */
```

Inheritance

Some properties are passed down automatically from a parent element to its children; others are not.

Inherited properties	Non-inherited properties
color, font-family, font-size, font-weight, font-style, line-height, letter-spacing, text-align, text-indent, visibility, list-style, cursor	margin, padding, border, width, height, background, display, position, top/right/bottom/left, float, overflow, z-index

Inheritance can be forced or cancelled with the keywords `inherit`, `initial`, `unset` and `revert`:

CSS

```
body { font-family: Arial; color: #333; } /* children inherit both */
button { font-family: inherit; } /* force inheritance */
.reset { color: initial; } /* back to default value */
```

Common Mistake: Avoid `!important`. It short-circuits the cascade and makes later maintenance extremely difficult, because the only way to beat one `!important` is with another. Fix the specificity of your selectors instead.

3.2 Style Specification Formats, Style Classes, Properties and Values, Colour, Positioning and Layout

3.2.1 Colour in CSS

Colour can be specified in several formats. All of them can be used with any colour property such as `color`, `background-color`, `border-color` and `box-shadow`.

Format	Syntax	Example	Notes
Keyword	name	<code>color: red;</code>	About 148 named colours: red, blue, tomato, teal, navy, gold.
Hexadecimal	<code>#RRGGBB</code>	<code>color: #14456b;</code>	Two hex digits (00–FF) per channel. The most common format.
Short hex	<code>#RGB</code>	<code>color: #f60;</code>	Shorthand for <code>#ff6600</code> when each pair repeats.
Hex with alpha	<code>#RRGGBBAA</code>	<code>#14456b80</code>	The last pair is opacity, 00 (transparent) to FF (opaque).
RGB	<code>rgb(r,g,b)</code>	<code>rgb(20, 69, 107)</code>	Each channel 0–255.
RGBA	<code>rgba(r,g,b,a)</code>	<code>rgba(20,69,107,.5)</code>	Alpha 0 (transparent) to 1 (opaque).
HSL	<code>hsl(h,s%,l%)</code>	<code>hsl(206, 68%, 25%)</code>	Hue 0–360°, saturation and lightness in per cent. Easy to create colour schemes.
HSLA	<code>hsla(h,s%,l%,a)</code>	<code>hsla(206,68%,25%,.6)</code>	HSL with alpha.
<code>transparent</code>	keyword	<code>background: transparent;</code>	Fully transparent.
<code>currentColor</code>	keyword	<code>border-color: currentColor;</code>	Reuses the element's own <code>color</code> value.

CSS

```

:root {
    /* CSS custom properties (variables) */
    --brand: #14456b;
    --brand-light: #1f6091;
    --accent: #f1c40f;
    --text: #1a1a1a;
    --bg: #f4f6f8;
}
body { background: var(--bg); color: var(--text); }
h1, h2 { color: var(--brand); }
.btn { background: var(--brand); color: #fff; }
.btn:hover { background: var(--brand-light); }

```

Note: CSS custom properties (variables) are declared on `:root` and used with `var()`. They make a colour scheme changeable from a single place and are fully supported in every modern browser.

3.2.2 Text and Font Properties

Property	Purpose	Common values
<code>color</code>	Colour of the text itself	any colour value
<code>font-family</code>	Typeface, with fallbacks separated by commas	"Segoe UI", Arial, sans-serif
<code>font-size</code>	Size of the text	16px, 1.2rem, 120%, larger
<code>font-weight</code>	Thickness	normal, bold, 100–900
<code>font-style</code>	Slant	normal, italic, oblique
<code>font-variant</code>	Small capitals	normal, small-caps
<code>font</code>	Shorthand for style, variant, weight, size/line-height, family	italic bold 16px/1.5 Arial
<code>text-align</code>	Horizontal alignment	left, right, center, justify
<code>text-decoration</code>	Lines on the text	none, underline, line-through, overline
<code>text-transform</code>	Letter case	uppercase, lowercase, capitalize, none
<code>text-indent</code>	Indent of the first line	30px, 2em
<code>line-height</code>	Vertical spacing between lines	1.6 (unitless is best), 24px
<code>letter-spacing</code>	Space between characters	1px, .05em
<code>word-spacing</code>	Space between words	4px
<code>text-shadow</code>	Shadow behind the text	2px 2px 4px rgba(0,0,0,.4)
<code>text-overflow</code>	How clipped text is signalled	clip, ellipsis
<code>white-space</code>	How whitespace is handled	normal, nowrap, pre, pre-wrap
<code>vertical-align</code>	Vertical alignment of inline content	baseline, middle, top, sub, super

Generic font families and web fonts

- **serif** — letters have small finishing strokes: Times New Roman, Georgia. Traditional and good for long printed text.
- **sans-serif** — no finishing strokes: Arial, Helvetica, Segoe UI. Clean on screen; the usual choice for web pages.
- **monospace** — every character has the same width: Consolas, Courier New. Used for code.
- **cursive** and **fantasy** — decorative; used rarely and only for headings.

CSS

/ Always list a specific font first, then a generic family last. */*

```
body { font-family: "Segoe UI", Roboto, Arial, sans-serif; }
code { font-family: Consolas, "Courier New", monospace; }
```

/ Loading a custom web font */*

```
@font-face {
  font-family: "MyFont";
  src: url("fonts/myfont.woff2") format("woff2");
  font-display: swap;
}
h1 { font-family: "MyFont", Georgia, serif; }
```

CSS units

Unit	Type	Meaning	When to use
px	Absolute	One CSS pixel	Borders, small fixed details
%	Relative	Percentage of the parent element	Fluid widths
em	Relative	Multiple of the parent font size	Padding and margins that scale with text
rem	Relative	Multiple of the root (html) font size	Font sizes and spacing — the modern default
vw / vh	Relative	1 % of the viewport width / height	Full-screen sections, hero banners
vmin / vmax	Relative	1 % of the smaller / larger viewport dimension	Responsive typography
ch	Relative	Width of the character "0"	Limiting line length for readability
pt, cm, in	Absolute	Print units	Print style sheets only

3.2.3 The CSS Box Model

Definition

The **CSS box model** describes how every HTML element is rendered as a rectangular box made of four concentric layers: the **content**, the **padding**, the **border** and the **margin**. Understanding this model is essential, because almost every layout problem in CSS comes from misunderstanding it.

Figure 3.5 The CSS box model

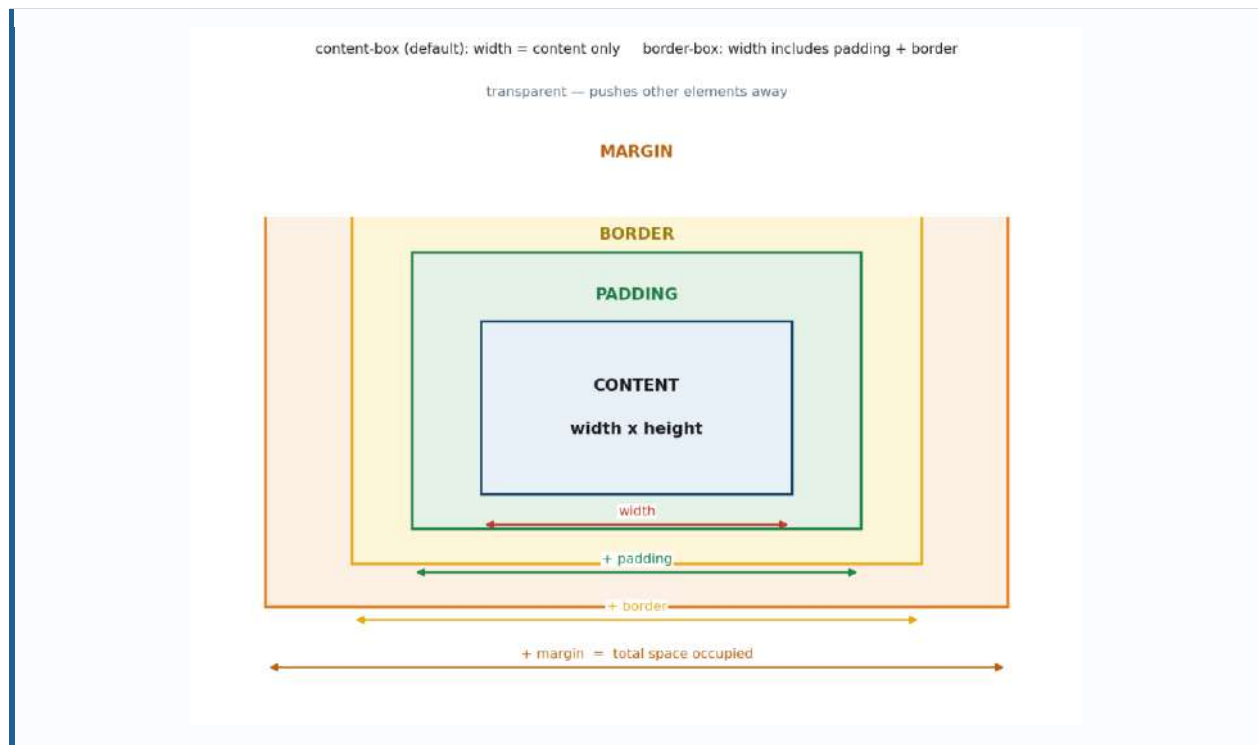


Figure 3.5 — Content, padding, border and margin.

Worked calculation

CSS

```
.box {
  width: 300px;
  padding: 20px;
  border: 5px solid navy;
  margin: 10px;
}
```

/ With the DEFAULT box-sizing: content-box*

visible width = $300 + 20 + 20 + 5 + 5 = 350\text{px}$
 space taken = $350 + 10 + 10 = 370\text{px}$ **/*

/ With box-sizing: border-box*

visible width = 300px exactly (padding and border fit inside)
 content width = $300 - 40 - 10 = 250\text{px}$ **/*

**, *::before, *::after { box-sizing: border-box; }*

/ ^ put this at the top of every style sheet you write */*

Margin and padding shorthand

Declaration	Top	Right	Bottom	Left
<code>margin: 10px;</code>	10	10	10	10
<code>margin: 10px 20px;</code>	10	20	10	20
<code>margin: 10px 20px 30px;</code>	10	20	30	20

Declaration	Top	Right	Bottom	Left
<code>margin: 10px 20px 30px 40px;</code>	10	20	30	40

Note: Values in the four-value shorthand run clockwise from the top: **T R B L** — remember it as "TRouBLe". `margin: 0 auto;` sets top and bottom to zero and centres a block element horizontally.

Border properties

```
CSS
.card {
  border: 2px solid #14456b; /* width | style | colour */
  border-radius: 10px; /* rounded corners */
  border-bottom: 4px dashed red; /* one side only */
  box-shadow: 0 4px 12px rgba(0,0,0,.15);
}
/* styles: none, solid, dashed, dotted, double, groove, ridge,
inset, outset, hidden */
```

Background properties

```
CSS
.hero {
  background-color: #14456b;
  background-image: url("images/campus.jpg");
  background-repeat: no-repeat; /* repeat | repeat-x | repeat-y */
  background-position: center center;
  background-size: cover; /* cover | contain | 100% 300px */
  background-attachment: fixed; /* parallax effect */
}

/* shorthand */
.hero { background: #14456b url("images/campus.jpg") no-repeat
center/cover fixed; }

/* gradient background — no image file needed */
.banner { background: linear-gradient(135deg, #14456b, #1f6091); }
```

3.2.4 The `display` Property

Value	Effect
<code>block</code>	Starts on a new line and takes the full available width. Width and height apply.
<code>inline</code>	Flows within the line. Width, height and vertical margins are ignored.
<code>inline-block</code>	Flows within the line and accepts width, height and all margins. Ideal for buttons and menu items.
<code>none</code>	The element is removed from the page completely; it occupies no space.
<code>flex</code>	Makes the element a flex container — one-dimensional layout.

Value	Effect
<code>grid</code>	Makes the element a grid container — two-dimensional layout.
<code>table</code> , <code>table-row</code> , <code>table-cell</code>	Makes non-table elements behave like table parts.

Common Mistake: `display: none` removes the element entirely, while `visibility: hidden` hides it but leaves its space empty. Use `display:none` for menus that are toggled and `visibility:hidden` when the layout must not shift.

3.2.5 Positioning

Definition

The `position` property determines how an element is placed in the document, and whether the offset properties `top`, `right`, `bottom` and `left` have any effect.

Value	Behaviour	Offsets work?	Removed from flow?
<code>static</code>	The default. The element sits in the normal document flow.	No	No
<code>relative</code>	Positioned relative to where it <i>would</i> have been. The original space is preserved.	Yes	No
<code>absolute</code>	Positioned relative to the nearest positioned ancestor (an ancestor whose position is not static); if there is none, relative to the initial containing block.	Yes	Yes
<code>fixed</code>	Positioned relative to the browser viewport. Stays in place while the page scrolls.	Yes	Yes
<code>sticky</code>	Behaves as relative until a scroll threshold is crossed, then behaves as fixed within its parent.	Yes	No

Figure 3.6 The five positioning schemes

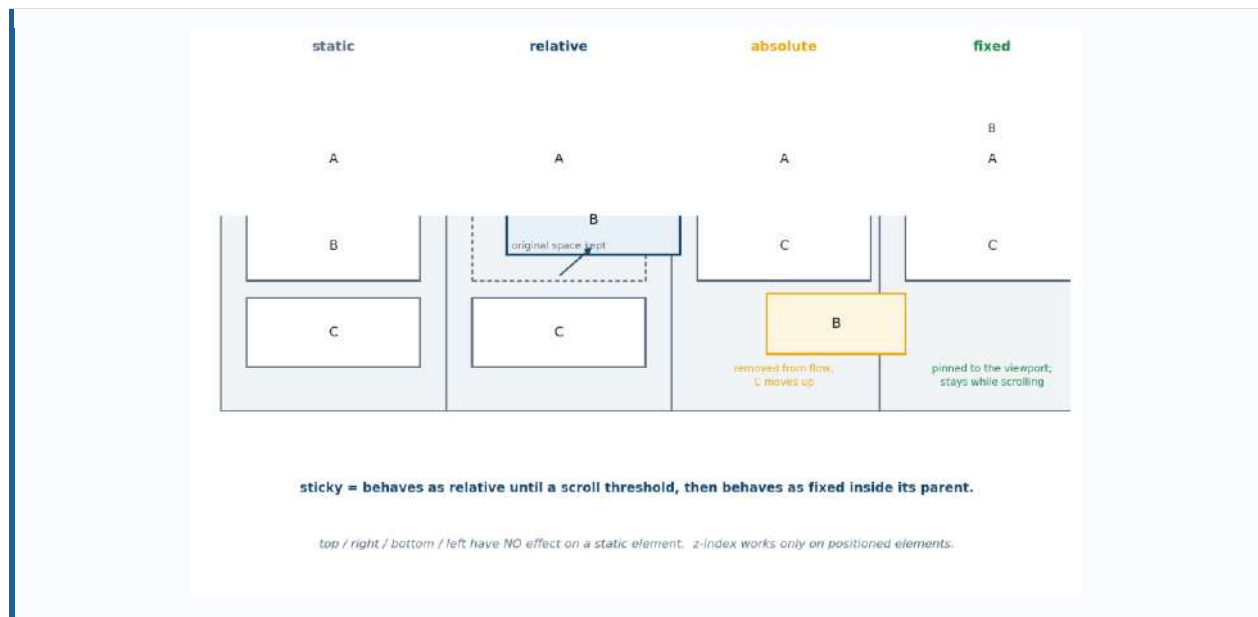


Figure 3.6 — relative keeps its space, absolute and fixed do not.

CSS

/ A common, practical combination */*

```
.card { position: relative; } /* becomes the reference */
.card .tag { position: absolute; top: 10px; right: 10px; }
```

```
.navbar { position: fixed; top: 0; left: 0; width: 100%;
          z-index: 1000; }
.sidebar h3 { position: sticky; top: 0; background: #fff; }
```

/ z-index only works on positioned elements (not static) */*

```
.modal { position: fixed; inset: 0; z-index: 9999; }
```

Float and clear

float removes an element from the normal flow and pushes it to the left or right, allowing text to wrap around it. It was the main layout tool before Flexbox; today it is used mainly for wrapping text around images.

CSS

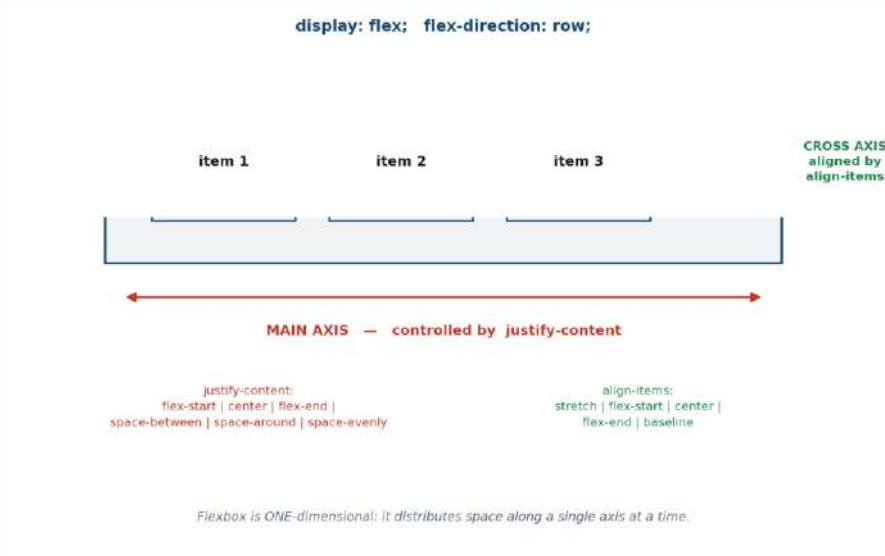
```
img.left { float: left; margin: 0 15px 10px 0; }
img.right { float: right; margin: 0 0 10px 15px; }
.clear { clear: both; } /* stop wrapping */
```

/ Modern clearfix so a parent contains its floated children */*

```
.row::after { content: ""; display: block; clear: both; }
```

3.2.6 Layout with Flexbox

Flexbox is a one-dimensional layout system that distributes space along a single axis — either a row or a column. It is the easiest reliable way to build navigation bars, card rows and centred content.

Figure 3.7 Flexbox axes and alignment*Figure 3.7 — justify-content works along the main axis; align-items works across it.***CSS**

```
.navbar {
  display: flex;
  justify-content: space-between; /* logo left, menu right */
  align-items: center;
  gap: 20px;
  padding: 12px 24px;
}

.cards {
  display: flex;
  flex-wrap: wrap; /* allow items to move to a new line */
  gap: 16px;
}

.cards .card {
  flex: 1 1 280px; /* grow | shrink | basis */
}

/* Perfect centring — the classic problem, solved in three lines */
.center-box {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}
```

3.2.7 Layout with CSS Grid

CSS Grid is a two-dimensional layout system that controls rows and columns at the same time. It is the right tool for whole-page layouts.

CSS

```
.page {
```

```

display: grid;
grid-template-columns: 220px 1fr 260px; /* 3 columns */
grid-template-rows: auto 1fr auto; /* 3 rows */
grid-template-areas:
  "header header header"
  "nav main aside"
  "footer footer footer";
gap: 16px;
min-height: 100vh;
}
.page > header { grid-area: header; }
.page > nav { grid-area: nav; }
.page > main { grid-area: main; }
.page > aside { grid-area: aside; }
.page > footer { grid-area: footer; }

/* A responsive card grid with no media query at all */
.gallery {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
gap: 14px;
}

```

Difference between Flexbox and Grid

Basis	Flexbox	CSS Grid
Dimension	One-dimensional — a row or a column	Two-dimensional — rows and columns together
Approach	Content-first: item sizes drive the layout	Layout-first: the grid is defined, then items are placed
Best for	Navigation bars, toolbars, card rows, centring	Whole page layouts, dashboards, image galleries
Gaps	gap supported	gap supported
Overlapping items	Difficult	Easy, using explicit row and column lines
Typical use together	Grid for the page skeleton, Flexbox inside each grid area	They complement rather than compete

3.2.8 Responsive Design and Media Queries

Responsive web design is the practice of building one set of HTML that automatically adapts its layout to any screen size, from a small phone to a large desktop monitor. The three ingredients are a flexible grid, flexible images and **media queries**.

HTML

```

<!-- Step 1: this meta tag is compulsory, in the <head> -->
<meta name="viewport" content="width=device-width, initial-scale=1.0">

```

CSS

```

/* Step 2: flexible images */

```

```
img { max-width: 100%; height: auto; }

/* Step 3: media queries — mobile-first, smallest styles first */
.container { width: 92%; margin: 0 auto; }
.cards { display: grid; grid-template-columns: 1fr; gap: 14px; }

@media screen and (min-width: 600px) { /* tablet and above */
  .cards { grid-template-columns: repeat(2, 1fr); }
}

@media screen and (min-width: 992px) { /* desktop and above */
  .container { width: 960px; }
  .cards { grid-template-columns: repeat(3, 1fr); }
}

@media print { /* printing */
  nav, .no-print { display: none; }
  body { color: #000; font-size: 12pt; }
}
```

Device category	Typical breakpoint	Media query
Mobile phone	up to 599 px	base styles (no query needed)
Tablet	600 px and above	<code>@media (min-width: 600px)</code>
Small laptop	992 px and above	<code>@media (min-width: 992px)</code>
Large desktop	1200 px and above	<code>@media (min-width: 1200px)</code>

Exam Tip: Write **mobile-first** CSS: put the phone layout in the base rules and use **min-width** queries to add complexity for larger screens. This produces shorter, simpler CSS than starting from the desktop with **max-width** queries.

3.2.9 Transitions, Transforms and Animation

```
CSS

/* TRANSITION: smoothly animate a change of state */
.btn {
  background: #14456b; color:#fff; padding:10px 20px; border:none;
  transition: background .3s ease, transform .2s ease;
}
.btn:hover { background:#1f6091; transform: translateY(-2px); }
.btn:active { transform: translateY(1px); }

/* TRANSFORM: move, scale, rotate or skew without affecting layout */
.thumb:hover { transform: scale(1.08) rotate(2deg); }

/* ANIMATION: a named sequence of keyframes */
@keyframes fadeInUp {
  from { opacity: 0; transform: translateY(20px); }
  to { opacity: 1; transform: translateY(0); }
}
.card { animation: fadeInUp .6s ease-out both; }
```

3.2.10 Complete Worked Example — A Fully Styled Responsive Page

This example combines everything in the chapter: an external style sheet, variables, the box model, Flexbox, Grid, pseudo-classes including `:focus` and `:active`, positioning and a media query.

HTML

```
<!-- ===== file: index.html ===== -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gandaki University — Department of IT</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>

  <header class="site-header">
    <div class="logo">Gandaki University</div>
    <nav>
      <ul class="menu">
        <li><a href="#home" class="active">Home</a></li>
        <li><a href="#courses">Courses</a></li>
        <li><a href="#faculty">Faculty</a></li>
        <li><a href="#contact">Contact</a></li>
      </ul>
    </nav>
  </header>

  <section class="hero" id="home">
    <h1>Department of Information Technology</h1>
    <p>Bachelor of Information Technology &mdash; Semester II</p>
    <a href="#courses" class="btn">Explore Courses</a>
  </section>

  <main class="container" id="courses">
    <h2>Our Courses</h2>
    <div class="cards">
      <article class="card">
        <h3>Web Technology I</h3>
        <p>HTML, CSS, JavaScript, PHP and MySQL for beginners.</p>
        <span class="tag">3 Cr</span>
      </article>
      <article class="card">
        <h3>Data Structures</h3>
        <p>Arrays, linked lists, stacks, queues, trees and graphs.</p>
        <span class="tag">3 Cr</span>
      </article>
      <article class="card">
        <h3>Discrete Mathematics</h3>
        <p>Logic, sets, relations, counting and graph theory.</p>
        <span class="tag">3 Cr</span>
      </article>
    </div>
  </main>

  <footer class="site-footer">
    <p>&copy; 2026 Gandaki University, Pokhara, Nepal</p>
  </footer>
```

```
</body>
</html>
```

CSS

```
/* ===== file: css/style.css ===== */
:root {
  --brand: #14456b;
  --brand2: #1f6091;
  --accent: #f1c40f;
  --text: #1a1a1a;
  --bg: #f4f6f8;
  --radius: 10px;
}

*, *::before, *::after { box-sizing: border-box; }

body {
  margin: 0;
  font-family: "Segoe UI", Arial, sans-serif;
  line-height: 1.6;
  color: var(--text);
  background: var(--bg);
}

/* ----- header: Flexbox + fixed positioning ----- */
.site-header {
  position: sticky; top: 0; z-index: 100;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 14px 26px;
  background: var(--brand);
  color: #fff;
  box-shadow: 0 2px 8px rgba(0,0,0,.18);
}
.logo { font-size: 1.3rem; font-weight: 700; letter-spacing: .5px; }

.menu { list-style: none; display: flex; gap: 6px; margin: 0; padding: 0; }
.menu a {
  display: inline-block;
  padding: 8px 14px;
  color: #dfe9f3;
  text-decoration: none;
  border-radius: 6px;
  transition: background .25s, color .25s;
}
.menu a:hover { background: var(--brand2); color: #fff; }
.menu a:focus { outline: 3px solid var(--accent); outline-offset: 2px; }
.menu a:active { background: #0d2f4a; }
.menu a.active { background: var(--brand2); color: #fff; }

/* ----- hero: gradient + centring ----- */
.hero {
  background: linear-gradient(135deg, var(--brand), var(--brand2));
  color: #fff;
  text-align: center;
  padding: 70px 20px;
}
.hero h1 { margin: 0 0 10px; font-size: clamp(1.6rem, 4vw, 2.6rem); }
```

```

.btn {
  display: inline-block;
  margin-top: 18px;
  padding: 11px 26px;
  background: var(--accent);
  color: #1a1a1a;
  text-decoration: none;
  font-weight: 600;
  border-radius: var(--radius);
  transition: transform .2s, box-shadow .2s;
}
.btn:hover { transform: translateY(-2px);
             box-shadow: 0 6px 14px rgba(0,0,0,.22); }
.btn:active { transform: translateY(1px); box-shadow: none; }

/* ----- main content: Grid ----- */
.container { max-width: 1080px; margin: 0 auto; padding: 40px 20px; }
h2 { color: var(--brand); border-bottom: 3px solid var(--accent);
     display: inline-block; padding-bottom: 4px; }

.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
  margin-top: 24px;
}

.card {
  position: relative;          /* reference for the tag */
  background: #fff;
  padding: 22px;
  border-radius: var(--radius);
  border-top: 4px solid var(--brand2);
  box-shadow: 0 2px 10px rgba(0,0,0,.08);
  transition: transform .25s, box-shadow .25s;
}
.card:hover { transform: translateY(-5px);
              box-shadow: 0 10px 22px rgba(0,0,0,.14); }
.card h3 { margin-top: 0; color: var(--brand); }

.tag {
  position: absolute;          /* absolute inside .card */
  top: 14px; right: 14px;
  background: var(--accent);
  padding: 3px 10px;
  border-radius: 20px;
  font-size: .78rem;
  font-weight: 700;
}

/* ----- footer ----- */
.site-footer {
  background: var(--brand);
  color: #cfe0ee;
  text-align: center;
  padding: 18px;
  font-size: .9rem;
}

/* ----- responsive ----- */
@media screen and (max-width: 640px) {
  .site-header { flex-direction: column; gap: 10px; }
  .menu { flex-wrap: wrap; justify-content: center; }

```

```
.hero { padding: 46px 16px; }  
}
```

3.3 Exam-Based Questions — Chapter 3

Group A — Very Short Answer Questions (2 marks each)

1. Define CSS. Write its full form and the file extension used.
2. Write the basic syntax of a CSS rule and label its parts.
3. What does the word "cascading" mean in Cascading Style Sheets?
4. Name the three levels of style sheet and write their order of priority.
5. Write the syntax for linking an external style sheet to an HTML page.
6. Differentiate between a class selector and an ID selector.
7. What is a universal selector? Write its syntax and one use.
8. Differentiate between the `:hover` and `:focus` pseudo-classes.
9. What is the `:active` pseudo-class? Give one example.
10. Differentiate between a pseudo-class and a pseudo-element.
11. Write any four ways of specifying a colour in CSS.
12. What is the CSS box model? Name its four layers.
13. Differentiate between `padding` and `margin`.
14. Differentiate between `display:none` and `visibility:hidden`.
15. Differentiate between the `em` and `rem` units.
16. What is the purpose of the viewport meta tag?
17. Write the syntax of a media query for screens wider than 768 pixels.
18. What is `!important`? Why should it be avoided?
19. What is the use of `box-sizing: border-box`?
20. What is a CSS custom property? Write one example.

Group B — Short Answer Questions (5 marks each)

1. What is CSS? Explain any six advantages of using CSS with suitable justification.
2. Explain the three levels of style sheet with syntax and a code example of each, and compare them in tabular form.
3. Explain any six types of CSS selector with syntax and an example of each.
4. Explain the combinator selectors (descendant, child, adjacent sibling and general sibling) with examples.
5. Explain attribute selectors with at least five examples.
6. What is specificity? Calculate the specificity of the following selectors and state which rule wins: `p`, `.note`, `div p.note`, `#main .note`.
7. Explain inheritance in CSS. Which properties are inherited and which are not? How can inheritance be forced or cancelled?

8. Explain the CSS box model with a labelled diagram and calculate the total rendered width of a box with `width:400px; padding:15px; border:5px; margin:10px` under both `content-box` and `border-box`.
9. Explain the `:link`, `:visited`, `:hover`, `:focus` and `:active` pseudo-classes with a complete code example for styling a navigation menu.
10. Explain the `position` property with all five of its values, using a diagram.
11. Differentiate between `relative` and `absolute` positioning with examples.
12. What is `float`? Explain `float` and `clear` with a code example, and explain the clearfix technique.
13. Explain Flexbox with its main properties and write code to build a horizontal navigation bar.
14. Explain CSS Grid and write code to create a three-column responsive card layout.
15. Differentiate between Flexbox and CSS Grid.
16. What is responsive web design? Explain media queries with a code example covering mobile, tablet and desktop.
17. Explain any eight text and font properties with syntax and examples.
18. Explain the different units used in CSS with their suitability.
19. Explain the background properties of CSS with a code example that uses an image, position, size and repeat.
20. Explain CSS transitions and transforms with a code example of a button hover effect.

Group C — Long Answer Questions (7.5 marks each)

1. What is CSS? Explain its advantages, describe the three levels of style sheet in detail with code examples, compare them in a table, and explain the order of precedence when all three define the same property.
2. Explain CSS selectors in complete detail. Cover basic, combinator, attribute, pseudo-class and pseudo-element selectors with syntax and examples, and explain how specificity resolves conflicts among them.
3. Explain the CSS box model in detail with a labelled diagram. Describe `margin`, `border`, `padding` and `content`, explain shorthand notation, explain margin collapsing, and demonstrate the difference between `content-box` and `border-box` with a numerical calculation.
4. Explain positioning and layout in CSS. Describe all five values of the `position` property with diagrams, explain `float` and `clear`, and compare Flexbox and CSS Grid with code examples of each.
5. Design a complete responsive web page for a college department. Write both the HTML and the external CSS, using a Flexbox navigation bar, a Grid card layout, the `:hover`, `:focus` and `:active` pseudo-classes, absolute positioning for a badge, and at least two media queries.
6. Explain how a browser decides which style to apply when several rules conflict. Cover the cascade, origin, importance, specificity calculation with worked examples, source order and inheritance.

Laboratory / Practical Exercises — Chapter 3

1. Take the HTML pages you created in Chapter 2 and style all of them from a single external style sheet.
2. Create one page that demonstrates inline, internal and external CSS on the same property and observe which one wins. Write your observation.
3. Build a horizontal navigation bar with hover, focus and active effects, using only CSS.

4. Create a card layout of at least six cards using CSS Grid that automatically becomes one column on a phone, two on a tablet and three on a desktop.
5. Style the registration form of Chapter 2 with `:focus` highlighting, `:invalid` red borders and a styled submit button.
6. Create a page that demonstrates all five positioning values, with a sticky header and a fixed "back to top" button.
7. Build a photo gallery with a hover zoom effect using `transform: scale()` and `transition`.
8. Validate your CSS with the W3C CSS Validation Service and correct every error reported.

Chapter 4: Learning JavaScript

Allotted time: 10 hours

Note: HTML is the skeleton, CSS is the skin, and JavaScript is the muscle. It is the only programming language that runs natively inside every web browser. In this chapter your pages stop being static documents and start responding to the user.

Chapter Learning Outcomes

After completing this chapter you will be able to:

- Explain what JavaScript is, where it runs, and how to include it in a page.
- Declare variables correctly with `let` and `const`, and explain scope and hoisting.
- Use every category of operator and write conditional and looping statements.
- Write and call functions, including function expressions and arrow functions.
- Explain the Document Object Model and select, read and modify any element on a page.
- Handle events using all three models and explain bubbling and capturing.
- Change the colour, font, style, content and position of elements dynamically.
- Write a complete client-side form-validation script.

4.1 Introduction to JavaScript

4.1.1 Definition

JavaScript is a high-level, interpreted, object-based, event-driven, loosely typed programming language used to make web pages interactive and dynamic. It was created by **Brendan Eich at Netscape in 1995**, originally under the names Mocha and LiveScript. It was standardised by ECMA International as **ECMAScript**, whose major revision **ES6 (ECMAScript 2015)** introduced most of the modern syntax used today.

Characteristics of JavaScript

Characteristic	Explanation
Client-side (and server-side)	It normally runs inside the visitor's browser, so no round trip to the server is needed. With Node.js it also runs on the server.
Interpreted / JIT compiled	No separate compilation step; the browser's engine (V8 in Chrome, SpiderMonkey in Firefox) executes the source directly.
Loosely (dynamically) typed	A variable's type is decided at run time and can change: <code>let x = 5; x = "hello";</code> is legal.
Case sensitive	<code>myName</code> , <code>MyName</code> and <code>myname</code> are three different identifiers.
Event driven	Code runs in response to things that happen — a click, a key press, a page load.

Characteristic	Explanation
Object based	Almost everything is an object; it supports objects and prototypal inheritance but is not a classical class-based language.
Single threaded	One statement at a time, with an event loop that manages asynchronous work.
Platform independent	The same script runs on any operating system that has a browser.

What JavaScript can do — its uses

- **Validate forms** before they are sent to the server, so the user is told about mistakes instantly.
- **Manipulate the page** — add, delete, hide, show and change any element or its style.
- **Respond to events** — clicks, key presses, mouse movement, scrolling, page loading.
- **Create animation and effects** — slideshows, image galleries, dropdown menus, modal dialogues.
- **Communicate with a server without reloading** the page (AJAX and the Fetch API).
- **Store data in the browser** using cookies, `localStorage` and `sessionStorage`.
- **Detect the browser and device**, and read the user's geographic location with permission.
- **Build complete applications** — with frameworks (React, Vue, Angular) on the client and Node.js on the server.

Limitations of JavaScript

- It cannot read or write arbitrary files on the user's computer, for security reasons.
- It cannot access a database directly; that needs a server-side language such as PHP.
- The code is visible to anyone, so it can never be trusted for security-critical checks.
- The user can disable JavaScript, so the site should still be usable without it wherever possible.
- Different browsers may support new features at different times.

Difference between Java and JavaScript

Basis	Java	JavaScript
Developed by	Sun Microsystems (now Oracle), 1995	Netscape (Brendan Eich), 1995
Type	Compiled, class-based, object-oriented language	Interpreted, object-based scripting language
Typing	Strongly and statically typed — the type must be declared	Loosely and dynamically typed
Executed by	The Java Virtual Machine	The browser's JavaScript engine
File extension	<code>.java</code> , compiled to <code>.class</code>	<code>.js</code>
Must be compiled	Yes, with <code>javac</code>	No

Basis	Java	JavaScript
Used for	Desktop, Android and enterprise applications	Web page interactivity, and servers with Node.js
Relationship	None. The similar name was a marketing decision of 1995.	None.

Common Mistake: "Java is to JavaScript as ham is to hamster." The two languages are unrelated. This comparison is a very common two-mark question.

4.1.2 Adding JavaScript to a Page

Exactly like CSS, JavaScript can be included in three ways.

HTML

```
<!-- 1. INLINE: inside an HTML event attribute (avoid in real work) -->
<button onclick="alert('Hello!')">Click me</button>

<!-- 2. INTERNAL: inside a <script> element -->
<script>
  function greet() {
    alert("Hello from an internal script!");
  }
</script>

<!-- 3. EXTERNAL: a separate .js file (RECOMMENDED) -->
<script src="js/script.js"></script>

<!-- defer: download in parallel, run after the HTML is parsed -->
<script src="js/script.js" defer></script>
```

Placement	Effect
In <code><head></code> without <code>defer</code>	The script runs before the body exists, so <code>document.getElementById()</code> returns <code>null</code> . Blocks page rendering.
At the end of <code><body></code>	The HTML is already parsed, so all elements are available. A simple, reliable choice.
In <code><head></code> with <code>defer</code>	Downloaded in parallel and executed after parsing completes, in document order. The best modern practice.
In <code><head></code> with <code>async</code>	Downloaded in parallel and executed as soon as it arrives, in unpredictable order. Suitable only for independent scripts such as analytics.

Output methods — how to display something

JAVASCRIPT

```
// 1. A dialogue box shown to the user
alert("This is an alert box.");
```

```
// 2. Ask the user to confirm; returns true or false
let ok = confirm("Do you really want to delete this record?");

// 3. Ask the user for input; returns a STRING, or null if cancelled
let name = prompt("What is your name?", "Guest");

// 4. Write directly into the document (only while the page loads)
document.write("<h2>Hello " + name + "</h2>");

// 5. Insert into a specific element (the normal, correct way)
document.getElementById("output").innerHTML = "<b>Welcome!</b>";

// 6. Print to the browser console — the developer's best friend (F12)
console.log("Value of name is:", name);
console.error("Something went wrong");
console.table([ {a:1,b:2}, {a:3,b:4} ]);
```

4.1.3 Basic Syntax Rules

- Statements normally end with a semicolon `;`. It is optional but strongly recommended.
- JavaScript is **case sensitive**.
- Single-line comments start with `//`; multi-line comments are written `/* ... */`.
- Identifiers may contain letters, digits, `_` and `$`, but must not start with a digit.
- The convention is **camelCase** for variables and functions (`studentName`), **PascalCase** for constructors (`Student`), and **UPPER_SNAKE_CASE** for constants (`MAX_MARKS`).
- Adding `"use strict"`; at the top of a file enables strict mode, which turns silent mistakes into errors.

4.2 Variables, Data Types and Scope

4.2.1 Declaring Variables

A **variable** is a named container that stores a value in memory. JavaScript offers three declaration keywords.

JAVASCRIPT

```
var count = 10;    // old, function-scoped -> avoid in new code
let score = 85;    // ES6, block-scoped, can be reassigned
const PI = 3.14159; // ES6, block-scoped, CANNOT be reassigned

score = 90;        // OK
// PI = 3.14;      // TypeError: Assignment to constant variable

// const on an object: the BINDING is fixed, the CONTENTS are not
const student = { name: "Sita" };
student.name = "Ram"; // allowed — we changed a property
// student = {};     // NOT allowed — we tried to rebind

let a = 1, b = 2, c = 3; // several declarations in one statement
let x;                  // declared but not initialised -> undefined
```

Difference between `var`, `let` and `const`

Basis	var	let	const
Introduced	Original JavaScript	ES6 (2015)	ES6 (2015)
Scope	Function scope	Block scope { }	Block scope { }
Re-declaration in the same scope	Allowed	Not allowed	Not allowed
Re-assignment	Allowed	Allowed	Not allowed
Hoisting	Hoisted and initialised to undefined	Hoisted but in the "temporal dead zone" — using it first throws an error	Same as let
Must be initialised	No	No	Yes , at the moment of declaration
Recommendation	Avoid	Use when the value will change	Use by default

JAVASCRIPT

// SCOPE demonstration

```
function demo() {
  if (true) {
    var v = "I am var";    // function scoped
    let l = "I am let";    // block scoped
  }
  console.log(v);          // "I am var" -> works
  // console.log(l);        // ReferenceError -> l is not defined
}
```

// HOISTING demonstration

```
console.log(hoisted);      // undefined (declaration was moved up)
var hoisted = 5;

// console.log(notHoisted); // ReferenceError: Cannot access before
// let notHoisted = 5;      // initialization
```

Types of scope

Scope	Meaning	Example
Global scope	Declared outside every function and block; accessible everywhere in the script.	let siteName = "GU";
Function (local) scope	Declared inside a function; accessible only within that function.	function f(){ let x = 1; }
Block scope	Declared with let or const inside { }; accessible only within those braces.	if (true){ let y = 2; }
Lexical scope	An inner function can read the variables of the function that encloses it. This is what makes closures possible.	function outer(){ let a=1; function inner(){ return a; } }

4.2.2 Data Types

JavaScript has two categories of data type: **primitive** (a single immutable value) and **non-primitive / reference** (a collection).

Type	Category	Description	Example
number	Primitive	All numbers — integer and floating point, plus Infinity and NaN	let n = 42; let f = 3.14;
string	Primitive	A sequence of characters in single quotes, double quotes or back-ticks	let s = "Hello";
boolean	Primitive	Only true or false	let ok = true;
undefined	Primitive	A variable that has been declared but never assigned	let u;
null	Primitive	A deliberate "no value". Note that typeof null returns "object" , a long-standing bug in the language.	let empty = null;
bigint	Primitive	Integers larger than Number.MAX_SAFE_INTEGER	let big = 9007199254740993n;
symbol	Primitive	A unique identifier, used mainly as an object key	let id = Symbol("id");
object	Reference	A collection of key–value pairs	let o = {name:"Ram", age:20};
array	Reference	An ordered list; technically an object	let a = [10, 20, 30];
function	Reference	A callable object	function f(){}

JAVASCRIPT

// Checking a type

```
console.log(typeof 42);      // "number"
console.log(typeof "hi");   // "string"
console.log(typeof true);   // "boolean"
console.log(typeof undefined); // "undefined"
console.log(typeof null);   // "object" <-- famous quirk
console.log(typeof [1,2,3]); // "object"
console.log(Array.isArray([1,2])); // true <-- correct test
```

// TYPE CONVERSION

```
let str = "25";
console.log(Number(str) + 5); // 30 explicit conversion
console.log(parseInt("25.9")); // 25
console.log(parseFloat("25.9")); // 25.9
console.log(String(25) + 5); // "255"
console.log("25" + 5); // "255" implicit — + concatenates
console.log("25" - 5); // 20 implicit — - converts
```

// EQUALITY: always prefer ===

```
console.log(5 == "5"); // true (loose: converts type)
console.log(5 === "5"); // false (strict: type must match)
```

Difference between `==` and `===`

Basis	== (loose equality)	=== (strict equality)
Name	Equality operator	Identity / strict equality operator

Basis	<code>==</code> (loose equality)	<code>===</code> (strict equality)
Type checking	Converts the operands to a common type first	Compares value and type without conversion
<code>5 == "5"</code>	<code>true</code>	<code>false</code>
<code>0 == false</code>	<code>true</code>	<code>false</code>
<code>null == undefined</code>	<code>true</code>	<code>false</code>
Recommendation	Avoid — it hides bugs	Always use this

Strings, arrays and objects — the essentials

JAVASCRIPT

// ----- STRINGS -----

```
let name = "Gandaki University";
console.log(name.length); // 19
console.log(name.toUpperCase()); // "GANDAKI UNIVERSITY"
console.log(name.indexOf("Uni")); // 8
console.log(name.slice(0, 7)); // "Gandaki"
console.log(name.replace("Gandaki", "Pokhara"));
console.log(" trim me ".trim()); // "trim me"
console.log(name.split(" ")); // ["Gandaki", "University"]
console.log(name.includes("Univ")); // true
```

// Template literal (ES6) — back-ticks allow \${} and line breaks

```
let marks = 85;
console.log(`${name} student scored ${marks} out of 100.`);
```

// ----- ARRAYS -----

```
let subjects = ["Web Tech", "DSA", "Maths"];
subjects.push("English"); // add at the end
subjects.unshift("Physics"); // add at the beginning
subjects.pop(); // remove the last
subjects.shift(); // remove the first
console.log(subjects.length); // 3
console.log(subjects[0]); // "Web Tech"
console.log(subjects.join(", "));
console.log(subjects.indexOf("DSA"));
```

```
let nums = [5, 2, 9, 1];
console.log(nums.sort((a,b) => a-b)); // [1,2,5,9]
console.log(nums.map(n => n * 2)); // [2,4,10,18]
console.log(nums.filter(n => n > 3)); // [5,9]
console.log(nums.reduce((s,n) => s+n, 0)); // 17
nums.forEach(n => console.log(n));
```

// ----- OBJECTS -----

```
let student = {
  name: "Sita Gurung",
  roll: "BIT-2026-001",
  marks: [78, 85, 90],
  getAverage: function () {
    let total = this.marks.reduce((s, m) => s + m, 0);
    return total / this.marks.length;
  }
};
console.log(student.name); // dot notation
console.log(student["roll"]); // bracket notation
```

```
console.log(student.getAverage()); // 84.33
student.email = "sita@gu.edu.np"; // add a new property
delete student.roll; // remove a property
```

4.3 Operators

An **operator** performs an operation on one or more values (operands) and produces a result.

Arithmetic operators

Operator	Name	Example (a = 10, b = 3)	Result
+	Addition	a + b	13
-	Subtraction	a - b	7
*	Multiplication	a * b	30
/	Division	a / b	3.333...
%	Modulus (remainder)	a % b	1
**	Exponentiation (ES7)	a ** 2	100
++	Increment	a++ (post) / ++a (pre)	11
--	Decrement	a-- / --a	9

Assignment operators

Operator	Example	Equivalent to
=	x = 5	assigns 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
**=	x **= 2	x = x ** 2

Comparison operators

Operator	Meaning	Example	Result
==	Equal in value (type converted)	5 == "5"	true
===	Equal in value and type	5 === "5"	false
!=	Not equal in value	5 != "5"	false

Operator	Meaning	Example	Result
<code>!=</code>	Not equal in value or type	<code>5 != "5"</code>	true
<code>></code>	Greater than	<code>10 > 3</code>	true
<code><</code>	Less than	<code>10 < 3</code>	false
<code>>=</code>	Greater than or equal	<code>10 >= 10</code>	true
<code><=</code>	Less than or equal	<code>3 <= 10</code>	true

Logical, string and other operators

Operator	Name	Description	Example
<code>&&</code>	Logical AND	True only when both sides are true	<code>(a>5 && b<10)</code>
<code> </code>	Logical OR	True when at least one side is true	<code>(a>5 b<10)</code>
<code>!</code>	Logical NOT	Reverses a boolean	<code>!isValid</code>
<code>?:</code>	Ternary (conditional)	A compact if–else expression	<code>let r = (m>=40) ? "Pass" : "Fail";</code>
<code>+</code>	Concatenation	Joins two strings	<code>"Web" + " Tech"</code>
<code>typeof</code>	Type operator	Returns the type as a string	<code>typeof x</code>
<code>??</code>	Nullish coalescing	Returns the right side only if the left is <code>null</code> or <code>undefined</code>	<code>let n = input ?? "default";</code>
<code>&, , ^, ~, <<, >></code>	Bitwise	Operate on the individual bits of a number	<code>5 & 3</code> is 1

4.4 Conditional Statements

A **conditional (decision-making) statement** executes a block of code only when a given condition is true.

Figure 4.1 Flow of an if–else statement

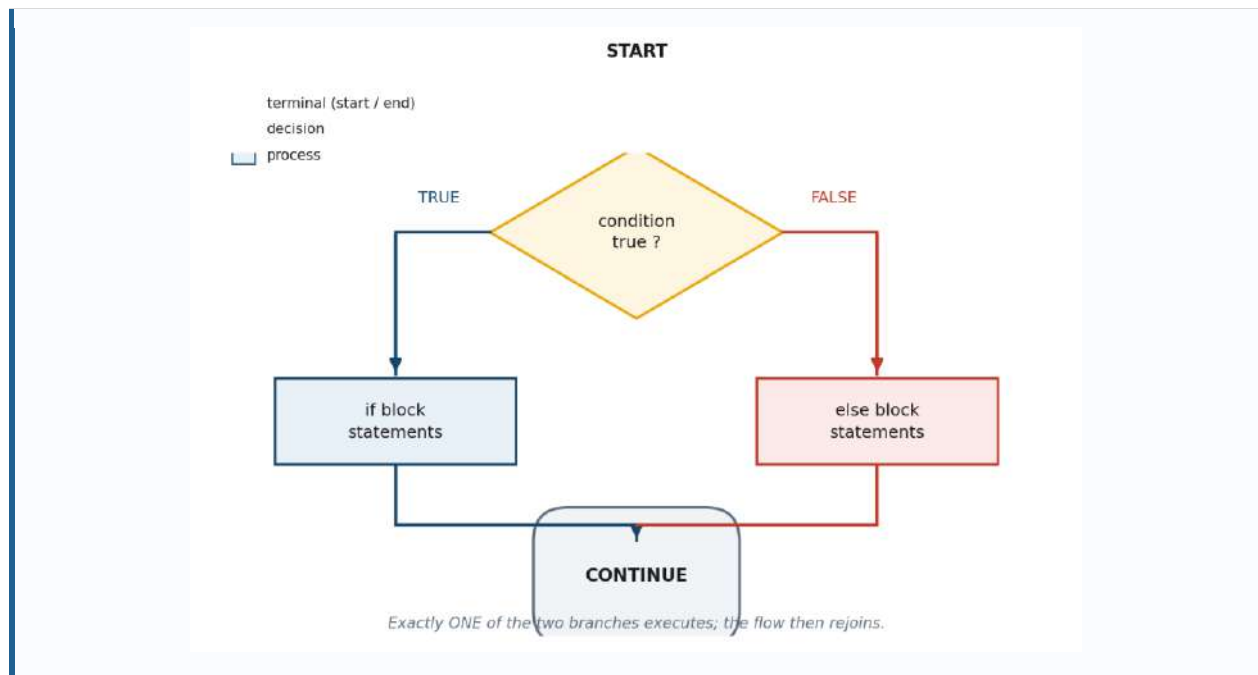


Figure 4.1 — Only one of the two branches ever executes.

JAVASCRIPT

// ----- 1. simple if -----

```

let marks = 75;
if (marks >= 40) {
  console.log("Pass");
}

```

// ----- 2. if...else -----

```

if (marks >= 40) {
  console.log("Pass");
} else {
  console.log("Fail");
}

```

// ----- 3. if...else if...else (a grading system) -----

```

let grade;
if (marks >= 90) { grade = "A+"; }
else if (marks >= 80) { grade = "A"; }
else if (marks >= 70) { grade = "B+"; }
else if (marks >= 60) { grade = "B"; }
else if (marks >= 40) { grade = "C"; }
else { grade = "F"; }
console.log("Grade: " + grade); // Grade: B+

```

// ----- 4. nested if -----

```

let age = 20, hasID = true;
if (age >= 18) {
  if (hasID) { console.log("Allowed to vote"); }
  else { console.log("Bring your ID card"); }
} else {
  console.log("Not eligible");
}

```

// ----- 5. ternary operator -----

```

let result = (marks >= 40) ? "Pass" : "Fail";

```

```
// ----- 6. switch -----
let day = 3, dayName;
switch (day) {
  case 1: dayName = "Sunday"; break;
  case 2: dayName = "Monday"; break;
  case 3: dayName = "Tuesday"; break;
  case 4: dayName = "Wednesday"; break;
  default: dayName = "Invalid day";
}
console.log(dayName);      // Tuesday
```

Common Mistake: Never forget **break** in a **switch**. Without it, execution "falls through" into the following cases and every statement after the matching case runs.

Difference between `if...else` and `switch`

Basis	if...else	switch
Condition type	Any boolean expression, including ranges and complex logic	Compares one expression against fixed constant values
Comparison	Any relational or logical operator	Strict equality (===) only
Readability	Becomes messy with many branches	Clean when there are many discrete values
Data types	Works with any type and any range	Works with numbers, strings and booleans
Speed	Conditions are evaluated one by one	Can be optimised into a jump table
break needed	No	Yes, otherwise cases fall through
Best for	Ranges, e.g. marks between 60 and 69	Menus, day or month names, fixed command codes

4.5 Looping Statements

A **loop** repeats a block of statements while a condition remains true. Every loop needs three things: **initialisation**, a **condition**, and an **update**.

Figure 4.2 Flow of a for loop

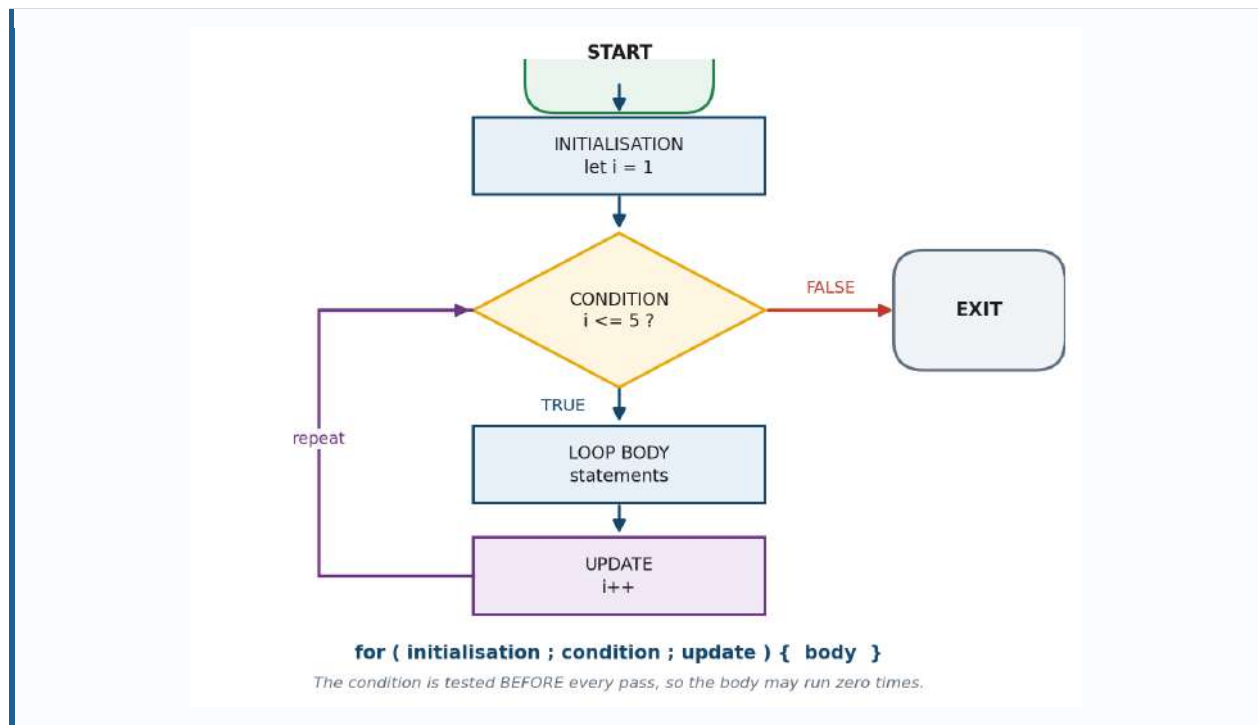


Figure 4.2 — The condition is tested before every iteration.

JAVASCRIPT

// ----- 1. for loop: when the count is known -----

```
for (let i = 1; i <= 5; i++) {
  console.log("Iteration " + i);
}
```

// multiplication table of 7

```
for (let i = 1; i <= 10; i++) {
  console.log(`7 x ${i} = ${7 * i}`);
}
```

// ----- 2. while loop: condition tested FIRST -----

```
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;          // never forget the update!
}
```

// ----- 3. do...while: body runs AT LEAST ONCE -----

```
let j = 10;
do {
  console.log("This prints once even though the condition is false");
  j++;
} while (j < 5);
```

// ----- 4. for...of : iterate over VALUES of an array -----

```
let subjects = ["Web Tech", "DSA", "Maths"];
for (let subject of subjects) {
  console.log(subject);
}
```

// ----- 5. for...in : iterate over KEYS of an object -----

```
let student = { name: "Sita", roll: 1, marks: 85 };
for (let key in student) {
```

```

    console.log(key + " = " + student[key]);
}

// ----- 6. break and continue -----
for (let k = 1; k <= 10; k++) {
    if (k === 4) continue; // skip this iteration only
    if (k === 8) break;    // leave the loop completely
    console.log(k);        // 1 2 3 5 6 7
}

// ----- 7. nested loop: a star pattern -----
let pattern = "";
for (let row = 1; row <= 5; row++) {
    for (let col = 1; col <= row; col++) { pattern += "* "; }
    pattern += "\n";
}
console.log(pattern);

```

Difference between `while` and `do...while`

Basis	while	do...while
Type	Entry-controlled loop	Exit-controlled loop
Condition tested	Before the body executes	After the body executes
Minimum executions	Zero — the body may never run	One — the body always runs at least once
Syntax	while (cond) { ... }	do { ... } while (cond);
Semicolon	Not required after the closing brace	Required after while(cond)
Use when	The body should run only if the condition holds	The body must run at least once, e.g. showing a menu

4.6 Functions

4.6.1 Definition

A **function** is a named, reusable block of code that performs a specific task. It runs only when it is called (invoked). Functions provide **modularity**, **reusability**, and **abstraction** — the three pillars of maintainable code.

Syntax

```

SYNTAX
function functionName(parameter1, parameter2) {
    // statements
    return value;    // optional
}

functionName(argument1, argument2); // calling the function

```

JAVASCRIPT

// ----- 1. Function declaration (hoisted) -----

```
function greet(name) {  
  return "Namaste, " + name + "!";  
}  
console.log(greet("Sita"));    // Namaste, Sita!
```

// ----- 2. Function with several parameters -----

```
function calculateTotal(theory, lab, project) {  
  return theory + lab + project;  
}  
console.log(calculateTotal(18, 14, 5)); // 37
```

// ----- 3. Default parameter values (ES6) -----

```
function greetUser(name = "Guest", greeting = "Hello") {  
  return `${greeting}, ${name}!`;  
}  
console.log(greetUser());    // Hello, Guest!  
console.log(greetUser("Ram", "Hi")); // Hi, Ram!
```

// ----- 4. Function expression (NOT hoisted) -----

```
const square = function (n) {  
  return n * n;  
};  
console.log(square(7));    // 49
```

// ----- 5. Arrow function (ES6) -----

```
const cube = (n) => n * n * n;    // implicit return  
const add = (a, b) => a + b;  
const sayHi = () => console.log("Hi");  
const average = (arr) => {  
  let sum = arr.reduce((s, v) => s + v, 0);  
  return sum / arr.length;  
};  
console.log(cube(3), add(2,3), average([80, 90, 70]));
```

// ----- 6. Rest parameters: any number of arguments -----

```
function sumAll(...numbers) {  
  return numbers.reduce((total, n) => total + n, 0);  
}  
console.log(sumAll(1, 2, 3, 4, 5)); // 15
```

// ----- 7. Callback: a function passed to another function -----

```
function processMarks(marks, callback) {  
  return callback(marks);  
}  
console.log(processMarks(85, m => (m >= 40 ? "Pass" : "Fail")));
```

// ----- 8. Recursion: a function that calls itself -----

```
function factorial(n) {  
  if (n <= 1) return 1;    // base case — always required  
  return n * factorial(n - 1); // recursive case  
}  
console.log(factorial(5)); // 120
```

// ----- 9. IIFE: runs immediately, keeps variables private -----

```
(function () {  
  let secret = "not visible outside";  
  console.log("IIFE ran immediately");  
})();
```

Difference between a parameter and an argument

Basis	Parameter	Argument
Definition	The variable named in the function definition	The actual value passed when the function is called
Where it appears	In the declaration: <code>function add(a, b)</code>	In the call: <code>add(5, 3)</code>
Nature	A placeholder	A real value
Example	<code>a</code> and <code>b</code>	<code>5</code> and <code>3</code>

Difference between a function declaration and an arrow function

Basis	Function declaration	Arrow function
Syntax	<code>function f(a){ return a; }</code>	<code>const f = a => a;</code>
Hoisting	Fully hoisted — can be called before it is defined	Not hoisted — must be defined before use
<code>this</code> binding	Has its own <code>this</code> , determined by how it is called	Inherits <code>this</code> from the surrounding scope (lexical <code>this</code>)
<code>arguments</code> object	Available	Not available; use rest parameters instead
Use as a constructor	Possible with <code>new</code>	Not possible
Best for	General-purpose named functions, object methods	Short callbacks, array methods, event handlers that need the outer <code>this</code>

4.7 The Document Object Model (DOM)

4.7.1 Definition

The **Document Object Model (DOM)** is a programming interface created by the browser for an HTML document. When a page is loaded, the browser parses the HTML and builds a **tree of objects** in memory, in which every element, attribute and piece of text becomes a **node**. JavaScript can then read and change any node, and the browser immediately re-renders the page to match.

Note: The DOM is not part of the JavaScript language itself. It is an API provided by the browser. This is why the same JavaScript language can also run outside a browser (in Node.js) where no DOM exists.

The DOM tree

```
HTML
<!DOCTYPE html>
<html>
  <head>
    <title>My Page</title>
```

```

</head>
<body>
  <h1 id="main">Hello</h1>
  <p class="intro">Welcome to <a href="#">GU</a></p>
</body>
</html>

```

Figure 4.3 The DOM tree of the document above

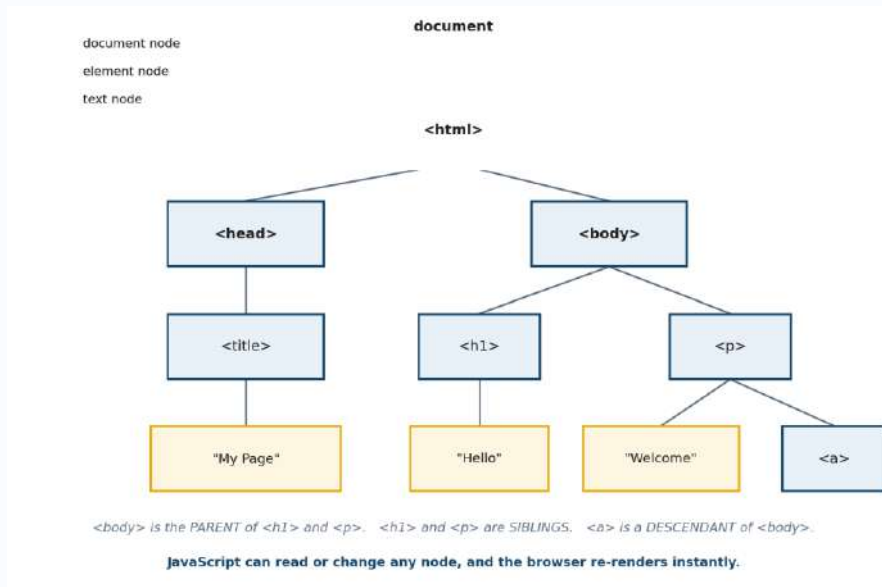


Figure 4.3 — Every part of the document becomes a node in a tree.

Why the DOM matters

- It gives JavaScript a **structured, addressable model** of the page instead of plain text.
- Any change made to the DOM is reflected on the screen **instantly**, without reloading.
- It is the mechanism behind every dynamic feature: validation messages, dropdown menus, live search, single-page applications.

4.7.2 Selecting Elements

Method	Returns	Example
<code>document.getElementById(id)</code>	One element, or <code>null</code>	<code>document.getElementById("email")</code>
<code>document.getElementsByClassName(c)</code>	A live <code>HTMLCollection</code>	<code>document.getElementsByClassName("card")</code>
<code>document.getElementsByTagName(t)</code>	A live <code>HTMLCollection</code>	<code>document.getElementsByTagName("p")</code>
<code>document.getElementsByName(n)</code>	A live <code>NodeList</code>	<code>document.getElementsByName("gender")</code>
<code>`document.querySelector(css)`</code>	The first element matching any CSS selector	<code>document.querySelector(".card h3")</code>
<code>`document.querySelectorAll(css)`</code>	A static <code>NodeList</code> of all matches	<code>document.querySelectorAll("input[type=text]")</code>
<code>element.children</code>	The child elements	<code>box.children</code>

Method	Returns	Example
<code>element.parentElement</code>	The parent element	<code>item.parentElement</code>
<code>element.nextElementSibling</code>	The next sibling element	<code>row.nextElementSibling</code>

JAVASCRIPT

*// The two modern methods handle every case, because they accept
// any CSS selector you already learnt in Chapter 3.*

```
let title = document.querySelector("#main");
let firstP = document.querySelector("p.intro");
let allBtns = document.querySelectorAll(".btn");
```

```
allBtns.forEach(function (btn) {
  btn.style.background = "#14456b";
});
```

*// An HTMLCollection is LIVE (it updates automatically);
// a NodeList from querySelectorAll is STATIC (a snapshot).
// Only a NodeList has .forEach() built in.*

4.7.3 Reading and Changing Content

Property / Method	Purpose
<code>innerHTML</code>	Gets or sets the HTML inside an element. Tags in the string are parsed as HTML.
<code>textContent</code>	Gets or sets the plain text. Tags are treated as literal text — safer.
<code>innerText</code>	Like <code>textContent</code> , but respects CSS and returns only visible text.
<code>value</code>	Gets or sets the value of a form control (<code>input</code> , <code>select</code> , <code>textarea</code>).
<code>setAttribute(name, val)</code>	Sets any attribute.
<code>getAttribute(name)</code>	Reads any attribute.
<code>removeAttribute(name)</code>	Deletes an attribute.
<code>classList.add / remove / toggle / contains</code>	Manages CSS classes — the cleanest way to change appearance.

JAVASCRIPT

// ----- reading -----

```
let name = document.getElementById("fullname").value;
let heading = document.querySelector("h1").textContent;
```

// ----- writing -----

```
document.getElementById("output").innerHTML =
  "<strong>Welcome, " + name + "!</strong>";
document.getElementById("msg").textContent = "Saved successfully";
document.getElementById("photo").setAttribute("src", "new.jpg");
```

// ----- classes (the recommended way to restyle) -----

```

let box = document.querySelector(".box");
box.classList.add("highlight");
box.classList.remove("hidden");
box.classList.toggle("dark-mode");
if (box.classList.contains("active")) { console.log("it is active"); }

// ----- creating and removing elements -----
let li = document.createElement("li"); // 1. create
li.textContent = "New subject"; // 2. fill
li.className = "list-item";
document.querySelector("ul").appendChild(li); // 3. attach

let old = document.getElementById("temp");
old.remove(); // delete a node

```

Difference between `innerHTML` and `textContent`

Basis	innerHTML	textContent
Content handled	HTML markup — tags are parsed and rendered	Plain text — tags appear literally as characters
Setting "Hi"	Displays a bold Hi	Displays the characters Hi
Speed	Slower — the browser must parse the string	Faster
Security	Risky: inserting user input can create a cross-site scripting (XSS) vulnerability	Safe: any script tag is inert
Use when	You deliberately need to insert markup you control	You are inserting text, especially user-supplied text

4.8 Events and Event Handling

4.8.1 Definition

An **event** is an action or occurrence detected by the browser — a click, a key press, the page finishing loading, a form being submitted. **Event handling** is the process of writing a function (an **event handler** or **listener**) that runs in response to that event. JavaScript is fundamentally an **event-driven** language.

Common events

Category	Event	Fires when...
Mouse	click	The element is clicked
	dblclick	The element is double-clicked
	mouseover / mouseout	The pointer enters / leaves the element
	mousedown / mouseup	A mouse button is pressed / released

Category	Event	Fires when...
	<code>mousemove</code>	The pointer moves over the element
Keyboard	<code>keydown</code>	A key is pressed down
	<code>keyup</code>	A key is released
	<code>keypress</code>	A character key is pressed (deprecated)
Form	<code>submit</code>	A form is submitted — the key event for validation
	<code>reset</code>	A form is reset
	<code>change</code>	A control loses focus after its value changed
	<code>input</code>	The value changes, on every keystroke
	<code>focus / blur</code>	A control gains / loses focus
Window	<code>load</code>	The page and all its resources have finished loading
	<code>DOMContentLoaded</code>	The HTML is parsed — faster than <code>load</code>
	<code>resize</code>	The browser window is resized
	<code>scroll</code>	The page or an element is scrolled
	<code>unload / beforeunload</code>	The user is leaving the page
Touch	<code>touchstart</code> , <code>touchmove</code> , <code>touchend</code>	Touch interaction on a mobile device

4.8.2 The Three Models of Event Handling

(a) *Inline HTML attribute — the oldest model*

HTML

```
<button onclick="alert('Clicked!')">Click me</button>
<button onclick="showMessage()">Call a function</button>
<form onsubmit="return validateForm()"> ... </form>
<input type="text" onblur="checkField(this)">
```

- **Advantage:** very simple for a beginner to read.
- **Disadvantage:** mixes behaviour into the markup, only one handler per event, hard to maintain.

(b) *DOM property (traditional) model*

JAVASCRIPT

```
let btn = document.getElementById("myBtn");
btn.onclick = function () {
  alert("Button was clicked");
};

// Assigning a second handler REPLACES the first one:
btn.onclick = function () { alert("Only this one runs"); };
```

- **Advantage:** keeps JavaScript out of the HTML.

- **Disadvantage:** still only one handler per event per element.

(c) `addEventListener()` — the modern standard

```
JAVASCRIPT
let btn = document.getElementById("myBtn");

btn.addEventListener("click", function (event) {
  console.log("Handler 1 ran");
  console.log("Clicked element:", event.target);
});

btn.addEventListener("click", () => console.log("Handler 2 also ran"));

// Syntax: element.addEventListener(type, handler, options)
// type -> the event name WITHOUT "on" ("click", not "onclick")
// handler -> the function to run
// options -> true for the capturing phase, or { once: true }

// Removing a listener requires a NAMED function
function sayHi() { alert("Hi"); }
btn.addEventListener("click", sayHi);
btn.removeEventListener("click", sayHi);
```

Comparison of the three models

Basis	Inline attribute	DOM property	<code>addEventListener()</code>
Written in	The HTML tag	A JavaScript file	A JavaScript file
Syntax	<code>onclick="fn()"</code>	<code>el.onclick = fn;</code>	<code>el.addEventListener("click", fn);</code>
Handlers per event	One	One	Many
Separation of concerns	Poor	Good	Excellent
Can remove a handler	Difficult	<code>el.onclick = null</code>	<code>removeEventListener()</code>
Capturing phase	No	No	Yes
Recommendation	Avoid	Acceptable	Preferred

4.8.3 The DOM Event Model — Bubbling and Capturing

When an event occurs on a nested element it does not fire only there. It travels through the DOM tree in **three phases**.

Figure 4.4 The three phases of the DOM event model

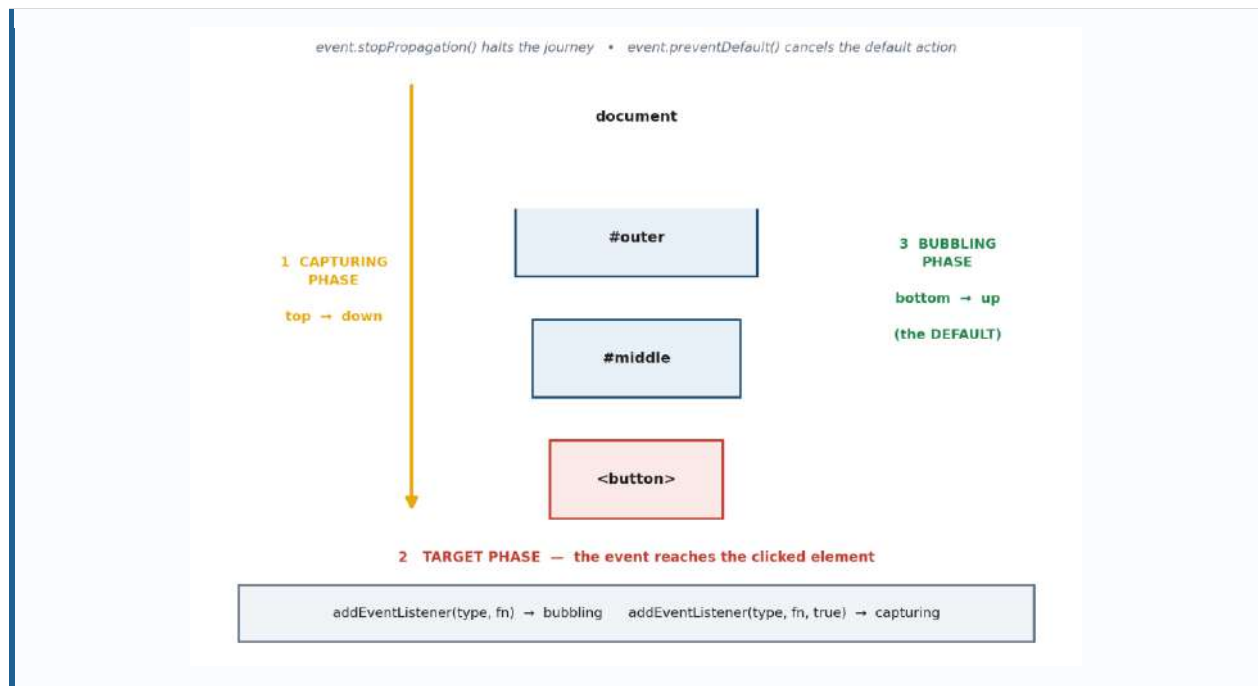


Figure 4.4 — Capturing goes down the tree, bubbling comes back up.

JAVASCRIPT

```
document.getElementById("outer").addEventListener("click", () => {
  console.log("outer div clicked (bubbling)");
});

document.querySelector("button").addEventListener("click", (e) => {
  console.log("button clicked");
  // e.stopPropagation(); // uncomment to stop the outer handler
});

// Clicking the button logs: "button clicked" then
// "outer div clicked (bubbling)"

// EVENT DELEGATION: one listener on the parent handles all children.
// This works even for elements added later.
document.querySelector("#list").addEventListener("click", function (e) {
  if (e.target.tagName === "LI") {
    e.target.classList.toggle("done");
  }
});
```

Useful properties of the event object

Property / Method	Meaning
<code>event.target</code>	The element on which the event actually occurred
<code>event.currentTarget</code>	The element whose listener is currently running
<code>event.type</code>	The name of the event, e.g. <code>"click"</code>
<code>event.preventDefault()</code>	Cancels the browser's default action — stops a form submitting or a link navigating

Property / Method	Meaning
<code>event.stopPropagation()</code>	Stops the event travelling further up or down the tree
<code>event.key</code> / <code>event.keyCode</code>	The key involved in a keyboard event
<code>event.clientX</code> / <code>clientY</code>	The pointer coordinates within the viewport

Difference between bubbling and capturing

Basis	Event bubbling	Event capturing (trickling)
Direction	From the target element upwards to <code>document</code>	From <code>document</code> downwards to the target element
Order of phases	Third and last phase	First phase
Default behaviour	Yes — this is what happens unless you ask otherwise	No — must be requested explicitly
How to use	<code>addEventListener("click", fn)</code>	<code>addEventListener("click", fn, true)</code>
Typical use	Event delegation, handling many children with one listener	Intercepting an event before a child can handle it
Stopped by	<code>event.stopPropagation()</code>	<code>event.stopPropagation()</code>

4.9 Changing Colours, Fonts, Content and Element Positioning

4.9.1 Changing Style with the `style` Property

Every element object has a `style` property that maps to its **inline** CSS. CSS property names written with hyphens become camelCase in JavaScript: `background-color` becomes `backgroundColor`, `font-size` becomes `fontSize`.

CSS property	JavaScript property
<code>color</code>	<code>element.style.color</code>
<code>background-color</code>	<code>element.style.backgroundColor</code>
<code>font-family</code>	<code>element.style.fontFamily</code>
<code>font-size</code>	<code>element.style.fontSize</code>
<code>font-weight</code>	<code>element.style.fontWeight</code>
<code>text-align</code>	<code>element.style.textAlign</code>
<code>border-radius</code>	<code>element.style.borderRadius</code>
<code>margin-top</code>	<code>element.style.marginTop</code>
<code>z-index</code>	<code>element.style.zIndex</code>

JAVASCRIPT

```

let box = document.getElementById("box");

// ----- colours -----
box.style.color      = "white";
box.style.backgroundColor = "#14456b";
box.style.borderColor = "rgb(241, 196, 15)";

// ----- fonts -----
box.style.fontFamily = "Georgia, serif";
box.style.fontSize   = "22px";    // the UNIT is compulsory
box.style.fontWeight = "bold";
box.style.fontStyle  = "italic";
box.style.textAlign  = "center";
box.style.textTransform = "uppercase";

// ----- several properties at once -----
box.style.cssText = "color:#fff; background:#14456b; padding:20px;";

// ----- reading a COMPUTED style (from the style sheet) -----
let actualColor = window.getComputedStyle(box).color;
console.log(actualColor);

// ----- the CLEANER approach: swap a CSS class -----
// CSS: .theme-dark { background:#1a1a1a; color:#f4f6f8; }
box.classList.toggle("theme-dark");

```

Exam Tip: Setting `element.style.x` writes an inline style, which has very high specificity and is hard to override later. For anything more than one or two properties, define a CSS class and switch it on with `classList`.

4.9.2 Positioning Elements with JavaScript

To move an element with JavaScript, its CSS `position` must be `relative`, `absolute`, `fixed` or `sticky` — the offsets have no effect on a `static` element.

HTML

```

<style>
  #ball { position: absolute; left: 0; top: 100px;
    width: 50px; height: 50px; border-radius: 50%;
    background: #14456b; }
</style>

<div id="ball"></div>
<button onclick="moveRight()">Move right</button>

```

JAVASCRIPT

```

let ball = document.getElementById("ball");
let pos = 0;

function moveRight() {
  pos += 20;
  ball.style.left = pos + "px";    // remember the unit
}

```

```
// ----- continuous animation with setInterval -----
function animate() {
  let x = 0;
  let timer = setInterval(function () {
    if (x >= 300) {
      clearInterval(timer);    // stop the animation
    } else {
      x += 5;
      ball.style.left = x + "px";
    }
  }, 20);                // every 20 milliseconds
}

// ----- reading the size and position of an element -----
let rect = ball.getBoundingClientRect();
console.log(rect.top, rect.left, rect.width, rect.height);
console.log(ball.offsetLeft, ball.offsetTop); // relative to parent
console.log(window.innerWidth, window.innerHeight); // viewport size
console.log(window.scrollY);                // how far page scrolled

// ----- following the mouse -----
document.addEventListener("mousemove", function (e) {
  ball.style.left = (e.clientX - 25) + "px";
  ball.style.top = (e.clientY - 25) + "px";
});

// ----- scroll the page to an element -----
document.getElementById("section3").scrollIntoView({behavior:"smooth"});
```

Complete worked example — interactive style changer

```
HTML
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Dynamic Styling with JavaScript</title>
  <style>
    body { font-family: Arial, sans-serif; padding: 30px; }
    #demo { padding: 30px; border: 2px solid #ccc; border-radius: 8px;
      transition: all .3s ease; }
    button { margin: 5px; padding: 9px 16px; cursor: pointer;
      border: none; border-radius: 5px; background: #14456b;
      color: #fff; }
    button:hover { background: #1f6091; }
    button:focus { outline: 3px solid #f1c40f; }
    .highlight { background: #fff8dc; border-color: #f1c40f; }
  </style>
</head>
<body>

  <div id="demo">
    <h2 id="title">Change my appearance</h2>
    <p id="para">Click the buttons below to change my style.</p>
  </div>

  <button onclick="changeColour()">Change colour</button>
  <button onclick="changeFont()">Change font</button>
  <button onclick="toggleHighlight()">Toggle highlight</button>
  <button onclick="changeContent()">Change content</button>
```

```
<button onclick="resetAll()">Reset</button>

<script>
const colours = ["#c0392b", "#27ae60", "#8e44ad", "#14456b"];
const fonts = ["Georgia, serif", "Courier New, monospace",
               "Verdana, sans-serif", "Arial, sans-serif"];
let cIndex = 0, fIndex = 0;

const demo = document.getElementById("demo");
const title = document.getElementById("title");
const para = document.getElementById("para");

function changeColour() {
  title.style.color = colours[cIndex];
  cIndex = (cIndex + 1) % colours.length;
}

function changeFont() {
  demo.style.fontFamily = fonts[fIndex];
  demo.style.fontSize = "18px";
  fIndex = (fIndex + 1) % fonts.length;
}

function toggleHighlight() {
  demo.classList.toggle("highlight");
}

function changeContent() {
  para.innerHTML = "<strong>The content was replaced at " +
    new Date().toLocaleTimeString() + "</strong>";
}

function resetAll() {
  demo.style.cssText = "";
  title.style.cssText = "";
  demo.classList.remove("highlight");
  para.textContent = "Click the buttons below to change my style.";
}
</script>
</body>
</html>
```

4.10 Form Validation

4.10.1 Definition and Importance

Form validation is the process of checking that the data a user has entered is complete, correctly formatted and reasonable, before it is accepted. Validation performed in the browser is called **client-side validation**; validation performed by the server (in PHP, Chapter 5) is called **server-side validation**.

Why validate?

- **Instant feedback** — the user is told about a mistake without waiting for a page reload.
- **Reduced server load** — obviously wrong data never reaches the server.
- **Better data quality** — the database receives clean, consistent records.

- **Improved user experience** — errors are shown next to the field concerned.

Common Mistake: Client-side validation is a **convenience, never a security measure**. A user can disable JavaScript or send a request directly with a tool such as curl. **You must always repeat every check on the server.** This is one of the most important professional lessons in this course.

Difference between client-side and server-side validation

Basis	Client-side validation	Server-side validation
Where it runs	In the user's browser	On the web server
Language used	JavaScript, or HTML5 attributes	PHP, Python, Java, etc.
Speed	Immediate — no network round trip	Slower — a request and response are needed
Server load	Reduces it	Increases it
Can be bypassed	Yes — by disabling JavaScript or using a direct request	No — the user cannot reach the code
Security	Not secure; purely for convenience	The only trustworthy check
Access to the database	None (cannot check whether an e-mail already exists)	Full access
Best practice	Use both: client-side for a good experience, server-side for safety	Compulsory in every application

4.10.2 HTML5 Built-in Validation

HTML

```
<!-- The browser performs these checks with no JavaScript at all -->
<input type="text" required minlength="3" maxlength="50">
<input type="email" required>
<input type="url">
<input type="number" min="18" max="60" step="1">
<input type="tel" pattern="[0-9]{10}"
  title="Enter exactly 10 digits">
<input type="password" required minlength="8">
<input type="date" max="2010-12-31">
```

HTML5 validation is quick to write but its error messages cannot be styled and its wording cannot be changed. Most real projects add JavaScript on top, and switch the built-in behaviour off with `novalidate` on the `<form>`.

4.10.3 Complete JavaScript Form Validation

This script validates the Student Registration Form written in Chapter 2. Study it carefully — a question asking you to write a validation script appears in almost every examination.

JAVASCRIPT

```
/* ===== file: js/validate.js ===== */
"use strict";

/* ----- helper: show an error under a field ----- */
function showError(fieldId, message) {
  const field = document.getElementById(fieldId);
  const span = document.getElementById(fieldId + "Error");
  span.textContent = message;
  span.style.color = "#c0392b";
  span.style.fontSize = "0.85rem";
  field.style.border = "2px solid #c0392b";
  field.setAttribute("aria-invalid", "true");
}

/* ----- helper: clear an error ----- */
function clearError(fieldId) {
  const field = document.getElementById(fieldId);
  document.getElementById(fieldId + "Error").textContent = "";
  field.style.border = "1px solid #b9c6d4";
  field.removeAttribute("aria-invalid");
}

/* ===== the main validation function ===== */
function validateForm() {

  let isValid = true;

  /* ----- 1. FULL NAME: required, letters and spaces only ---- */
  const name = document.getElementById("fullname").value.trim();
  clearError("fullname");
  if (name === "") {
    showError("fullname", "Full name is required.");
    isValid = false;
  } else if (name.length < 3) {
    showError("fullname", "Name must be at least 3 characters long.");
    isValid = false;
  } else if (!/^[A-Za-z ]+$/.test(name)) {
    showError("fullname", "Name may contain letters and spaces only.");
    isValid = false;
  }

  /* ----- 2. EMAIL: required and correctly formatted ----- */
  const email = document.getElementById("email").value.trim();
  const emailPattern = /^[^\s@]+@[^\s@]+\.[a-zA-Z]{2,}$/;
  clearError("email");
  if (email === "") {
    showError("email", "Email address is required.");
    isValid = false;
  } else if (!emailPattern.test(email)) {
    showError("email", "Enter a valid email, e.g. name@example.com");
    isValid = false;
  }

  /* ----- 3. PHONE: optional, but 10 digits if present ----- */
  const phone = document.getElementById("phone").value.trim();
  clearError("phone");
  if (phone !== "" && /^[0-9]{10}$/.test(phone)) {
    showError("phone", "Phone number must be exactly 10 digits.");
    isValid = false;
  }
}
```

```
/* ----- 4. PASSWORD: length and strength ----- */
const pwd = document.getElementById("pwd").value;
clearError("pwd");
if (pwd === "") {
  showError("pwd", "Password is required.");
  isValid = false;
} else if (pwd.length < 8) {
  showError("pwd", "Password must be at least 8 characters.");
  isValid = false;
} else if (!/[A-Z]/.test(pwd) || ![a-z]/.test(pwd) ||
  ![0-9]/.test(pwd)) {
  showError("pwd",
    "Password needs an upper case, a lower case and a digit.");
  isValid = false;
}

/* ----- 5. CONFIRM PASSWORD: must match ----- */
const cpwd = document.getElementById("cpwd").value;
clearError("cpwd");
if (cpwd !== pwd) {
  showError("cpwd", "The two passwords do not match.");
  isValid = false;
}

/* ----- 6. GENDER: one radio button must be chosen ----- */
const genders = document.getElementsByName("gender");
let genderChosen = false;
for (let i = 0; i < genders.length; i++) {
  if (genders[i].checked) { genderChosen = true; break; }
}
document.getElementById("genderError").textContent = "";
if (!genderChosen) {
  document.getElementById("genderError").textContent =
    "Please select a gender.";
  document.getElementById("genderError").style.color = "#c0392b";
  isValid = false;
}

/* ----- 7. PROGRAMME: a select box must not be empty ----- */
const prog = document.getElementById("prog").value;
clearError("prog");
if (prog === "") {
  showError("prog", "Please select a programme.");
  isValid = false;
}

/* ----- 8. INTERESTS: at least one check box ----- */
const boxes = document.querySelectorAll("input[name='interest[]']");
let checkedCount = 0;
boxes.forEach(b => { if (b.checked) checkedCount++; });
document.getElementById("interestError").textContent = "";
if (checkedCount === 0) {
  document.getElementById("interestError").textContent =
    "Select at least one area of interest.";
  document.getElementById("interestError").style.color = "#c0392b";
  isValid = false;
}

/* ----- 9. FILE: type and size ----- */
const photo = document.getElementById("photo");
clearError("photo");
if (photo.files.length > 0) {
  const file = photo.files[0];
```

```

const allowed = ["image/jpeg", "image/png"];
if (!allowed.includes(file.type)) {
  showError("photo", "Only JPG and PNG images are allowed.");
  isValid = false;
} else if (file.size > 2 * 1024 * 1024) {
  showError("photo", "The image must be smaller than 2 MB.");
  isValid = false;
}
}

/* ----- 10. DECLARATION check box ----- */
if (!document.getElementById("agree").checked) {
  alert("You must accept the declaration before registering.");
  isValid = false;
}

/* ----- final result ----- */
if (isValid) {
  alert("Validation passed. Your form is being submitted.");
}
return isValid; // returning false CANCELS the submission
}

/* ===== live validation as the user types ===== */
document.addEventListener("DOMContentLoaded", function () {

  // clear the error as soon as the user starts correcting the field
  ["fullname", "email", "phone", "pwd", "cpwd"].forEach(function (id) {
    const el = document.getElementById(id);
    if (el) { el.addEventListener("input", () => clearError(id)); }
  });

  // block non-numeric keys in the phone field
  const phone = document.getElementById("phone");
  if (phone) {
    phone.addEventListener("keypress", function (e) {
      if (!/[0-9]/.test(e.key)) { e.preventDefault(); }
    });
  }

  // attach the handler with addEventListener instead of onsubmit
  const form = document.getElementById("regForm");
  if (form) {
    form.addEventListener("submit", function (e) {
      if (!validateForm()) { e.preventDefault(); }
    });
  }
});

```

The HTML changes needed for the error messages

HTML

```

<!-- add an empty span after every field, and load the script -->
<label for="fullname">Full Name *</label>
<input type="text" id="fullname" name="fullname">
<span id="fullnameError" class="error"></span>

<label for="email">Email *</label>
<input type="email" id="email" name="email">
<span id="emailError" class="error"></span>

```

```
<span id="genderError" class="error"></span>
<span id="interestError" class="error"></span>

<script src="js/validate.js" defer></script>
```

Regular expressions used in validation

Pattern	Validates	Meaning
<code>/^[A-Za-z]+\$/</code>	A name	One or more letters or spaces, from start to end
<code>/^[^s@]+@[^\s@]+\.[a-zA-Z]{2,}\$/</code>	An e-mail address	text, @, text, dot, at least two letters
<code>/^[0-9]{10}\$/</code>	A phone number	Exactly ten digits
<code>/^[A-Za-z0-9_]{4,20}\$/</code>	A user name	4 to 20 letters, digits or underscores
<code>/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}\$/</code>	A strong password	At least 8 characters with lower case, upper case and a digit
<code>/^\d{4}-\d{2}-\d{2}\$/</code>	A date	YYYY-MM-DD

Note: Anchors: `^` marks the start of the string and `$` the end. Quantifiers: `+` means one or more, `*` means zero or more, `{n}` means exactly n, `{n,}` means n or more. Classes: `\d` is a digit, `\w` is a word character, `\s` is whitespace. The method `pattern.test(string)` returns `true` or `false`.

4.11 Exam-Based Questions — Chapter 4

Group A — Very Short Answer Questions (2 marks each)

1. Define JavaScript. Who developed it and in which year?
2. Write any two differences between Java and JavaScript.
3. Write the three ways of including JavaScript in an HTML page.
4. Differentiate between `alert()`, `confirm()` and `prompt()`.
5. Differentiate between `var`, `let` and `const`.
6. What is variable hoisting? Give one example.
7. Differentiate between `==` and `===` with an example.
8. What is the ternary operator? Write its syntax and one example.
9. Differentiate between `while` and `do...while` loops.
10. Differentiate between `break` and `continue`.
11. What is a function? Write the syntax of a function declaration.
12. Differentiate between a parameter and an argument.
13. What is an arrow function? Write one example.
14. Define the DOM. Why is it needed?
15. Differentiate between `getElementById()` and `querySelector()`.

16. Differentiate between `innerHTML` and `textContent`.
17. What is an event? Write any four common events.
18. What is the use of `addEventListener()`? Write its syntax.
19. Differentiate between event bubbling and event capturing.
20. What is the use of `preventDefault()` and `stopPropagation()`?
21. Why is client-side validation not sufficient on its own?
22. Write the JavaScript statement to change the background colour of an element with id `box` to blue.

Group B — Short Answer Questions (5 marks each)

1. What is JavaScript? Explain its features and any five uses in web development.
2. Differentiate between Java and JavaScript with at least six points.
3. Explain the three ways of adding JavaScript to a web page with code examples, and state the advantages and disadvantages of each.
4. Explain variables and scope in JavaScript. Differentiate between `var`, `let` and `const` in tabular form.
5. Explain the data types of JavaScript with examples, and explain type conversion with code.
6. Explain any five categories of operator in JavaScript with examples.
7. Explain conditional statements in JavaScript with syntax and a program that finds the grade of a student from marks.
8. Differentiate between `if...else` and `switch` and write a program using `switch`.
9. Explain the different types of loop in JavaScript with syntax and a code example of each.
10. Write a JavaScript program to display the multiplication table of a number entered by the user.
11. Write a JavaScript program to find the factorial of a number using both a loop and recursion.
12. What is a function? Explain function declarations, function expressions and arrow functions with examples.
13. What is the DOM? Draw the DOM tree of a simple HTML document and explain the node types.
14. Explain any six methods of selecting and manipulating DOM elements with examples.
15. What is an event? Explain the three models of event handling with code examples and compare them.
16. Explain the DOM event model with a diagram of the capturing, target and bubbling phases.
17. What is event delegation? Explain with a code example why it is useful.
18. Explain how to change the colour, font and content of an element dynamically using JavaScript, with a complete code example.
19. Explain how an element can be positioned and animated using JavaScript, with a code example.
20. Differentiate between client-side and server-side validation.
21. Write a JavaScript function to validate an e-mail address and a ten-digit phone number using regular expressions.
22. Write a JavaScript program that changes the background colour of the page every time a button is clicked.

Group C — Long Answer Questions (7.5 marks each)

1. What is JavaScript? Explain its characteristics, uses and limitations. Describe the three ways of including it in a page, and write a complete program that uses variables, an operator, a conditional statement, a loop and a function.
2. Explain control statements in JavaScript in detail. Cover `if`, `if...else`, `if...else if`, nested `if`, the ternary operator and `switch`, and all five kinds of loop, with a flowchart and a complete working program for each category.
3. What is the Document Object Model? Draw and explain the DOM tree of a given HTML document, describe all the methods of selecting elements, and write a complete program that adds, modifies and deletes elements dynamically.
4. Explain events and event handling in JavaScript in complete detail. Describe at least ten events, compare the three event-handling models, explain the capturing, target and bubbling phases with a diagram, and demonstrate event delegation with working code.
5. Explain form validation in detail. Differentiate between client-side and server-side validation, describe HTML5 validation attributes, and write a complete JavaScript program that validates a registration form for a required name, a valid e-mail, a ten-digit phone number, a strong password, a matching confirmation password, a selected radio button and at least one selected check box.
6. Explain how JavaScript is used to change the appearance and position of elements. Cover the `style` property, `classList`, `getComputedStyle()`, and the positioning and size properties, and write a complete interactive page that changes colours and fonts and animates an element across the screen.
7. Explain functions in JavaScript in complete detail. Cover declarations, expressions, arrow functions, default and rest parameters, callbacks, recursion and IIFEs, with a working example of each, and explain scope and hoisting with a diagram.

Laboratory / Practical Exercises — Chapter 4

1. Write a program that takes two numbers from the user with `prompt()` and displays their sum, difference, product and quotient on the page.
2. Write a program that reads marks and displays the grade using an `if...else if` ladder, and repeat it using `switch`.
3. Write a program to print the multiplication table of any number, a star pattern using nested loops, and the first twenty Fibonacci numbers.
4. Write a program to check whether a number is prime, a string is a palindrome, and to reverse an array — each using a separate function.
5. Create a page with a list and buttons that add a new item, delete the last item and change the colour of every item, using DOM methods.
6. Create an image gallery in which clicking a thumbnail changes the large image, using the `click` event and `setAttribute()`.
7. Demonstrate event bubbling with three nested divisions, then use `stopPropagation()` to prevent it. Record your observations.
8. Create a digital clock that updates every second using `setInterval()`.
9. Add complete client-side validation to the registration form you created in Chapter 2, showing error messages next to each field rather than in an alert box.

- 10.** Create a simple calculator page with buttons for the four arithmetic operations, using functions and DOM manipulation.

Chapter 5: Programming in PHP and MySQL

Allotted time: 16 hours

Note: This is the largest chapter of the course and the one that turns a website into a web *application*. Everything you have built so far runs in the browser. PHP runs on the server, where it can read and write a database, handle files, remember who a user is, and send back HTML that no visitor could have produced by editing the page source.

Chapter Learning Outcomes

After completing this chapter you will be able to:

- Explain the server-side model and PHP's place in the web world.
- Write PHP programs using variables, operators, control statements and functions.
- Work with strings, numbers and all three kinds of array.
- Receive and safely process data submitted from an HTML form.
- Read from and write to files on the server.
- Design a database, connect to MySQL and perform complete CRUD operations.
- Use prepared statements to prevent SQL injection.
- Track users with cookies and sessions, and build a login system.

5.1 PHP's Place in the Web World, Basic Rules of PHP Programs, Uses of PHP

5.1.1 Client-Side versus Server-Side Programming

Figure 5.1 How a PHP page is processed

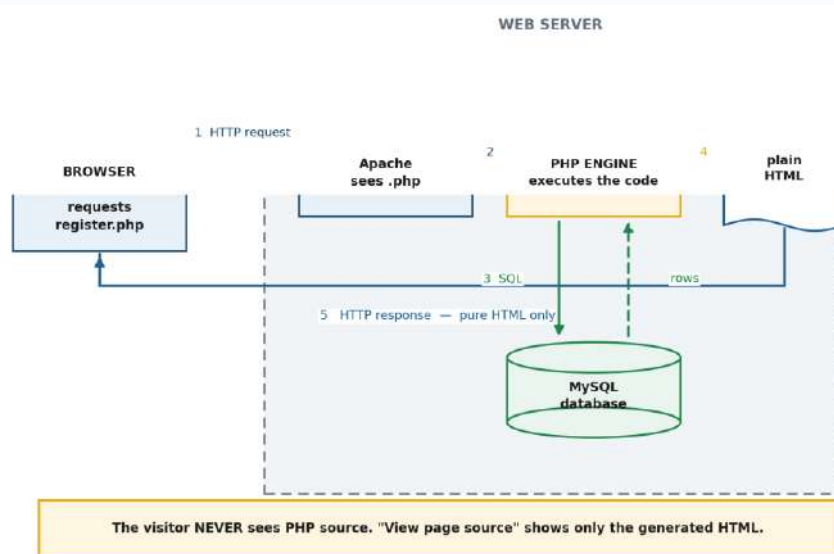


Figure 5.1 — PHP runs entirely on the server and sends back only HTML.

Basis	Client-side scripting (JavaScript)	Server-side scripting (PHP)
Where it runs	In the visitor's browser	On the web server
Source code visibility	Fully visible to anyone	Never visible to the visitor
Database access	Not possible directly	Full access
File system access	Not possible (except a chosen upload)	Full access to server files
Effect on server	None — reduces server load	Consumes server resources
Speed of response	Instant, no network delay	Requires a request and response
Can be disabled by user	Yes	No
Security of validation	Convenience only	Authoritative and trustworthy
Typical tasks	Validation feedback, animation, DOM changes	Authentication, database work, e-mail, payment, file upload

5.1.2 What is PHP?

Definition

PHP is a widely used, open-source, server-side scripting language especially suited to web development, which can be embedded directly inside HTML. It was created by **Rasmus Lerdorf in 1994**. The name was originally *Personal Home Page* and is now the recursive acronym **PHP: Hypertext Preprocessor**.

Features of PHP

- **Open source and free** — no licence cost, with a very large community.
- **Cross platform** — runs on Windows, Linux, macOS and Unix, with Apache, Nginx or IIS.
- **Embedded in HTML** — PHP code can be mixed directly into a page between `<?php` and `?>`.
- **Loosely typed** — a variable's type is decided by the value assigned to it.
- **Database support** — works with MySQL, MariaDB, PostgreSQL, SQLite, Oracle and MongoDB.
- **Large standard library** — thousands of built-in functions for strings, arrays, files, dates, images, e-mail and encryption.
- **Fast** — PHP 8 introduced a JIT compiler and is significantly faster than earlier versions.
- **Session and cookie management** built into the language.

Uses of PHP

Use	Example
Dynamic page generation	Building a page whose content comes from a database

Use	Example
Form processing	Receiving, validating and storing registration or contact data
Database operations	Insert, select, update and delete records (CRUD)
User authentication	Login systems, password hashing, access control
Session management	Shopping carts, "remember me", user dashboards
File handling	Uploading images, generating reports, reading CSV files
E-mail	Sending confirmation, notification and password-reset messages
Content management	WordPress, Drupal and Joomla are all written in PHP
E-commerce	Magento, WooCommerce, OpenCart
APIs	Producing JSON for a mobile app or a JavaScript front end

5.1.3 Basic Rules of PHP Programs

- A PHP file must be saved with the extension `.php` and placed inside the web root (`htdocs` for XAMPP).
- It must be opened through the server as `http://localhost/folder/file.php`, **never** by double-clicking the file.
- PHP code is written between `<?php` and `?>`. In a file containing only PHP, the closing tag should be omitted.
- Every statement ends with a **semicolon**.
- Every variable name begins with a **dollar sign \$**, followed by a letter or underscore.
- **Variable names are case sensitive** (`$name` and `$Name` differ), but **keywords and function names are not** (`ECHO` and `echo` are the same).
- Comments are written `// single line`, `# single line` or `/* multi-line */`.

PHP

```

<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>My First PHP Page</title></head>
<body>

<h1>Welcome</h1>

<?php
// this is a single-line comment
# this is also a single-line comment
/* this is a
   multi-line comment */

$name = "Sita Gurung"; // a variable
echo "<p>Hello, $name!</p>";
echo "<p>Today is " . date("l, d F Y") . "</p>";
?>

```

```

<p>This line is ordinary HTML again.</p>

<?php for ($i = 1; $i <= 3; $i++) { ?>
    <p>Loop pass number <?php echo $i; ?></p>
<?php } ?>

</body>
</html>

```

Note: `echo` and `print` both output text. `echo` is marginally faster, can take several comma-separated arguments and returns nothing; `print` takes one argument and returns 1, so it can be used in an expression. `echo` is the usual choice. The shorthand `<?= $name ?>` is equivalent to `<?php echo $name; ?>`.

5.2 Text, Numbers and Variables in PHP

5.2.1 Variables and Data Types

```

PHP

<?php
$name  = "Ram Thapa";    // string
$age   = 20;             // integer
$marks = 78.5;           // float (double)
$passed = true;          // boolean
$subjects = ["Web", "DSA"]; // array
$empty  = NULL;          // null

echo gettype($age);      // integer
var_dump($marks);        // float(78.5) -> type AND value
print_r($subjects);      // readable array output

// CONSTANTS: no dollar sign, cannot be changed
define("SITE_NAME", "Gandaki University");
const MAX_MARKS = 100;
echo SITE_NAME;

// VARIABLE SCOPE
$globalVar = "I am global";
function testScope() {
    global $globalVar;    // one way to reach it
    echo $globalVar;
    echo $GLOBALS["globalVar"]; // another way
    static $counter = 0;  // keeps its value between calls
    $counter++;
    echo $counter;
}
?>

```

Data type	Description	Example
String	A sequence of characters	<code>\$s = "Hello";</code>
Integer	A whole number	<code>\$n = 42;</code>

Data type	Description	Example
Float / double	A number with a decimal part	<code>\$f = 3.14;</code>
Boolean	<code>TRUE</code> or <code>FALSE</code>	<code>\$b = true;</code>
Array	An ordered map of key–value pairs	<code>\$a = [1, 2, 3];</code>
Object	An instance of a class	<code>\$o = new Student();</code>
NULL	A variable with no value	<code>\$x = NULL;</code>
Resource	A handle to an external resource such as a file or a database connection	<code>\$conn = mysqli_connect(...)</code>

5.2.2 Working with Strings

Single versus double quotes — an important distinction

```

PHP
<?php
$name = "Sita";

echo "Hello $name";    // Hello Sita -> variables ARE parsed
echo 'Hello $name';    // Hello $name -> taken literally
echo "Hello {$name}!"; // braces remove any ambiguity
echo "Line one\nLine two"; // \n works only in double quotes

// CONCATENATION uses a dot, not a plus
$first = "Web"; $last = "Technology";
$full = $first . " " . $last;    // "Web Technology"
$full .= " I";                  // append
?>

```

Essential string functions

Function	Purpose	Example → result
<code>strlen(\$s)</code>	Length in bytes	<code>strlen("Hello")</code> → 5
<code>strtoupper(\$s)</code> / <code>strtolower(\$s)</code>	Change case	<code>strtoupper("php")</code> → "PHP"
<code>ucfirst(\$s)</code> / <code>ucwords(\$s)</code>	Capitalise the first letter / every word	<code>ucwords("ram thapa")</code> → "Ram Thapa"
<code>trim(\$s)</code>	Remove whitespace from both ends	<code>trim(" hi ")</code> → "hi"
<code>str_replace(\$a,\$b,\$s)</code>	Replace text	<code>str_replace("a","o","cat")</code> → "cot"
<code>substr(\$s,\$start,\$len)</code>	Extract part of a string	<code>substr("Gandaki",0,3)</code> → "Gan"
<code>strpos(\$s,\$find)</code>	Position of the first occurrence, or <code>false</code>	<code>strpos("hello","l")</code> → 2
<code>explode(\$sep,\$s)</code>	Split a string into an array	<code>explode(",", "a,b,c")</code> → ["a","b","c"]
<code>implode(\$sep,\$arr)</code>	Join an array into a string	<code>implode("-", [1,2,3])</code> → "1-2-3"
<code>str_repeat(\$s,\$n)</code>	Repeat a string	<code>str_repeat("ab",3)</code> → "ababab"
<code>strrev(\$s)</code>	Reverse a string	<code>strrev("abc")</code> → "cba"

Function	Purpose	Example → result
<code>nl2br(\$s)</code>	Convert newlines into <code>
</code>	used when displaying a textarea value
<code>sprintf(\$fmt, ...)</code>	Format a string	<code>sprintf("%.2f", 3.14159) → "3.14"</code>
<code>htmlspecialchars(\$s)</code>	Escape HTML characters — essential for security	<code></code> becomes <code>&lt;b&gt;</code> ;

5.2.3 Working with Numbers

Function	Purpose	Example → result
<code>abs(\$n)</code>	Absolute value	<code>abs(-5) → 5</code>
<code>round(\$n, \$p)</code>	Round to <code>\$p</code> decimal places	<code>round(3.567, 2) → 3.57</code>
<code>ceil(\$n) / floor(\$n)</code>	Round up / down	<code>ceil(4.1) → 5</code> , <code>floor(4.9) → 4</code>
<code>sqrt(\$n)</code>	Square root	<code>sqrt(16) → 4</code>
<code>pow(\$b,\$e)</code>	Power	<code>pow(2,10) → 1024</code>
<code>max() / min()</code>	Largest / smallest value	<code>max(3,9,5) → 9</code>
<code>rand(\$a,\$b)</code>	Random integer in a range	<code>rand(1,100)</code>
<code>number_format(\$n,\$d)</code>	Format with thousands separators	<code>number_format(1234567.891, 2) → "1,234,567.89"</code>
<code>intdiv(\$a,\$b)</code>	Integer division	<code>intdiv(10,3) → 3</code>
<code>is_numeric(\$v)</code>	Check whether a value is a number	<code>is_numeric("25") → true</code>
<code>fmod(\$a,\$b)</code>	Floating-point remainder	<code>fmod(10,3) → 1</code>

PHP

```
<?php
$price = 1250.756;
echo number_format($price, 2);      // 1,250.76
echo round($price);                 // 1251
echo intval("25 apples");           // 25
echo (int) "3.9";                    // 3
echo 10 / 3;                         // 3.3333333333333
echo 10 % 3;                         // 1
echo 2 ** 10;                       // 1024
echo PHP_INT_MAX;                   // 9223372036854775807
?>
```

5.2.4 Operators in PHP

Category	Operators	Notes
Arithmetic	<code>+ - * / % **</code>	As in mathematics
Assignment	<code>= += -= *= /= %= .=</code>	<code>.=</code> appends to a string

Category	Operators	Notes
Comparison	<code>== === != <> !== > < >= <= <=></code>	<code>===</code> also compares the type; <code><=></code> is the spaceship operator
Increment / decrement	<code>++ --</code>	Pre and post forms
Logical	<code>&& ! and or xor</code>	<code>&&</code> binds more tightly than <code>and</code>
String	<code>.</code> and <code>.=</code>	Concatenation — not <code>+</code>
Array	<code>+</code> <code>==</code> <code>===</code> <code>!=</code> <code>!==</code>	<code>+</code> is union, not addition
Null coalescing	<code>??</code> and <code>??=</code>	<code>\$name = \$_POST["name"] ?? "Guest";</code> — avoids an undefined-index notice
Ternary	<code>?:</code>	<code>\$r = (\$m >= 40) ? "Pass" : "Fail";</code>
Error control	<code>@</code>	Suppresses errors — avoid it

Common Mistake: In PHP the dot `.` joins strings and the plus `+` adds numbers. Writing `"Hello" + "World"` is an error in PHP 8, not a concatenation. This differs from JavaScript and is a very common mistake.

5.3 Primitives, Operations, Expressions and Control Statements

5.3.1 Conditional Statements

```

PHP
<?php
// ----- if / elseif / else -----
$marks = 78;

if ($marks >= 90) { $grade = "A+"; }
elseif ($marks >= 80) { $grade = "A"; }
elseif ($marks >= 70) { $grade = "B+"; }
elseif ($marks >= 60) { $grade = "B"; }
elseif ($marks >= 40) { $grade = "C"; }
else { $grade = "F"; }
echo "Grade: $grade";

// ----- ternary and null coalescing -----
$result = ($marks >= 40) ? "Pass" : "Fail";
$user = $_GET["user"] ?? "Guest";

// ----- switch -----
$day = 3;
switch ($day) {
    case 1: $name = "Sunday"; break;
    case 2: $name = "Monday"; break;
    case 3: $name = "Tuesday"; break;
    default: $name = "Invalid";
}

// ----- match (PHP 8): strict, returns a value -----
$type = match(true) {
    $marks >= 80 => "Distinction",
    $marks >= 60 => "First Division",

```

```

    $marks >= 40 => "Pass",
    default    => "Fail",
};

// ----- alternative syntax: ideal when mixing with HTML ----
?>
<?php if ($marks >= 40): ?>
    <p class="pass">Congratulations, you passed.</p>
<?php else: ?>
    <p class="fail">Please appear in the re-examination.</p>
<?php endif; ?>

```

5.3.2 Looping Statements

```

PHP

<?php
// ----- for -----
for ($i = 1; $i <= 5; $i++) {
    echo "Row $i<br>";
}

// ----- while -----
$i = 1;
while ($i <= 5) { echo $i; $i++; }

// ----- do...while: runs at least once -----
$j = 10;
do { echo "runs once"; $j++; } while ($j < 5);

// ----- foreach: THE loop for arrays -----
$subjects = ["Web Tech", "DSA", "Maths"];
foreach ($subjects as $subject) {
    echo "<li>$subject</li>";
}

$student = ["name" => "Sita", "roll" => 1, "marks" => 85];
foreach ($student as $key => $value) {
    echo "$key : $value <br>";
}

// ----- break and continue -----
for ($k = 1; $k <= 10; $k++) {
    if ($k == 4) continue; // skip 4
    if ($k == 8) break;    // stop at 8
    echo $k;              // 1 2 3 5 6 7
}

// ----- a practical example: an HTML table from a loop ----
echo "<table border='1'>";
for ($i = 1; $i <= 5; $i++) {
    echo "<tr>";
    for ($j = 1; $j <= 5; $j++) {
        echo "<td>" . ($i * $j) . "</td>";
    }
    echo "</tr>";
}
echo "</table>";
?>

```

5.3.3 Include and Require — Reusing Code

PHP

```
<?php
include "header.php";    // warning if missing, script CONTINUES
require "config.php";    // FATAL error if missing, script STOPS
include_once "menu.php"; // includes only if not already included
require_once "db.php";   // the safest choice for a config file
?>
```

Basis	include	require
File missing	Emits a warning	Emits a fatal error
Script execution	Continues with the rest of the page	Stops immediately
Use for	Optional parts such as a sidebar or a banner	Essential files such as the database configuration
_once variant	include_once	require_once

Note: Splitting a site into **header.php**, **footer.php** and **db.php** and including them everywhere is the standard way to avoid repeating code. Change the menu once and every page updates.

5.4 Arrays, Functions and File Handling

5.4.1 Arrays

Definition

An **array** in PHP is an ordered map that associates keys with values. It can behave as a classic list, a dictionary, a stack or a queue. PHP has three kinds of array.

Type	Description	Example
Indexed array	Keys are automatic integers starting from 0	<code>\$a = ["Web", "DSA", "Maths"];</code>
Associative array	Keys are strings that you choose	<code>\$s = ["name" => "Sita", "roll" => 1];</code>
Multidimensional array	An array whose elements are themselves arrays	<code>\$m = [["Sita", 85], ["Ram", 78]];</code>

PHP

```
<?php
/* ----- 1. INDEXED ARRAY ----- */
$subjects = ["Web Tech", "DSA", "Maths"]; // short syntax
$colours = array("red", "green", "blue"); // older syntax

echo $subjects[0];    // Web Tech
$subjects[] = "English"; // append
echo count($subjects); // 4

foreach ($subjects as $index => $subject) {
```

```

    echo "$index : $subject <br>";
}

/* ----- 2. ASSOCIATIVE ARRAY ----- */
$student = [
    "name" => "Sita Gurung",
    "roll" => "BIT-2026-001",
    "email" => "sita@gu.edu.np",
    "marks" => 85
];
echo $student["name"];
$student["phone"] = "9800000000"; // add a new pair
unset($student["email"]); // remove a pair

foreach ($student as $key => $value) {
    echo ucfirst($key) . " : $value <br>";
}

/* ----- 3. MULTIDIMENSIONAL ARRAY ----- */
$students = [
    ["name" => "Sita", "marks" => 85, "grade" => "A"],
    ["name" => "Ram", "marks" => 78, "grade" => "B+"],
    ["name" => "Hari", "marks" => 92, "grade" => "A+"]
];

echo "<table border='1'>";
echo "<tr><th>Name</th><th>Marks</th><th>Grade</th></tr>";
foreach ($students as $s) {
    echo "<tr><td>{$s['name']}</td>";
    echo "<td>{$s['marks']}</td>";
    echo "<td>{$s['grade']}</td></tr>";
}
echo "</table>";
?>

```

Essential array functions

Function	Purpose
<code>count(\$a)</code>	Number of elements
<code>array_push(\$a, \$v) / array_pop(\$a)</code>	Add / remove at the end
<code>array_shift(\$a) / array_unshift(\$a,\$v)</code>	Remove / add at the beginning
<code>in_array(\$v, \$a)</code>	Does the value exist? Returns true or false
<code>array_search(\$v, \$a)</code>	Returns the key of a value, or <code>false</code>
<code>array_key_exists(\$k, \$a)</code>	Does the key exist?
<code>array_keys(\$a) / array_values(\$a)</code>	All the keys / all the values as a new array
<code>sort() / rsort()</code>	Sort an indexed array ascending / descending
<code>asort() / arsort()</code>	Sort an associative array by value , keeping keys
<code>krsort() / ksort()</code>	Sort an associative array by key
<code>array_merge(\$a, \$b)</code>	Join two arrays
<code>array_slice(\$a, \$off, \$len)</code>	Extract a portion

Function	Purpose
<code>array_map(\$fn, \$a)</code>	Apply a function to every element
<code>array_filter(\$a, \$fn)</code>	Keep only the elements that satisfy a test
<code>array_sum(\$a) / array_product(\$a)</code>	Total / product of all values
<code>array_unique(\$a)</code>	Remove duplicate values
<code>array_reverse(\$a)</code>	Reverse the order
<code>explode()</code> / <code>implode()</code>	Convert between a string and an array
<code>print_r(\$a)</code> / <code>var_dump(\$a)</code>	Display an array for debugging

PHP

```
<?php
$marks = [78, 85, 92, 65, 85];

echo array_sum($marks);           // 405
echo count($marks);               // 5
echo array_sum($marks) / count($marks); // 81 -> the average
echo max($marks) . " " . min($marks); // 92 65

sort($marks);      print_r($marks); // ascending
rsort($marks);     print_r($marks); // descending
print_r(array_unique($marks));      // duplicates removed
print_r(array_filter($marks, fn($m) => $m >= 80));
print_r(array_map(fn($m) => $m + 5, $marks)); // grace marks
?>
```

5.4.2 Functions**PHP**

```
<?php
/* ----- a simple function ----- */
function greet($name) {
    return "Namaste, $name!";
}
echo greet("Sita");

/* ----- default parameter values ----- */
function calculateTotal($theory, $lab, $project = 0) {
    return $theory + $lab + $project;
}
echo calculateTotal(18, 14); // 32
echo calculateTotal(18, 14, 5); // 37

/* ----- type declarations (PHP 7+) ----- */
function getGrade(float $marks): string {
    if ($marks >= 90) return "A+";
    if ($marks >= 80) return "A";
    if ($marks >= 60) return "B";
    if ($marks >= 40) return "C";
    return "F";
}
```

```

/* ----- pass by value versus pass by reference ----- */
function addTenByValue($n)    { $n += 10; }
function addTenByRef(&$n)     { $n += 10; } // note the &

$x = 5; addTenByValue($x); echo $x; // 5 -> unchanged
$y = 5; addTenByRef($y);  echo $y; // 15 -> changed

/* ----- variable number of arguments ----- */
function sumAll(...$numbers) {
    return array_sum($numbers);
}
echo sumAll(1, 2, 3, 4, 5);    // 15

/* ----- returning an array ----- */
function getStats(array $marks): array {
    return [
        "total" => array_sum($marks),
        "average" => round(array_sum($marks) / count($marks), 2),
        "highest" => max($marks),
        "lowest" => min($marks)
    ];
}
$stats = getStats([78, 85, 92]);
echo "Average: {$stats['average']}";

/* ----- recursion ----- */
function factorial(int $n): int {
    return ($n <= 1) ? 1 : $n * factorial($n - 1);
}
echo factorial(5);           // 120

/* ----- anonymous function and arrow function ----- */
$square = function ($n) { return $n * $n; };
$cube = fn($n) => $n ** 3; // PHP 7.4+
echo $square(4) . " " . $cube(3); // 16 27
?>

```

Useful built-in date and time functions

```

PHP
<?php
echo date("Y-m-d");           // 2026-07-18
echo date("d/m/Y H:i:s");     // 18/07/2026 14:35:20
echo date("l, d F Y");        // Saturday, 18 July 2026
echo time();                  // Unix timestamp
echo date("Y-m-d", strtotime("+7 days"));

date_default_timezone_set("Asia/Kathmandu");

$dob = new DateTime("2005-03-15");
$now = new DateTime();
echo $now->diff($dob)->y . " years old";
?>

```

5.5 Handling Form Data in PHP

5.5.1 Superglobal Variables

A **superglobal** is a built-in array that PHP makes available in every scope without any declaration.

Superglobal	Contains
<code>\$_GET</code>	Data sent through the URL query string or a form with <code>method="get"</code>
<code>\$_POST</code>	Data sent in the body of a form with <code>method="post"</code>
<code>\$_REQUEST</code>	The contents of <code>\$_GET</code> , <code>\$_POST</code> and <code>\$_COOKIE</code> combined — avoid it, since the source is ambiguous
<code>\$_FILES</code>	Information about uploaded files
<code>\$_SESSION</code>	Data stored on the server for the current visitor
<code>\$_COOKIE</code>	Data stored in the visitor's browser
<code>\$_SERVER</code>	Information about the server and the current request, such as <code>REQUEST_METHOD</code> , <code>PHP_SELF</code> , <code>REMOTE_ADDR</code>
<code>\$_ENV</code>	Environment variables
<code>\$GLOBALS</code>	All variables in the global scope

5.5.2 Receiving and Validating Form Data

This script is `register.php`, the server-side counterpart of the registration form from Chapter 2 and the JavaScript validation from Chapter 4. It repeats every check on the server, because client-side validation can always be bypassed.

```

PHP
<?php
/* ===== file: register.php ===== */

// initialise the variables that hold errors and values
$errors = [];
$name = $email = $phone = $gender = $programme = "";
$success = false;

/* only process when the page was reached by a POST submission */
if ($_SERVER["REQUEST_METHOD"] === "POST") {

    /* ----- a helper that cleans every input ----- */
    function cleanInput($data) {
        $data = trim($data);           // remove outer spaces
        $data = stripslashes($data);  // remove added slashes
        $data = htmlspecialchars($data); // neutralise HTML/JS
        return $data;
    }

    /* ----- 1. FULL NAME ----- */
    if (empty($_POST["fullname"])) {
        $errors["fullname"] = "Full name is required.";
    } else {

```

```

$name = cleanInput($_POST["fullname"]);
if (!preg_match("/^[a-zA-Z ]{3,50}$/", $name)) {
    $errors["fullname"] =
        "Name must be 3-50 letters and spaces only.";
}
}

/* ----- 2. EMAIL ----- */
if (empty($_POST["email"])) {
    $errors["email"] = "Email address is required.";
} else {
    $email = cleanInput($_POST["email"]);
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errors["email"] = "Please enter a valid email address.";
    }
}

/* ----- 3. PHONE (optional) ----- */
if (!empty($_POST["phone"])) {
    $phone = cleanInput($_POST["phone"]);
    if (!preg_match("/^[0-9]{10}$/", $phone)) {
        $errors["phone"] = "Phone must be exactly 10 digits.";
    }
}

/* ----- 4. PASSWORD ----- */
$password = $_POST["password"] ?? "";
$cpassword = $_POST["cpassword"] ?? "";
if (strlen($password) < 8) {
    $errors["password"] = "Password must be at least 8 characters.";
} elseif ($password !== $cpassword) {
    $errors["cpassword"] = "The two passwords do not match.";
}

/* ----- 5. RADIO BUTTON ----- */
if (empty($_POST["gender"])) {
    $errors["gender"] = "Please select a gender.";
} else {
    $gender = cleanInput($_POST["gender"]);
}

/* ----- 6. SELECT BOX ----- */
if (empty($_POST["programme"])) {
    $errors["programme"] = "Please select a programme.";
} else {
    $programme = cleanInput($_POST["programme"]);
}

/* ----- 7. CHECK BOXES (an array) ----- */
$interests = $_POST["interest"] ?? [];
if (count($interests) === 0) {
    $errors["interest"] = "Select at least one interest.";
}
$interestList = implode(", ", array_map("cleanInput", $interests));

/* ----- 8. FILE UPLOAD ----- */
$photoName = "";
if (!empty($_FILES["photo"]["name"])) {
    $allowed = ["image/jpeg", "image/png"];
    $maxSize = 2 * 1024 * 1024; // 2 MB
    $tmp = $_FILES["photo"]["tmp_name"];
    $type = mime_content_type($tmp);
    $size = $_FILES["photo"]["size"];
}

```

```

    if (!in_array($type, $allowed)) {
        $errors["photo"] = "Only JPG and PNG images are allowed.";
    } elseif ($size > $maxSize) {
        $errors["photo"] = "The image must be smaller than 2 MB.";
    } else {
        $ext = pathinfo($_FILES["photo"]["name"],
            PATHINFO_EXTENSION);
        $photoName = uniqid("img_") . "." . $ext; // unique name
        if (!is_dir("uploads")) { mkdir("uploads", 0755); }
        move_uploaded_file($tmp, "uploads/" . $photoName);
    }
}

/* ----- if everything passed, store the record ----- */
if (empty($errors)) {
    // hash the password — NEVER store it as plain text
    $hashed = password_hash($password, PASSWORD_DEFAULT);
    // the database INSERT is shown in section 5.6
    $success = true;
}
}
?>

<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Registration Result</title></head>
<body>
<?php if ($success): ?>
    <h2 style="color:green">Registration successful</h2>
    <p>Name: <?= $name ?></p>
    <p>Email: <?= $email ?></p>
    <p>Programme: <?= strtoupper($programme) ?></p>
    <p>Interests: <?= $interestList ?></p>
<?php elseif (!empty($errors)): ?>
    <h2 style="color:#c0392b">Please correct the following:</h2>
    <ul>
    <?php foreach ($errors as $field => $message): ?>
        <li><?= $message ?></li>
    <?php endforeach; ?>
    </ul>
<?php endif; ?>
</body>
</html>

```

Important security functions

Function	Protects against
<code>htmlspecialchars(\$s)</code>	Cross-site scripting (XSS) — converts <code><</code> , <code>></code> , <code>"</code> and <code>&</code> into entities so injected script tags are displayed rather than executed.
<code>filter_var(\$v, FILTER_VALIDATE_EMAIL)</code>	Invalid e-mail addresses.
<code>filter_var(\$v, FILTER_SANITIZE_STRING)</code>	Unwanted tags in input (deprecated in PHP 8.1; use <code>htmlspecialchars</code>).
<code>password_hash()</code> / <code>password_verify()</code>	Stolen passwords — stores a salted bcrypt hash instead of the plain password.

Function	Protects against
Prepared statements	SQL injection — covered in section 5.6.
<code>trim()</code> and <code>stripslashes()</code>	Stray whitespace and escape characters.

Common Mistake: Never store a password as plain text and never encrypt it with a reversible algorithm. Use `password_hash($password, PASSWORD_DEFAULT)` when registering and `password_verify($entered, $storedHash)` when logging in.

5.6 File Handling in PHP

PHP can create, read, write, append to and delete files on the server. File handling is used for logs, simple data storage, report generation and uploads.

Function	Purpose
<code>fopen(\$file, \$mode)</code>	Open a file and return a handle
<code>fread(\$handle, \$bytes)</code>	Read a given number of bytes
<code>fgets(\$handle)</code>	Read one line
<code>fwrite(\$handle, \$text)</code>	Write to the file
<code>fclose(\$handle)</code>	Close the file — always do this
<code>feof(\$handle)</code>	True when the end of the file is reached
<code>file_get_contents(\$file)</code>	Read the whole file into a string in one call
<code>file_put_contents(\$file, \$data)</code>	Write a whole string to a file in one call
<code>file(\$file)</code>	Read the file into an array of lines
<code>file_exists(\$file)</code>	Check whether the file exists
<code>unlink(\$file)</code>	Delete the file
<code>copy(\$src, \$dest)</code> / <code>rename()</code>	Copy or rename
<code>filesize(\$file)</code>	Size in bytes
<code>mkdir()</code> / <code>rmdir()</code> / <code>scandir()</code>	Directory operations
<code>move_uploaded_file(\$tmp, \$dest)</code>	Move an uploaded file into place

File open modes

Mode	Meaning	If the file does not exist	Pointer starts at
<code>r</code>	Read only	Returns <code>false</code>	Beginning
<code>r+</code>	Read and write	Returns <code>false</code>	Beginning

Mode	Meaning	If the file does not exist	Pointer starts at
w	Write only — erases all existing content	Creates it	Beginning
w+	Read and write — erases content	Creates it	Beginning
a	Append only — keeps existing content	Creates it	End
a+	Read and append	Creates it	End
x	Create and write only	Creates it; fails if it already exists	Beginning

PHP

```

<?php
/* ----- WRITING (the simple way) ----- */
$data = "Sita Gurung, BIT, 85\n";
file_put_contents("students.txt", $data, FILE_APPEND);

/* ----- WRITING (the classic way) ----- */
$file = fopen("students.txt", "a") or die("Cannot open the file");
fwrite($file, "Ram Thapa, BIT, 78\n");
fwrite($file, "Hari K.C., BCA, 92\n");
fclose($file);

/* ----- READING the whole file ----- */
if (file_exists("students.txt")) {
    echo nl2br(file_get_contents("students.txt"));
}

/* ----- READING line by line ----- */
$file = fopen("students.txt", "r");
while (!feof($file)) {
    $line = fgets($file);
    if (trim($line) === "") continue;
    $parts = explode(" ", $line);
    echo "<tr><td>{$parts[0]}</td><td>{$parts[1]}</td>";
    "<td>{$parts[2]}</td></tr>";
}
fclose($file);

/* ----- READING into an array ----- */
$lines = file("students.txt", FILE_IGNORE_NEW_LINES |
    FILE_SKIP_EMPTY_LINES);
foreach ($lines as $i => $line) { echo ($i+1) . ". $line <br>"; }

/* ----- other operations ----- */
echo filesize("students.txt") . " bytes";
copy("students.txt", "backup.txt");
// unlink("backup.txt"); // delete
?>

```

File upload — the complete pattern**HTML**

```

<!-- the form MUST use post and multipart/form-data -->
<form action="upload.php" method="post" enctype="multipart/form-data">
    <input type="file" name="document" accept=".pdf,.jpg,.png">

```

```

<button type="submit" name="submit">Upload</button>
</form>

PHP
<?php
/* ===== file: upload.php ===== */
if (isset($_POST["submit"]) && !empty($_FILES["document"]["name"])) {

    $targetDir = "uploads/";
    if (!is_dir($targetDir)) { mkdir($targetDir, 0755, true); }

    $original = basename($_FILES["document"]["name"]);
    $ext      = strtolower(pathinfo($original, PATHINFO_EXTENSION));
    $tmpPath  = $_FILES["document"]["tmp_name"];
    $size     = $_FILES["document"]["size"];
    $error    = $_FILES["document"]["error"];

    $allowedExt = ["pdf", "jpg", "jpeg", "png"];
    $maxSize    = 5 * 1024 * 1024; // 5 MB

    if ($error !== UPLOAD_ERR_OK) {
        echo "Upload error code: $error";
    } elseif (!in_array($ext, $allowedExt)) {
        echo "Only PDF, JPG and PNG files are allowed.";
    } elseif ($size > $maxSize) {
        echo "The file must be smaller than 5 MB.";
    } else {
        // a unique name prevents overwriting and hides the original
        $newName = uniqid("file_", true) . "." . $ext;
        $target  = $targetDir . $newName;

        if (move_uploaded_file($tmpPath, $target)) {
            echo "Uploaded successfully as $newName";
        } else {
            echo "Failed to move the uploaded file.";
        }
    }
}
?>

```

Common Mistake: Never trust the file extension supplied by the browser and never move an uploaded file into a folder where PHP can execute it. Check the real MIME type with `mime_content_type()`, rename the file, and store uploads outside the executable web path where possible.

5.7 Working with Databases: Organizing Data, Connecting, Creating a Table, CRUD

5.7.1 Organizing Data in a Database

Definitions

Term	Meaning
Database	An organised collection of related data stored so that it can be retrieved and managed efficiently.

Term	Meaning
DBMS	Database Management System — software that creates and manages databases, e.g. MySQL, Oracle, SQL Server.
RDBMS	Relational DBMS — stores data in tables that can be related to one another through keys. MySQL is an RDBMS.
Table (relation)	A structure of rows and columns holding data about one type of entity, such as students.
Record (row / tuple)	One complete entry — all the data about one student.
Field (column / attribute)	One property of the entity, such as the e-mail address.
Primary key	A column whose value uniquely identifies each row. It cannot be null and cannot repeat.
Foreign key	A column that refers to the primary key of another table, creating a relationship.
SQL	Structured Query Language — the standard language for talking to a relational database.

MySQL

MySQL is a free, open-source relational database management system, originally released in 1995 and now owned by Oracle. It is the "M" in the LAMP, WAMP and XAMPP stacks and is by far the most common database used with PHP. Its community fork, **MariaDB**, is what XAMPP actually installs today and is fully compatible.

Figure 5.2 A database, its tables, records, fields and keys



Figure 5.2 — The foreign key `student\_id` links each mark back to a student.

Common MySQL data types

Type	Description	Example use
INT	Whole number	id, age, quantity
BIGINT	A very large whole number	transaction id
DECIMAL(m,d)	Exact decimal with m digits and d after the point	price, marks — use for money
FLOAT / DOUBLE	Approximate decimal	scientific measurements
VARCHAR(n)	Variable-length text up to n characters	name, email
CHAR(n)	Fixed-length text	country code, gender
TEXT	Long text, up to 65 535 characters	address, description
DATE	YYYY-MM-DD	date of birth
DATETIME	YYYY-MM-DD HH:MM:SS	created_at
TIMESTAMP	Automatic date and time	last updated
ENUM(...)	One value from a fixed list	gender, status
BOOLEAN	Stored as TINYINT(1)	is_active

Essential SQL statements

SQL

-- ----- create the database and select it -----

```
CREATE DATABASE IF NOT EXISTS gu_web
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE gu_web;
```

-- ----- create a table -----

```
CREATE TABLE IF NOT EXISTS students (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  fullname VARCHAR(100) NOT NULL,
  email   VARCHAR(120) NOT NULL UNIQUE,
  phone   VARCHAR(15),
  password VARCHAR(255) NOT NULL,
  gender  ENUM("male","female","other") NOT NULL,
  programme VARCHAR(20) NOT NULL,
  semester INT DEFAULT 1,
  interests VARCHAR(255),
  photo    VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

-- ----- INSERT: create a record -----

```
INSERT INTO students (fullname, email, phone, password, gender,
  programme, semester)
VALUES ("Sita Gurung", "sita@gu.edu.np", "9800000000",
  "$2y$10$hashedvalue", "female", "BIT", 2);
```

-- ----- SELECT: read records -----

```
SELECT * FROM students;
SELECT fullname, email FROM students WHERE programme = "BIT";
SELECT * FROM students WHERE semester = 2 ORDER BY fullname ASC;
SELECT * FROM students WHERE fullname LIKE "S%";
```

```
SELECT programme, COUNT(*) AS total FROM students GROUP BY programme;
SELECT * FROM students LIMIT 10 OFFSET 0;
```

```
-- ----- UPDATE: modify a record -----
```

```
UPDATE students SET phone = "9811111111", semester = 3
WHERE id = 1;
```

```
-- ----- DELETE: remove a record -----
```

```
DELETE FROM students WHERE id = 5;
```

```
-- ----- ALTER: change the structure -----
```

```
ALTER TABLE students ADD address VARCHAR(200);
ALTER TABLE students MODIFY phone VARCHAR(20);
ALTER TABLE students DROP COLUMN address;
```

```
-- ----- JOIN two tables -----
```

```
SELECT s.fullname, m.subject, m.marks
FROM students s
INNER JOIN marks m ON s.id = m.student_id
WHERE m.marks >= 60
ORDER BY s.fullname;
```

Common Mistake: An **UPDATE** or **DELETE** without a **WHERE** clause changes or removes **every row in the table**. Always write the **WHERE** clause first, and test with a **SELECT** before running the change.

5.7.2 Connecting PHP to MySQL

PHP offers two modern ways to connect: the **MySQLi** extension (MySQL only, procedural or object-oriented) and **PDO** (PHP Data Objects, which works with twelve different database systems). The old **mysql_*** functions were removed in PHP 7 and must never be used.

PHP

```
<?php
/* ===== file: db.php — MySQLi procedural ===== */
$host = "localhost";
$username = "root";
$password = ""; // XAMPP default is an empty password
$database = "gu_web";

$conn = mysqli_connect($host, $username, $password, $database);

if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
mysqli_set_charset($conn, "utf8mb4");
// echo "Connected successfully";
?>
```

PHP

```
<?php
/* ===== MySQLi object-oriented ===== */
$conn = new mysqli("localhost", "root", "", "gu_web");
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
```

```

$conn->set_charset("utf8mb4");
?>

PHP
<?php
/* ===== PDO — the most flexible option ===== */
try {
    $pdo = new PDO("mysql:host=localhost;dbname=gu_web;charset=utf8mb4",
        "root", "");
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE,
        PDO::FETCH_ASSOC);
} catch (PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}
?>

```

Difference between MySQLi and PDO

Basis	MySQLi	PDO
Database support	MySQL and MariaDB only	Twelve systems including MySQL, PostgreSQL, SQLite, Oracle
API style	Both procedural and object-oriented	Object-oriented only
Named parameters	Not supported (only <code>?</code> placeholders)	Supported (<code>:name</code>)
Prepared statements	Supported	Supported
Error handling	Warnings, or exceptions if configured	Exceptions by default
Portability	Low — changing database means rewriting	High — usually only the connection string changes
Best for	Simple projects that will always use MySQL	Professional projects and frameworks

5.7.3 CRUD Operations — the Complete Application

CRUD stands for **Create, Read, Update, Delete** — the four operations every data-driven application performs. The following files form a complete, working student-management system.

Operation	SQL statement	PHP file	HTTP method
Create	INSERT	<code>create.php</code>	POST
Read	SELECT	<code>read.php</code>	GET
Update	UPDATE	<code>update.php</code>	POST
Delete	DELETE	<code>delete.php</code>	GET or POST

(a) CREATE — inserting a record with a prepared statement

```

PHP
<?php
/* ===== file: create.php ===== */
require_once "db.php";

$message = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {

    $name    = trim($_POST["fullname"]);
    $email    = trim($_POST["email"]);
    $phone    = trim($_POST["phone"]);
    $gender   = $_POST["gender"];
    $programme = $_POST["programme"];
    $semester = (int) $_POST["semester"];
    $hashedPwd = password_hash($_POST["password"], PASSWORD_DEFAULT);

    /* PREPARED STATEMENT — the correct, safe way
       Step 1: prepare the SQL with ? placeholders
       Step 2: bind the actual values, stating their types
       Step 3: execute
    */

    $sql = "INSERT INTO students
            (fullname, email, phone, password, gender,
             programme, semester)
            VALUES (?, ?, ?, ?, ?, ?)";

    $stmt = mysqli_prepare($conn, $sql);

    // "sssssi" = 6 strings then 1 integer, in order
    mysqli_stmt_bind_param($stmt, "sssssi", $name, $email, $phone,
                            $hashedPwd, $gender, $programme, $semester);

    if (mysqli_stmt_execute($stmt)) {
        $newId = mysqli_insert_id($conn);
        $message = "Record inserted successfully. New ID: $newId";
    } else {
        $message = "Error: " . mysqli_stmt_error($stmt);
    }

    mysqli_stmt_close($stmt);
}
?>

<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Add Student</title></head>
<body>
<h2>Add a New Student</h2>
<?php if ($message): ?><p><?= htmlspecialchars($message) ?></p>
<?php endif; ?>

<form method="post">
<input type="text" name="fullname" placeholder="Full name"
        required><br>
<input type="email" name="email" placeholder="Email"
        required><br>
<input type="tel" name="phone" placeholder="Phone"><br>
<input type="password" name="password" placeholder="Password"
        required><br>

```

```

<select name="gender" required>
  <option value="">Gender</option>
  <option value="male">Male</option>
  <option value="female">Female</option>
  <option value="other">Other</option>
</select><br>
<select name="programme" required>
  <option value="BIT">BIT</option>
  <option value="BCA">BCA</option>
</select><br>
<input type="number" name="semester" min="1" max="8" value="1"><br>
<button type="submit">Save</button>
</form>
<p><a href="read.php">View all students</a></p>
</body>
</html>

```

(b) READ — displaying all records

```

PHP
<?php
/* ===== file: read.php ===== */
require_once "db.php";

/* optional search term coming from the URL */
$search = $_GET["q"] ?? "";

if ($search !== "") {
    $sql = "SELECT * FROM students
          WHERE fullname LIKE ? OR email LIKE ?
          ORDER BY id DESC";
    $like = "%" . $search . "%";
    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, "ss", $like, $like);
    mysqli_stmt_execute($stmt);
    $result = mysqli_stmt_get_result($stmt);
} else {
    $result = mysqli_query($conn, "SELECT * FROM students
                              ORDER BY id DESC");
}
?>

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Student List</title>
  <style>
    body { font-family: Arial, sans-serif; padding: 24px; }
    table { border-collapse: collapse; width: 100%; }
    th, td { border: 1px solid #ccc; padding: 9px; text-align: left; }
    th { background: #14456b; color: #fff; }
    tr:nth-child(even) { background: #f2f6fa; }
    a.btn { padding: 4px 10px; text-decoration: none; border-radius: 4px;
            color: #fff; }
    .edit { background: #1f6091; } .del { background: #c0392b; }
  </style>
</head>
<body>

```

```

<h2>Registered Students</h2>

<form method="get">
  <input type="text" name="q" placeholder="Search by name or email"
    value="<?= htmlspecialchars($search) ?>">
  <button type="submit">Search</button>
  <a href="read.php">Clear</a>
</form>

<p>Total records: <?= mysqli_num_rows($result) ?></p>

<table>
  <tr>
    <th>ID</th><th>Name</th><th>Email</th><th>Phone</th>
    <th>Programme</th><th>Sem</th><th>Registered</th><th>Action</th>
  </tr>

  <?php if (mysqli_num_rows($result) > 0): ?>
  <?php while ($row = mysqli_fetch_assoc($result)): ?>
    <tr>
      <td><?= $row["id"] ?></td>
      <td><?= htmlspecialchars($row["fullname"]) ?></td>
      <td><?= htmlspecialchars($row["email"]) ?></td>
      <td><?= htmlspecialchars($row["phone"]) ?></td>
      <td><?= htmlspecialchars($row["programme"]) ?></td>
      <td><?= $row["semester"] ?></td>
      <td><?= date("d M Y", strtotime($row["created_at"])) ?></td>
      <td>
        <a class="btn edit" href="update.php?id=<?= $row['id'] ?>">
          Edit</a>
        <a class="btn del" href="delete.php?id=<?= $row['id'] ?>"
          onclick="return confirm('Delete this record?')">Delete</a>
      </td>
    </tr>
  <?php endwhile; ?>
  <?php else: ?>
    <tr><td colspan="8">No records found.</td></tr>
  <?php endif; ?>

</table>
<p><a href="create.php">Add a new student</a></p>

<?php mysqli_close($conn); ?>
</body>
</html>

```

Ways of fetching a result set

Function	Returns
<code>mysqli_fetch_assoc(\$result)</code>	One row as an associative array: <code>\$row["fullname"]</code>
<code>mysqli_fetch_array(\$result)</code>	One row as both associative and numeric
<code>mysqli_fetch_row(\$result)</code>	One row as a numeric array: <code>\$row[0]</code>
<code>mysqli_fetch_object(\$result)</code>	One row as an object: <code>\$row->fullname</code>
<code>mysqli_fetch_all(\$result, MYSQLI_ASSOC)</code>	Every row at once as an array of arrays

Function	Returns
<code>mysqli_num_rows(\$result)</code>	The number of rows returned
<code>mysqli_affected_rows(\$conn)</code>	The number of rows changed by INSERT, UPDATE or DELETE
<code>mysqli_insert_id(\$conn)</code>	The auto-increment id of the row just inserted

(c) UPDATE — editing an existing record

PHP

```
<?php
/* ===== file: update.php ===== */
require_once "db.php";

$message = "";
$id = (int) ($_GET["id"] ?? $_POST["id"] ?? 0);
if ($id <= 0) { die("Invalid record id."); }

/* ----- handle the submitted changes ----- */
if ($_SERVER["REQUEST_METHOD"] === "POST") {

    $name    = trim($_POST["fullname"]);
    $email    = trim($_POST["email"]);
    $phone    = trim($_POST["phone"]);
    $programme = $_POST["programme"];
    $semester = (int) $_POST["semester"];

    $sql = "UPDATE students
            SET fullname = ?, email = ?, phone = ?,
              programme = ?, semester = ?
            WHERE id = ?";

    $stmt = mysqli_prepare($conn, $sql);
    mysqli_stmt_bind_param($stmt, "ssssi", $name, $email, $phone,
                             $programme, $semester, $id);

    if (mysqli_stmt_execute($stmt)) {
        header("Location: read.php?updated=1"); // redirect
        exit;
    } else {
        $message = "Update failed: " . mysqli_stmt_error($stmt);
    }
}

/* ----- fetch the current values for the form ----- */
$stmt = mysqli_prepare($conn, "SELECT * FROM students WHERE id = ?");
mysqli_stmt_bind_param($stmt, "i", $id);
mysqli_stmt_execute($stmt);
$student = mysqli_fetch_assoc(mysqli_stmt_get_result($stmt));

if (!$student) { die("Record not found."); }
?>

<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Edit Student</title></head>
<body>
    <h2>Edit Student #<?= $student["id"] ?></h2>
```

```

<?php if ($message): ?><p><?= $message ?></p><?php endif; ?>

<form method="post">
  <input type="hidden" name="id" value="<?= $student["id"] ?>">

  <input type="text" name="fullname"
    value="<?= htmlspecialchars($student["fullname"]) ?>"
    required><br>
  <input type="email" name="email"
    value="<?= htmlspecialchars($student["email"]) ?>"
    required><br>
  <input type="tel" name="phone"
    value="<?= htmlspecialchars($student["phone"]) ?>"><br>

  <select name="programme">
    <option value="BIT" <?= $student["programme"]=="BIT"
      ? "selected" : "" ?>>BIT</option>
    <option value="BCA" <?= $student["programme"]=="BCA"
      ? "selected" : "" ?>>BCA</option>
  </select><br>

  <input type="number" name="semester" min="1" max="8"
    value="<?= $student["semester"] ?>"><br>

  <button type="submit">Update</button>
  <a href="read.php">Cancel</a>
</form>
</body>
</html>

```

(d) **DELETE** — removing a record

```

PHP
<?php
/* ===== file: delete.php ===== */
require_once "db.php";

$id = (int) ($_GET["id"] ?? 0);

if ($id <= 0) {
  die("Invalid record id.");
}

$stmt = mysqli_prepare($conn, "DELETE FROM students WHERE id = ?");
mysqli_stmt_bind_param($stmt, "i", $id);

if (mysqli_stmt_execute($stmt)) {
  if (mysqli_stmt_affected_rows($stmt) > 0) {
    header("Location: read.php?deleted=1");
  } else {
    header("Location: read.php?notfound=1");
  }
  exit;
} else {
  echo "Delete failed: " . mysqli_stmt_error($stmt);
}

mysqli_stmt_close($stmt);
mysqli_close($conn);

```

```
?>
```

5.7.4 SQL Injection and Prepared Statements

The problem

SQL injection is an attack in which a user types SQL fragments into a form field, and those fragments become part of the query the server executes. It is one of the most damaging and most common web vulnerabilities.

```
PHP
<?php
/* ===== VULNERABLE CODE — NEVER WRITE THIS ===== */
$user = $_POST["username"];
$pass = $_POST["password"];

$sql = "SELECT * FROM users
      WHERE username = '$user' AND password = '$pass'";

/* If the attacker types this into the username box:

    admin' --

the query the database actually receives becomes:

    SELECT * FROM users
    WHERE username = 'admin' -- ' AND password = "

Everything after -- is a comment, so the password check
disappears entirely and the attacker is logged in as admin.

Typing ' OR '1'='1' returns EVERY row in the table. */
?>
```

The solution — prepared statements

A **prepared statement** sends the SQL structure and the data to the database **separately**. The database compiles the query first, then treats the supplied values purely as data. They can never be interpreted as SQL commands, so injection becomes impossible.

```
PHP
<?php
/* ===== SAFE CODE — always write this ===== */
$sql = "SELECT * FROM users WHERE username = ? AND password = ?";
$stmt = mysqli_prepare($conn, $sql);
mysqli_stmt_bind_param($stmt, "ss", $user, $pass);
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);

/* Now the input admin' -- is searched for LITERALLY as a
   username. No row matches, and the login correctly fails. */

/* The same thing with PDO and named parameters */
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = :u");
```

```
$stmt->execute(["u" => $user]);
$row = $stmt->fetch();
?>
```

Type letter	Meaning
s	string
i	integer
d	double / float
b	blob (binary data)

Note: The number of type letters must exactly equal the number of ? placeholders and the number of variables that follow, and they must be in the same order. "ssi" means: first two values are strings, the third is an integer.

5.8 Cookies and Session Tracking

5.8.1 Why State Management is Needed

HTTP is a **stateless** protocol: each request is independent and the server has no memory of who sent the previous one. Without help, a user would be forgotten the moment they moved to the next page. **Cookies** and **sessions** are the two mechanisms that add state, and they make login systems and shopping carts possible.

Figure 5.3 How cookies and sessions differ

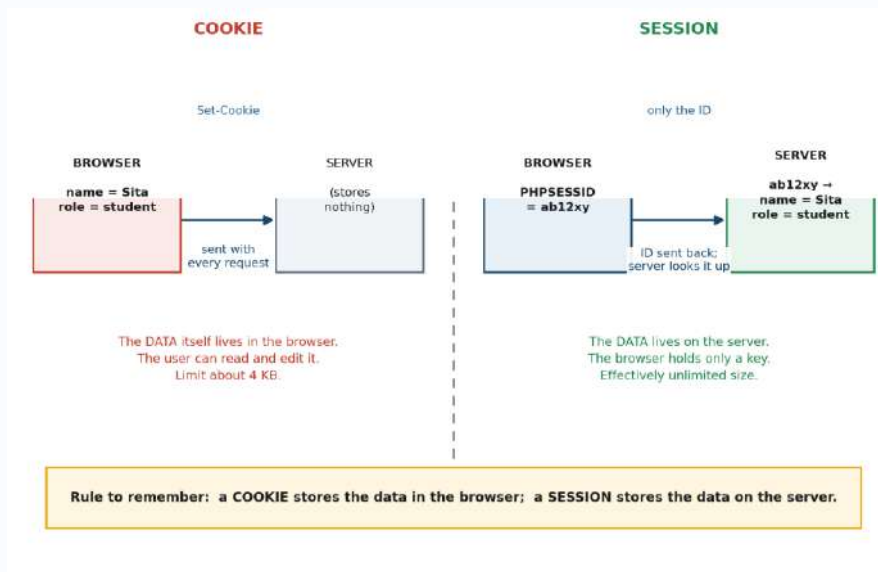


Figure 5.3 — A cookie stores the data itself; a session stores only a key to data held on the server.

5.8.2 Cookies

Definition

A **cookie** is a small text file (up to about 4 KB) that the server asks the browser to store on the visitor's computer. The browser sends it back with every subsequent request to the same site, allowing the server to recognise a returning visitor.

Syntax

SYNTAX

```
setcookie(name, value, expire, path, domain, secure, httponly);
```

PHP

```
<?php
/* setcookie() must be called BEFORE any HTML output */

// expires in 30 days
setcookie("username", "Sita", time() + (86400 * 30), "");

// a secure cookie, unreadable by JavaScript
setcookie("theme", "dark", time() + 3600, "/", "", true, true);

/* ----- reading a cookie ----- */
if (isset($_COOKIE["username"])) {
    echo "Welcome back, " . htmlspecialchars($_COOKIE["username"]);
} else {
    echo "Welcome, new visitor!";
}

/* ----- deleting a cookie: set a past expiry time ----- */
setcookie("username", "", time() - 3600, "/");

/* ----- a practical visit counter ----- */
$count = isset($_COOKIE["visits"]) ? (int)$_COOKIE["visits"] + 1 : 1;
setcookie("visits", $count, time() + (86400 * 365), "/");
echo "You have visited this page $count time(s).";
?>
```

Parameter	Purpose
name	The name of the cookie
value	The data stored (never store a password)
expire	A Unix timestamp. <code>time()+3600</code> is one hour from now. If omitted or 0, the cookie is deleted when the browser closes.
path	<code>"/"</code> makes it available across the whole site; <code>"/admin/"</code> restricts it.
domain	The domain that may read it, e.g. <code>".example.com"</code> for all subdomains.
secure	true means the cookie is sent only over HTTPS.
httponly	true means JavaScript cannot read it — an important defence against XSS.

5.8.3 Sessions

Definition

A **session** stores user data **on the server** and gives the browser only a unique session identifier, normally in a cookie named **PHPSESSID**. Because the data never leaves the server, sessions are far more secure than cookies and are the correct mechanism for login systems.

```

PHP
<?php
/* session_start() must be the FIRST thing in the file,
   before any HTML, blank line or space.          */
session_start();

/* ----- storing data ----- */
$_SESSION["user_id"] = 1;
$_SESSION["username"] = "Sita Gurung";
$_SESSION["role"] = "student";
$_SESSION["login_time"] = time();

/* ----- reading data on any other page ----- */
if (isset($_SESSION["username"])) {
    echo "Hello, " . htmlspecialchars($_SESSION["username"]);
}

/* ----- removing one value ----- */
unset($_SESSION["role"]);

/* ----- destroying the whole session (logout) ----- */
session_unset(); // clear all the variables
session_destroy(); // destroy the session file on the server
?>

```

Complete login system using sessions

```

PHP
<?php
/* ===== file: login.php ===== */
session_start();
require_once "db.php";

$error = "";

/* already logged in? go straight to the dashboard */
if (isset($_SESSION["user_id"])) {
    header("Location: dashboard.php");
    exit;
}

if ($_SERVER["REQUEST_METHOD"] === "POST") {

    $email = trim($_POST["email"]);
    $password = $_POST["password"];

    $stmt = mysqli_prepare($conn,
        "SELECT id, fullname, password FROM students
        WHERE email = ?");
    mysqli_stmt_bind_param($stmt, "s", $email);

```

```

mysqli_stmt_execute($stmt);
$user = mysqli_fetch_assoc(mysqli_stmt_get_result($stmt));

if ($user && password_verify($password, $user["password"])) {

    // guard against session fixation attacks
    session_regenerate_id(true);

    $_SESSION["user_id"] = $user["id"];
    $_SESSION["username"] = $user["fullname"];
    $_SESSION["login_time"] = time();

    header("Location: dashboard.php");
    exit;
} else {
    // one generic message: never reveal WHICH part was wrong
    $error = "Invalid email or password.";
}
}
?>
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Login</title></head>
<body>
<h2>Student Login</h2>
<?php if ($error): ?>
    <p style="color:#c0392b"><?=$error ?></p>
<?php endif; ?>
<form method="post">
    <input type="email" name="email" placeholder="Email"
        required><br>
    <input type="password" name="password" placeholder="Password"
        required><br>
    <button type="submit">Log in</button>
</form>
</body>
</html>

PHP
<?php
/* ===== file: dashboard.php ===== */
session_start();

/* ----- protect the page ----- */
if (!isset($_SESSION["user_id"])) {
    header("Location: login.php");
    exit;
}

/* ----- automatic timeout after 30 minutes ----- */
if (time() - $_SESSION["login_time"] > 1800) {
    session_unset();
    session_destroy();
    header("Location: login.php?timeout=1");
    exit;
}
$_SESSION["login_time"] = time(); // refresh on activity
?>
<h2>Welcome, <?= htmlspecialchars($_SESSION["username"]) ?></h2>
<p><a href="logout.php">Log out</a></p>

PHP

```

```

<?php
/* ===== file: logout.php ===== */
session_start();
session_unset();
session_destroy();

// also remove the session cookie from the browser
if (ini_get("session.use_cookies")) {
    setcookie(session_name(), "", time() - 3600, "/");
}

header("Location: login.php?loggedout=1");
exit;
?>

```

Difference between cookies and sessions

Basis	Cookie	Session
Storage location	On the client (the browser)	On the server
Data visible to the user	Yes — it can be read and edited	No — only the session ID is exposed
Security	Less secure; never store sensitive data	More secure; suitable for login information
Capacity	About 4 KB per cookie	Effectively unlimited (limited by server memory or disk)
Lifetime	Set explicitly; can survive for months	Ends when the browser closes or the session expires
Dependence on the browser	Fails if the user blocks cookies	Also uses a cookie for the ID, but can fall back to a URL parameter
Server resources	None	Consumes storage on the server
PHP superglobal	<code>\$_COOKIE</code>	<code>\$_SESSION</code>
Set with	<code>setcookie()</code>	<code>\$_SESSION["key"] = value</code> after <code>session_start()</code>
Typical use	"Remember me", language preference, theme, analytics	Login state, shopping cart, temporary form data

Exam Tip: The examiner's favourite one-liner: **a cookie stores the data in the browser, a session stores the data on the server and only the identifier in the browser.**

5.8.4 Putting It All Together — the Complete Application

Figure 5.4 How the five chapters build one application

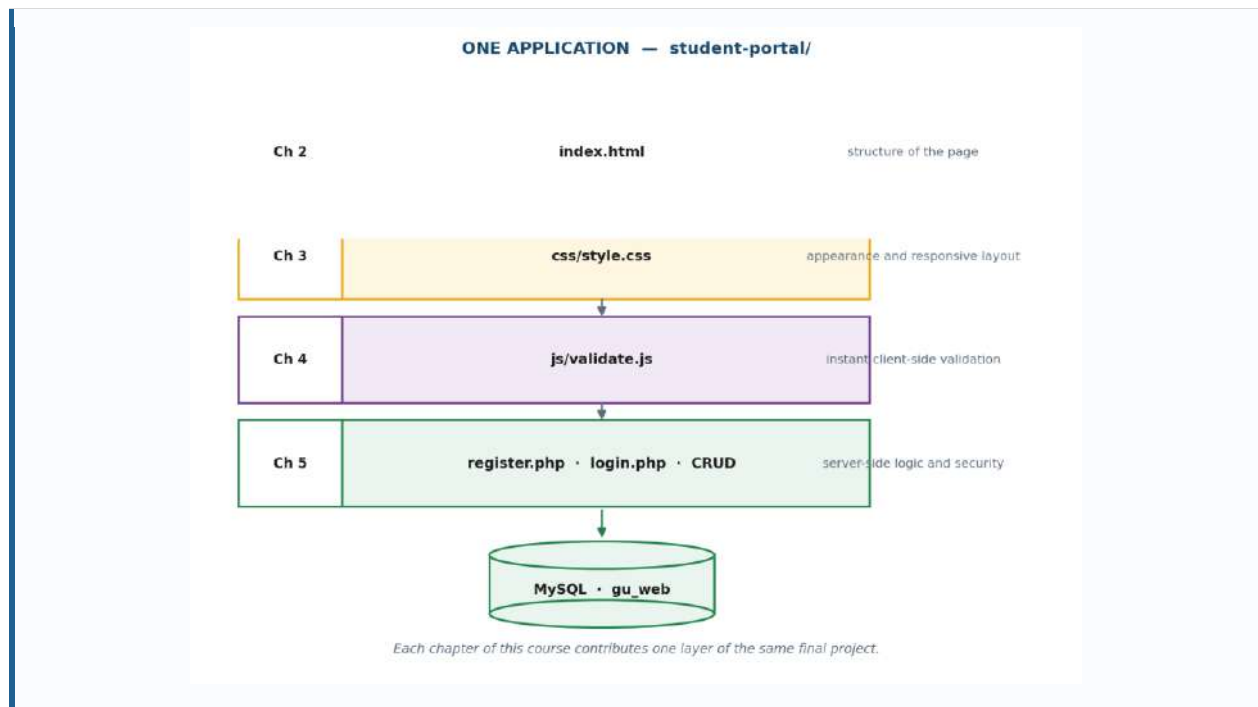


Figure 5.4 — The complete project structure for the laboratory work.

5.9 Exam-Based Questions — Chapter 5

Group A — Very Short Answer Questions (2 marks each)

1. Define PHP. Write its full form and who developed it.
2. Differentiate between client-side and server-side scripting.
3. Why can a visitor never see PHP source code in the browser?
4. Write any two rules for writing a PHP program.
5. Differentiate between `echo` and `print`.
6. Differentiate between single quotes and double quotes in PHP.
7. Write the operator used for string concatenation in PHP and give an example.
8. Differentiate between `include` and `require`.
9. What is an associative array? Give an example.
10. Differentiate between `$_GET` and `$_POST`.
11. What are superglobal variables? Write any four.
12. What is the purpose of `htmlspecialchars()`?
13. Differentiate between `password_hash()` and `password_verify()`.
14. Write the syntax of `fopen()` and explain the modes `r`, `w` and `a`.
15. What is a primary key? How does it differ from a foreign key?
16. What does CRUD stand for? Name the SQL statement for each operation.
17. Write the PHP code to connect to a MySQL database named `gu_web`.
18. What is SQL injection?
19. What is a prepared statement? Why is it used?

20. Differentiate between a cookie and a session.
21. Write the syntax of `setcookie()` and explain its parameters.
22. Why must `session_start()` be written before any HTML output?

Group B — Short Answer Questions (5 marks each)

1. What is PHP? Explain its features and any five uses with a diagram of how a PHP page is processed.
2. Differentiate between client-side and server-side scripting with at least six points.
3. Explain the basic rules of writing a PHP program and write a program that displays the current date and a greeting based on the time of day.
4. Explain variables, constants and data types in PHP with examples.
5. Explain any eight string functions of PHP with syntax and examples.
6. Explain the categories of operator in PHP. Why must `.` be used instead of `+` for joining strings?
7. Explain conditional statements in PHP and write a program to display the grade of a student from marks.
8. Explain the looping statements of PHP with syntax and write a program to display a multiplication table as an HTML table.
9. Explain the three types of array in PHP with examples and any six array functions.
10. Explain functions in PHP. Differentiate between passing by value and passing by reference with a code example.
11. Explain how form data is received in PHP. Differentiate between `$_GET`, `$_POST` and `$_REQUEST`.
12. Write a PHP program to receive a name, an e-mail address and a phone number from a form, validate them on the server, and display an appropriate message.
13. Explain file handling in PHP. Describe the file modes and write a program to write to and then read from a text file.
14. Explain the file-upload process in PHP with complete code, and describe the security checks that must be performed.
15. Explain the terms database, table, record, field, primary key and foreign key with a diagram.
16. Write the SQL statements to create a database, create a table with at least six fields, and insert, select, update and delete a record.
17. Write a PHP program to connect to a MySQL database and insert a record using a prepared statement.
18. Write a PHP program to fetch all records from a table and display them in an HTML table.
19. What is SQL injection? Explain with an example how it works and how prepared statements prevent it.
20. Explain cookies in PHP. Write a program to create, read and delete a cookie.
21. Explain sessions in PHP and write a complete login and logout program using sessions.
22. Differentiate between cookies and sessions with at least eight points.

Group C — Long Answer Questions (7.5 marks each)

1. What is PHP? Explain its place in the web world with a diagram, describe its features, uses and basic rules, and write a complete PHP program that demonstrates variables, an array, a conditional statement, a loop and a function.

2. Explain arrays in PHP in complete detail. Describe indexed, associative and multidimensional arrays, explain at least ten array functions, and write a program that stores the marks of five students and displays a mark sheet with the total, average, highest and lowest, sorted by total.
3. Explain how PHP processes data submitted from an HTML form. Differentiate between GET and POST, explain the superglobals, and write a complete program consisting of an HTML registration form and a PHP script that validates every field on the server and displays the result.
4. Explain file handling in PHP in detail. Describe all the file modes and the main functions, explain the complete file-upload process with its security checks, and write a program that uploads an image, validates its type and size, and stores it with a unique name.
5. Explain how data is organised in a relational database. Define database, table, record, field, primary key and foreign key with a diagram, describe the main MySQL data types, and write the SQL to create a database with two related tables and a JOIN query across them.
6. Explain the complete CRUD operation in PHP and MySQL. Write the code for connecting to the database, inserting, selecting, updating and deleting records using prepared statements, and explain how the four files work together.
7. What is SQL injection? Explain the attack in detail with an example of vulnerable code and the exact input that breaks it. Explain how prepared statements work internally, rewrite the vulnerable code safely, and describe at least four other measures that protect a PHP application.
8. Explain state management in PHP. Explain why HTTP is stateless, describe cookies and sessions with diagrams, compare them in a table, and write a complete login system with session timeout and logout.

Laboratory / Practical Exercises — Chapter 5

1. Install XAMPP, start Apache and MySQL, and run a PHP page that displays `phpinfo()`. Submit a screenshot.
2. Write PHP programs to: find the factorial of a number, check whether a number is prime, reverse a string, and count the vowels in a string.
3. Write a PHP program that stores the marks of five students in a multidimensional array and displays a formatted mark sheet with totals, averages and grades.
4. Connect the registration form of Chapter 2 to a PHP script that validates every field on the server and displays the submitted data.
5. Create a text file through PHP, write several records to it, then read the file and display its contents in an HTML table.
6. Build a file-upload page that accepts only JPG and PNG images under 2 MB, renames the file uniquely and displays the uploaded image.
7. Create the database `gu_web` and the table `students` using phpMyAdmin, then write PHP code to insert a record into it.
8. Implement the complete CRUD application: `create.php`, `read.php` with a search box, `update.php` and `delete.php`, all using prepared statements.
9. Demonstrate SQL injection on a deliberately vulnerable login page, then fix it with a prepared statement and show that the same input no longer works.
10. Build a complete login system with registration, hashed passwords, session-based authentication, a protected dashboard, a session timeout and logout.

11. Create a page that remembers the visitor's preferred theme (light or dark) using a cookie and counts how many times they have visited.
12. **Final project.** Combine every chapter into one web application: an HTML structure, an external CSS style sheet, JavaScript validation, PHP server-side processing and a MySQL database with full CRUD, login and file upload. Submit the source code, the database dump and a short report.

Appendix A: Quick Reference and Revision Material

Note: Use this appendix in the last week before the examination. Everything here is a condensed version of material already explained in full in the chapters.

A.1 Most Frequently Asked Comparison Questions

These comparisons appear in almost every examination paper. Learn at least five points for each.

#	Comparison	Chapter	The one-line answer to open with
1	Internet vs WWW	1	The Internet is the physical network; the WWW is a service running on it.
2	HTTP vs HTTPS	1	HTTPS is HTTP wrapped in TLS encryption, on port 443 instead of 80.
3	GET vs POST	1, 2	GET puts data in the URL; POST puts it in the request body.
4	Static vs dynamic website	1	A static site returns a stored file; a dynamic site builds the page for each request.
5	URL vs URI vs URN	1	A URI identifies; a URL identifies and locates; a URN names without locating.
6	HTML vs XHTML	2	HTML is based on SGML and is forgiving; XHTML is based on XML and is strict.
7	HTML4 vs HTML5	2	HTML5 added semantic elements, native media and new form types.
8	Block vs inline elements	2	A block element starts on a new line and takes the full width.
9	<code></code> vs <code></code>	2	<code></code> is presentational; <code></code> carries meaning.
10	<code>colspan</code> vs <code>rowspan</code>	2	<code>colspan</code> merges horizontally; <code>rowspan</code> merges vertically.
11	Radio buttons vs check boxes	2	Radio buttons allow one choice; check boxes allow many.
12	Inline vs internal vs external CSS	3	Scope of one element, one page, or many pages.
13	Class vs ID selector	3	A class may repeat; an ID must be unique.
14	<code>relative</code> vs <code>absolute</code> positioning	3	Relative keeps its original space; absolute is removed from the flow.
15	Flexbox vs Grid	3	Flexbox is one-dimensional; Grid is two-dimensional.
16	<code>display:none</code> vs <code>visibility:hidden</code>	3	The first removes the space; the second leaves it empty.
17	Java vs JavaScript	4	Unrelated languages; only the name was shared for marketing.
18	<code>var</code> vs <code>let</code> vs <code>const</code>	4	Function scope, block scope, block scope without reassignment.
19	<code>==</code> vs <code>===</code>	4	<code>===</code> also compares the data type.
20	<code>innerHTML</code> vs <code>textContent</code>	4	The first parses HTML; the second treats it as literal text.
21	Bubbling vs capturing	4	Bubbling goes up the tree; capturing comes down it.
22	Client-side vs server-side validation	4, 5	One is convenience, the other is security.

#	Comparison	Chapter	The one-line answer to open with
23	Client-side vs server-side scripting	5	JavaScript runs in the browser; PHP runs on the server.
24	include vs require	5	require stops the script; include only warns.
25	MySQLi vs PDO	5	MySQLi is MySQL only; PDO supports twelve databases.
26	Cookies vs sessions	5	A cookie stores data in the browser; a session stores it on the server.
27	Primary key vs foreign key	5	A primary key identifies a row; a foreign key points to one in another table.

A.2 HTML Quick Reference

HTML

```

<!-- STRUCTURE -->
<!DOCTYPE html> <html lang> <head> <meta charset> <title> <body>

<!-- SEMANTIC -->
<header> <nav> <main> <section> <article> <aside> <footer>
<figure> <figcaption> <time> <mark>

<!-- TEXT -->
<h1>..

###### 


```

A.3 CSS Quick Reference

CSS

```

/* SELECTORS */
* element .class #id s1, s2
A B (descendant) A > B (child) A + B (adjacent) A ~ B (sibling)
[attr] [attr="v"] [attr^="v"] [attr$="v"] [attr*="v"]
:link :visited :hover :focus :active :first-child :last-child
:nth-child(n) :checked :disabled :required :valid :invalid :not()
::before ::after ::first-line ::first-letter ::selection ::placeholder

/* BOX MODEL */
width height padding border margin box-sizing; border-box;
margin: T R B L; margin: 0 auto; border-radius; box-shadow;

/* TEXT */
color font-family font-size font-weight font-style line-height
text-align text-decoration text-transform text-indent
letter-spacing word-spacing text-shadow white-space

/* BACKGROUND */
background-color -image -repeat -position -size -attachment
linear-gradient() radial-gradient()

/* DISPLAY AND POSITION */
display: block | inline | inline-block | none | flex | grid
position: static | relative | absolute | fixed | sticky
top right bottom left z-index float clear overflow visibility

/* FLEXBOX */
display: flex; flex-direction; flex-wrap; justify-content; align-items;
align-content; gap; on items: flex; flex-grow; flex-shrink; order;

/* GRID */
display: grid; grid-template-columns; grid-template-rows;
grid-template-areas; gap; on items: grid-area; grid-column; grid-row;
repeat(auto-fit, minmax(220px, 1fr))

/* RESPONSIVE */
@media screen and (min-width: 768px) { ... }
@media print { ... }
units: px % em rem vw vh vmin vmax ch

/* EFFECTS */
transition: property duration timing; transform: translate/scale/rotate;
@keyframes name { from{} to{} } animation: name duration;

/* VARIABLES */
:root { --brand: #14456b; } color: var(--brand);

```

A.4 JavaScript Quick Reference

```

JAVASCRIPT
// VARIABLES AND TYPES
let const var
number string boolean undefined null bigint symbol object array function
typeof x Array.isArray(a) Number() parseInt() String()

// OPERATORS
+ - * / % **    = += -= *= /=    ++ --
== === != !== > < >= <=    && || ! ? : ??

```

// CONTROL

```
if / else if / else    switch(x){case v: break; default;}
for(;;) while do..while for..of (values) for..in (keys)
break continue
```

// FUNCTIONS

```
function f(a, b = 1) { return a; }
const f = function(a){};    const f = (a) => a * 2;
function f(...args) {}      // rest parameters
```

// STRINGS

```
length toUpperCase toLowerCase indexOf slice substring replace
trim split includes startsWith charAt `template ${literal}`
```

// ARRAYS

```
push pop shift unshift slice splice concat join reverse sort
indexOf includes forEach map filter reduce find some every
```

// DOM SELECTION

```
document.getElementById(id)
document.querySelector(css)    document.querySelectorAll(css)
document.getElementsByClassName() getElementsByTagName() ByName()
```

// DOM MANIPULATION

```
el.innerHTML el.textContent el.value el.style.property
el.setAttribute() getAttribute() removeAttribute()
el.classList.add / remove / toggle / contains
document.createElement() el.appendChild() el.remove()
```

// EVENTS

```
el.addEventListener("click", fn) el.removeEventListener("click", fn)
events: click dblclick mouseover mouseout keydown keyup
        submit reset change input focus blur load DOMContentLoaded
event.target .preventDefault() .stopPropagation() .key
```

// TIMERS

```
setTimeout(fn, ms) setInterval(fn, ms) clearInterval(id)
```

// VALIDATION PATTERNS

```
 /^[A-Za-z ]+$/      name
 /^[^s@]+@[^\s@]+\.[a-zA-Z]{2,}$/ email
 /^[0-9]{10}$/      phone
 pattern.test(string) -> true / false
```

A.5 PHP and MySQL Quick Reference

PHP

```
<?php
```

// BASICS

```
$var = value; echo print define() const // # /* */
gettype() var_dump() print_r() isset() empty() unset()
```

// STRINGS

```
strlen strtoupper strtolower ucfirst ucwords trim str_replace
substr strpos explode implode strev nl2br sprintf htmlspecialchars
concatenation uses . and .=
```

// NUMBERS

abs round ceil floor sqrt pow max min rand number_format is_numeric

// ARRAYS

\$a = [1,2,3]; \$b = ["k" => "v"];
 count in_array array_search array_keys array_values array_merge
 array_push array_pop sort rsort asort ksort array_sum array_map
 array_filter array_unique foreach (\$a as \$k => \$v) {}

// CONTROL

if elseif else switch match for while do..while foreach
 alternative syntax: if(): endif; foreach(): endforeach;
 include require include_once require_once

// SUPERGLOBALS

\$_GET \$_POST \$_REQUEST \$_FILES \$_SESSION \$_COOKIE \$_SERVER \$GLOBALS

// FILES

fopen fread fgets fwrite fclose feof modes: r r+ w w+ a a+ x
 file_get_contents file_put_contents file_exists unlink
 move_uploaded_file mime_content_type pathinfo

// SECURITY

htmlspecialchars() filter_var(\$v, FILTER_VALIDATE_EMAIL)
 password_hash(\$p, PASSWORD_DEFAULT) password_verify(\$p, \$hash)

// DATABASE

\$conn = mysqli_connect(\$host,\$user,\$pass,\$db);
 \$stmt = mysqli_prepare(\$conn, "SELECT * FROM t WHERE id = ?");
 mysqli_stmt_bind_param(\$stmt, "i", \$id); // s i d b
 mysqli_stmt_execute(\$stmt);
 \$res = mysqli_stmt_get_result(\$stmt);
 while (\$row = mysqli_fetch_assoc(\$res)) { echo \$row["col"]; }
 mysqli_num_rows mysqli_insert_id mysqli_affected_rows mysqli_close

// SESSIONS AND COOKIES

session_start(); \$_SESSION["k"]=v; session_unset(); session_destroy();
 setcookie(name, value, time()+3600, "/"); \$_COOKIE["name"]
 ?>

SQL**-- SQL**

CREATE DATABASE db; USE db;
 CREATE TABLE t (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100));
 INSERT INTO t (name) VALUES ("x");
 SELECT * FROM t WHERE c = v ORDER BY c ASC LIMIT 10;
 UPDATE t SET c = v WHERE id = 1;
 DELETE FROM t WHERE id = 1;
 ALTER TABLE t ADD c VARCHAR(50);
 SELECT a.x, b.y FROM a INNER JOIN b ON a.id = b.a_id;
 operators: = != > < >= <= LIKE IN BETWEEN AND OR NOT
 functions: COUNT() SUM() AVG() MAX() MIN() GROUP BY HAVING

A.6 Model Question Paper

Gandaki University — Faculty of Science and Technology

Bachelor of Information Technology — Semester II

Subject: Web Technology I (CIT123) — Full Marks: 60 — Time: 3 hours

Candidates are required to give their answers in their own words as far as practicable. The figures in the margin indicate full marks.

Group A — Attempt ALL questions. (6 × 2 = 12)

1. Differentiate between the Internet and the World Wide Web.
2. What is a URL? Write its general syntax with an example.
3. Differentiate between `colspan` and `rowspan` with an example of each.
4. Name the three levels of style sheet and write their order of priority.
5. Differentiate between `==` and `===` in JavaScript.
6. What is SQL injection? How is it prevented?

Group B — Attempt any FIVE questions. (5 × 5 = 25)

7. Explain the client-server architecture of the Web with a diagram, and describe the request-response cycle.
8. Differentiate between HTML and XHTML with at least six points.
9. Explain any six CSS selectors with syntax and examples, and explain how specificity resolves conflicts between them.
10. What is the DOM? Draw the DOM tree of a simple HTML document and explain any four methods used to select elements.
11. Explain the three models of event handling in JavaScript with code examples and compare them.
12. Explain the three types of array in PHP with examples, and any five array functions.
13. Differentiate between cookies and sessions, and write a PHP program that creates, reads and deletes a cookie.

Group C — Attempt any THREE questions. (3 × 7.5 = 22.5)

14. Explain HTML forms in detail. Describe the `<form>` element and its attributes, differentiate between GET and POST, explain at least eight input types, and write the complete code for a student registration form using text, password, radio, check box, select, file and hidden controls.
15. Explain the CSS box model with a labelled diagram. Describe each layer, explain the difference between `content-box` and `border-box` with a numerical calculation, and design a responsive card layout using CSS Grid with at least one media query.
16. Explain form validation in complete detail. Differentiate between client-side and server-side validation, and write a complete JavaScript program that validates a registration form for a required name, a valid e-mail address, a ten-digit phone number, a strong password and a selected radio button.
17. Explain the complete CRUD operation using PHP and MySQL. Write the SQL to create a suitable table, and the PHP code to connect to the database and to insert, select, update and delete records using prepared statements.

A.7 Glossary of Key Terms

Term	Definition
AJAX	Asynchronous JavaScript And XML — a technique for exchanging data with the server without reloading the page.
API	Application Programming Interface — a defined way for one program to use the services of another.
Attribute	Extra information written inside an HTML opening tag as a name–value pair.
Bandwidth	The maximum amount of data that can be transferred over a connection in a given time.
Cascade	The algorithm CSS uses to decide which of several conflicting rules applies.
CMS	Content Management System — software such as WordPress for building and managing sites without coding.
Cookie	A small piece of data stored in the browser and sent back to the server with each request.
CRUD	Create, Read, Update, Delete — the four basic data operations.
DNS	Domain Name System — translates a domain name into an IP address.
DOM	Document Object Model — the tree of objects a browser builds from an HTML document.
Element	An opening tag, its content and its closing tag taken together.
Event	An action detected by the browser, such as a click or a key press.
Favicon	The small icon shown on a browser tab.
Framework	A pre-written structure of code, such as Bootstrap, React or Laravel.
FTP	File Transfer Protocol — used to move files between a computer and a server.
Hosting	Renting space on a web server so that a website is reachable on the Internet.
HTTP / HTTPS	The protocol used to transfer web pages; HTTPS adds TLS encryption.
IP address	A numeric address identifying a device on a network.
JSON	JavaScript Object Notation — a lightweight text format for exchanging data.
Localhost	The address <code>127.0.0.1</code> , meaning "this computer" — used for local development.
Metadata	Data about data; in HTML, the information in the <code><head></code> describing the document.
MIME type	A label such as <code>text/html</code> or <code>image/png</code> identifying the kind of a file.
Parsing	Reading source code and converting it into a structure a program can work with.
Pixel	The smallest addressable unit of a display; the basic CSS length unit.
Protocol	An agreed set of rules governing communication between computers.
Query string	The part of a URL after <code>?</code> carrying name–value pairs.
Responsive design	Building one page that adapts its layout to any screen size.
Semantic HTML	Markup chosen for what content means rather than for how it looks.
Session	Server-side storage of data about one visitor, identified by a session ID.
SEO	Search Engine Optimisation — practices that help a page rank well in search results.

Term	Definition
SQL	Structured Query Language — the language used to work with relational databases.
SSL / TLS	Cryptographic protocols that encrypt data travelling between browser and server.
Superglobal	A PHP array such as <code>\$_POST</code> that is available in every scope.
Tag	A keyword in angle brackets that marks the start or end of an HTML element.
URL	Uniform Resource Locator — the complete address of a resource on the Web.
Validation	Checking that data is complete and correctly formatted; also, checking that code follows the standard.
Viewport	The visible area of a web page in the browser window.
Web server	Software or hardware that stores websites and responds to browser requests.
XSS	Cross-Site Scripting — an attack in which malicious script is injected into a page.

A.8 A Suggested Study Plan

Week	Focus	What you should be able to do by the end
1	Chapter 1 and the setup guide	Explain the Internet, WWW, protocols and DNS. XAMPP and VS Code installed and working.
2–3	Chapter 2: HTML structure, text, lists, links, media	Build a three-page website with navigation, images and a gallery.
4	Chapter 2: tables and forms	Build a mark sheet with <code>colspan</code> and <code>rowspan</code> , and a full registration form.
5–6	Chapter 3: CSS selectors, box model, text and colour	Style your whole site from one external style sheet.
7	Chapter 3: positioning, Flexbox, Grid, responsive design	Make the site fully responsive on a phone, tablet and desktop.
8–9	Chapter 4: syntax, control statements, functions	Write programs for grading, tables, patterns and factorials.
10	Chapter 4: DOM and events	Build an interactive gallery and a dynamic style changer.
11	Chapter 4: form validation	Add complete client-side validation to your registration form.
12–13	Chapter 5: PHP syntax, arrays, functions, forms	Process your form on the server and validate every field again.
14	Chapter 5: files and databases	Create the database and insert records from a PHP page.
15	Chapter 5: CRUD, cookies and sessions	Complete the CRUD application and the login system.
16	Revision and the project	Submit the integrated project and work through past papers.

Advice for the examination

- **Draw the diagrams.** A labelled diagram in a 7.5-mark answer earns marks that prose alone will not.

- **Use tables for comparisons.** Every "differentiate between" question should be answered in a table with a clearly named basis column.
- **Write complete, runnable code.** Include the `<!DOCTYPE html>`, the `<html>`, `<head>` and `<body>` elements. Fragments lose marks.
- **Comment your code.** One short comment per logical block shows the examiner you understand what you wrote.
- **Answer in the order of the marks.** Definition first, then explanation, then the example or code, then the diagram.
- **Practise on the computer.** You cannot learn to write code by reading about it. Type every example in this book at least once.

A.9 Recommended References

- Jon Duckett, **Beginning HTML, XHTML, CSS, and JavaScript**, Wiley.
- David Sklar, **Learning PHP 5: A Pain-Free Introduction to Building Interactive Web Sites**, O'Reilly Media.
- Robert W. Sebesta, **Programming the World Wide Web**, Addison-Wesley.
- Luke Welling and Laura Thomson, **PHP and MySQL Web Development**, Addison-Wesley.

Online references — free and authoritative

Resource	Address	Best for
MDN Web Docs	developer.mozilla.org	The definitive reference for HTML, CSS and JavaScript
W3Schools	www.w3schools.com	Quick beginner tutorials with a live editor
W3C Validator	validator.w3.org	Checking that your HTML is valid
W3C CSS Validator	jigsaw.w3.org/css-validator	Checking that your CSS is valid
PHP Manual	www.php.net/manual	The official documentation for every PHP function
MySQL Documentation	dev.mysql.com/doc	The official SQL and MySQL reference
Can I Use	caniuse.com	Checking which browsers support a feature
CSS-Tricks	css-tricks.com	Practical guides to Flexbox and Grid

Note: End of notes. Every example in this book has been written to run without modification. Type them, break them deliberately, and fix them again — that is how web development is actually learnt. Best wishes for your examination and your project.